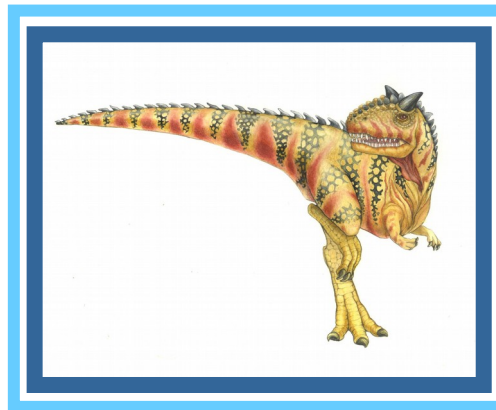


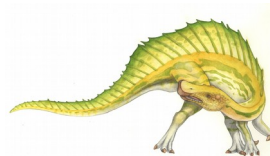
Chapter 1: Introduction





Introduction

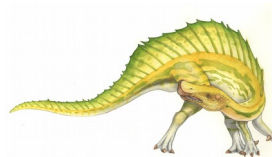
- What Operating Systems Do
- Computer-System Organization
- Computer-System Architecture
- Operating-System Operations
- Resource Management
- Security and Protection
- Virtualization
- Computing Environments
- Free/Libre and Open-Source Operating Systems





Objectives

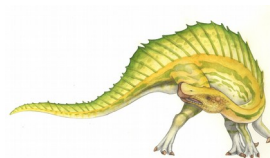
- Describe the general organization of a computer system and the role of interrupts
- Describe the components in a modern, multiprocessor computer system
- Illustrate the transition from user mode to kernel mode
- Discuss how operating systems are used in various computing environments
- Provide examples of free and open-source operating systems





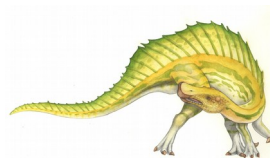
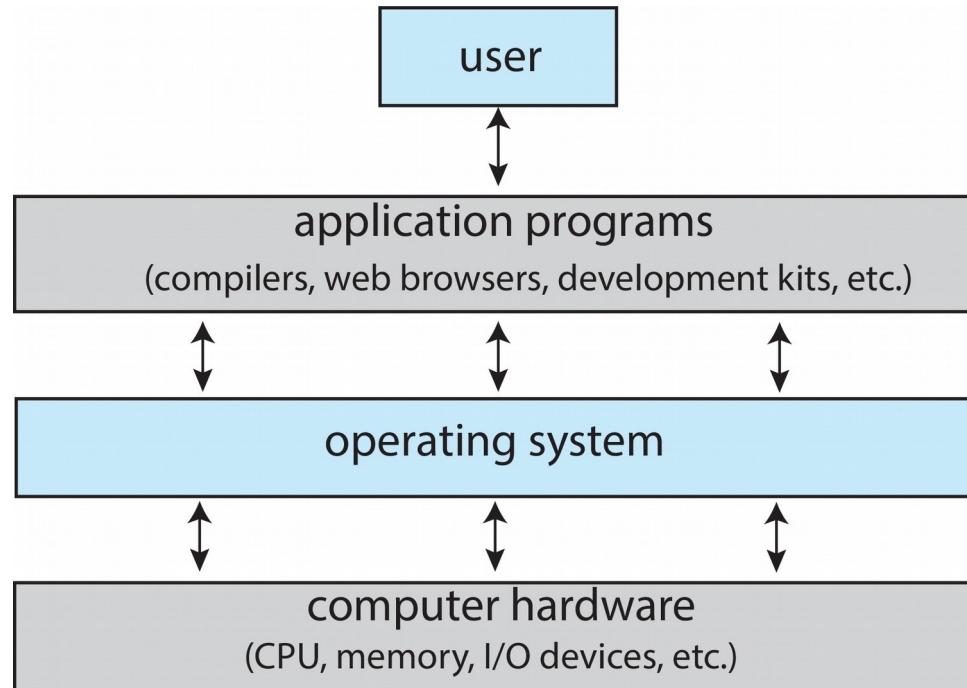
Computer System Structure

- Computer system can be divided into four components:
 - Hardware – provides basic computing resources
 - ▶ CPU, memory, I/O devices
 - Operating system
 - ▶ Controls and coordinates use of hardware among various applications and users
 - Application programs – define the ways in which the system resources are used to solve the computing problems of the users
 - ▶ Word processors, compilers, web browsers, database systems, video games
 - Users
 - ▶ People, machines, other computers





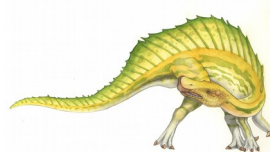
Abstract View of Components of Computer





What Operating Systems Do

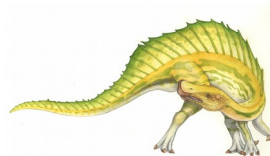
- Depends on the point of view
- Users want convenience, **ease of use** and **good performance**
 - Don't care about **resource utilization**
- But shared computer such as **mainframe** or **minicomputer** must keep all users happy
 - Operating system is a **resource allocator** and **control program** making efficient use of HW and managing execution of user programs
- Users of dedicated systems such as **workstations** have dedicated resources but frequently use shared resources from **servers**
- Mobile devices like smartphones and tables are resource poor, optimized for usability and battery life
 - Mobile user interfaces such as touch screens, voice recognition
- Some computers have little or no user interface, such as embedded computers in devices and automobiles
 - Run primarily without user intervention





Defining Operating Systems

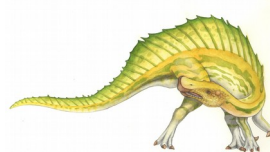
- Term OS covers many roles
 - Because of myriad designs and uses of OSES
 - Present in toasters through ships, spacecraft, game machines, TVs and industrial control systems
 - Born when fixed use computers for military became more general purpose and needed resource management and program control





Operating System Definition (Cont.)

- **No universally accepted definition**
- “Everything a vendor ships when you order an operating system” is a good approximation
 - But varies wildly
- “The one program running at all times on the computer” is the **kernel**, part of the operating system
- Everything else is either
 - a **system program** (ships with the operating system, but not part of the kernel) , or
 - an **application program**, all programs not associated with the operating system
- Today’s OSeS for general purpose and mobile computing also include **middleware** – a set of software frameworks that provide addition services to application developers such as databases, multimedia, graphics

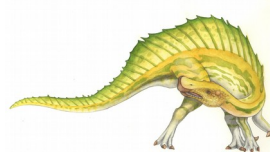
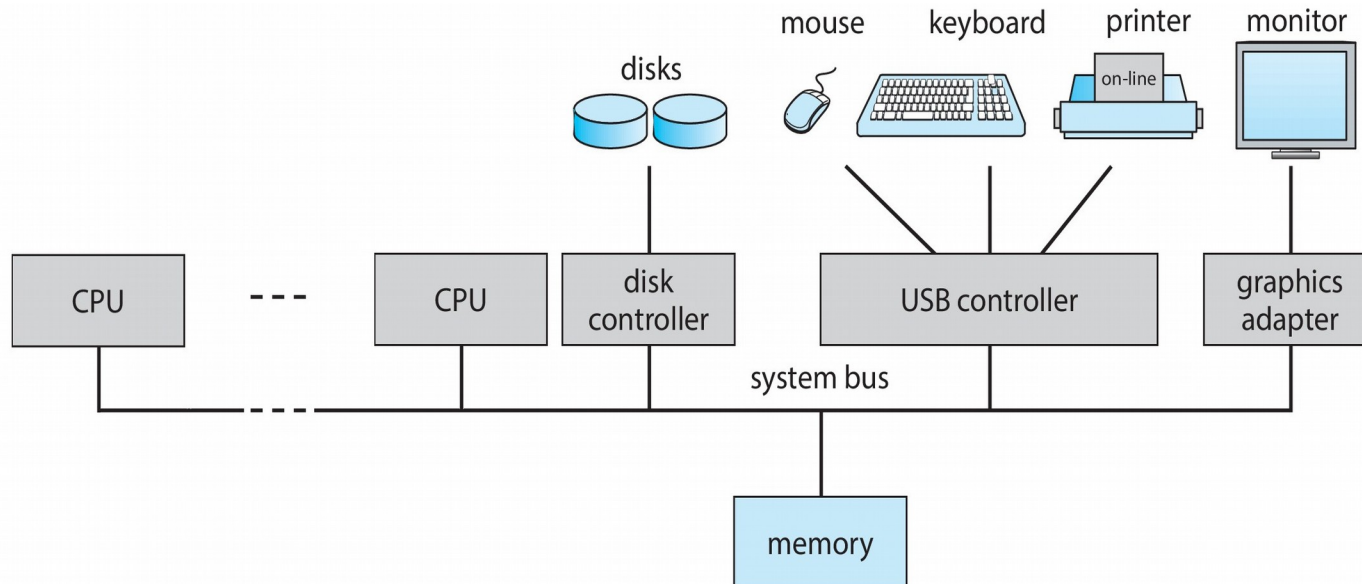




Computer System Organization

■ Computer-system operation

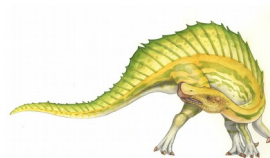
- One or more CPUs, device controllers connect through common **bus** providing access to shared memory
- Concurrent execution of CPUs and devices competing for memory cycles





Computer-System Operation

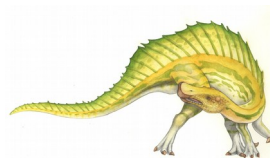
- I/O devices and the CPU can execute concurrently.
- Each device controller is in charge of a particular device type.
- Each device controller has a local buffer.
- Each device controller type has an operating system **device driver** to manage it.
- CPU moves data from/to main memory to/from local buffers.
- I/O is from the device to local buffer of controller
- Device controller informs CPU that it has finished its operation by causing an **interrupt**





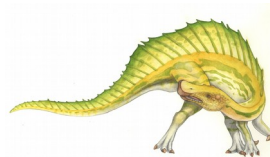
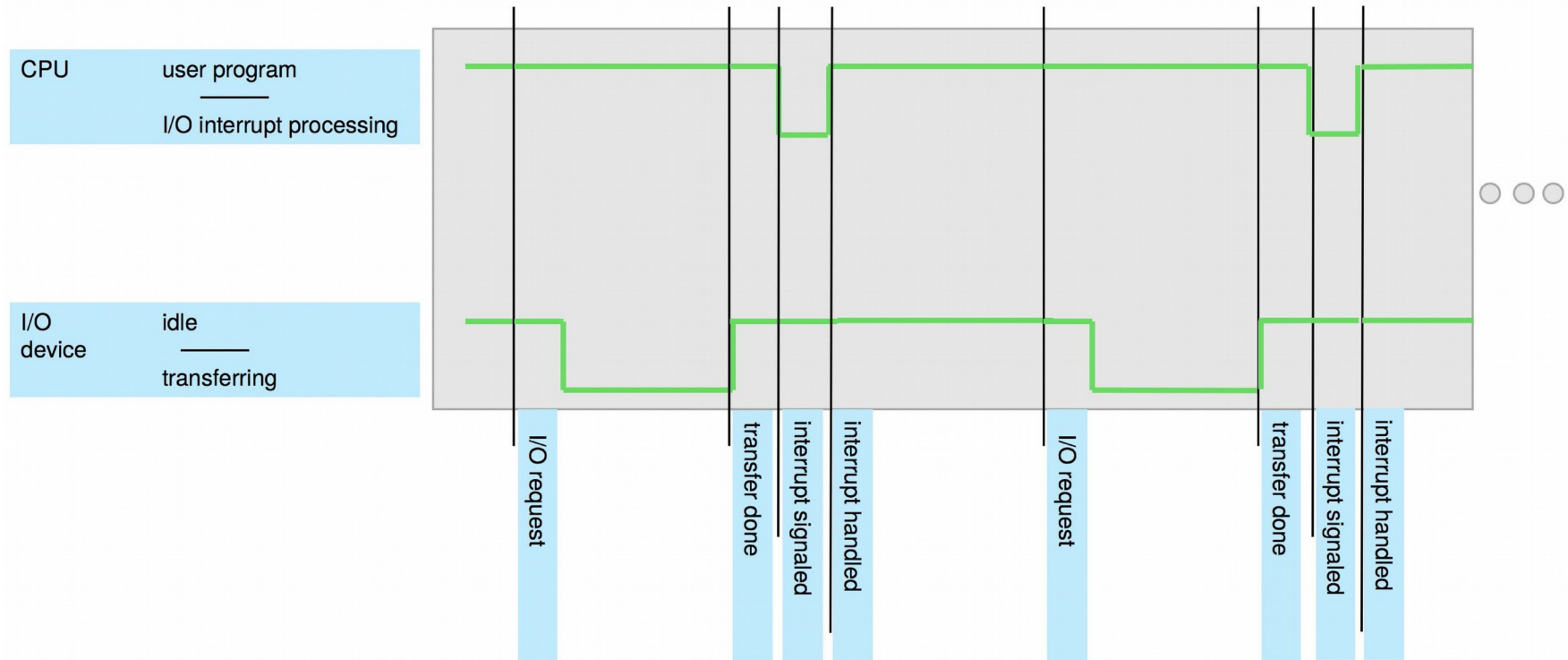
Common Functions of Interrupts

- Interrupt transfers control to the interrupt service routine generally, through the **interrupt vector**, which contains the addresses of all the service routines
- Interrupt architecture must save the address of the interrupted instruction and in general the state of the CPU and be able to restore that state later.
- A **trap** or **exception** is a software-generated interrupt caused either by an error or a user request
- An operating system is **interrupt driven** – it only runs as a result of interrupts (including exceptions).
- Usually two interrupt lines, one **non-maskable**, for events that must be handled immediately, and one for **maskable** interrupts, which can be turned off (deferred usually) by the CPU.





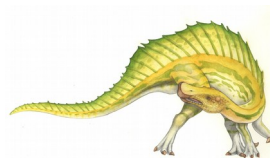
Interrupt Timeline





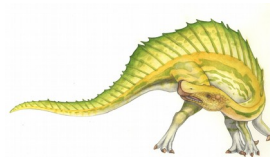
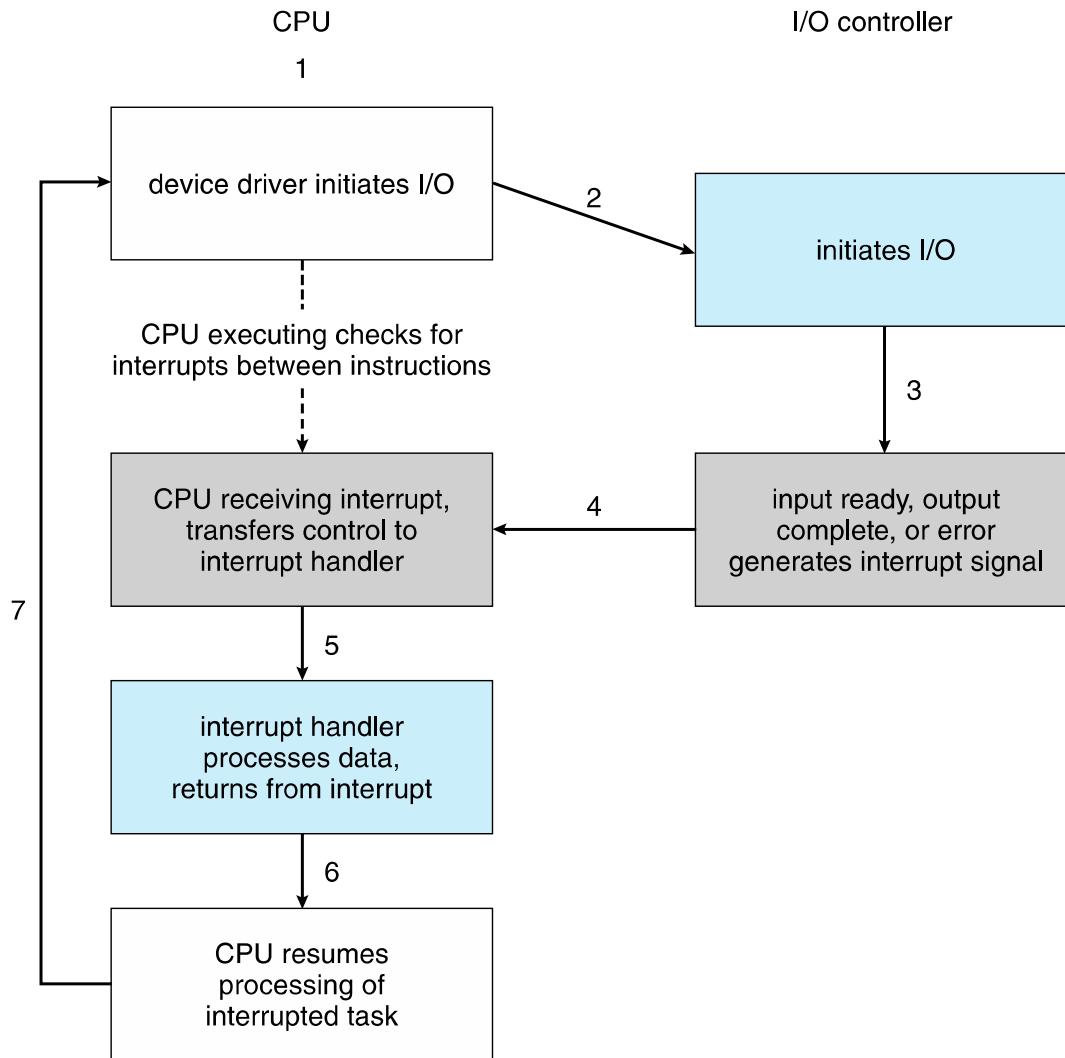
Interrupt Handling

- The operating system preserves the state of the CPU by storing registers and the program counter
- Determines which type of interrupt has occurred:
 - vectored interrupt system
 - polling to determine which device
 - hybrid – interrupt vector with linked list of devices (when number of devices exceeds table size)
- Separate segments of code determine what action should be taken for each type of interrupt – **interrupt service routines**, also called **interrupt handlers**





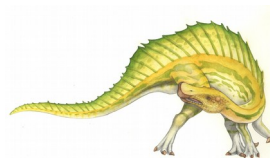
Interrupt-driven I/O Cycle





Synchronous vs Asynchronous I/O

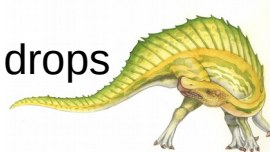
- **Synchronous:** After I/O starts, control returns to user program only upon I/O completion
 - Wait instruction idles the CPU until the next interrupt
 - Wait loop (contention for memory access)
 - At most one I/O request is outstanding at a time, no simultaneous I/O processing
- **Asynchronous:** After I/O starts, control returns to user program without waiting for I/O completion
 - **System call** – request to the OS to allow user to wait for I/O completion
 - **Device-status table** contains entry for each I/O device indicating its type, address, and state
 - OS indexes into I/O device table to determine device status and to modify table entry to include interrupt





Storage Structure

- Main memory – only large storage medium that the CPU can access directly
 - Random access
 - Typically **volatile**
 - Typically **random-access memory** in the form of **Dynamic Random-access Memory (DRAM)**
- Secondary storage – extension of main memory that provides large **nonvolatile** storage capacity
- **Hard Disk Drives (HDD)** – rigid metal or glass platters covered with magnetic recording material
 - Disk surface is logically divided into **tracks**, which are subdivided into **sectors**
 - The **disk controller** determines the logical interaction between the device and the computer
- **Non-volatile memory (NVM)** devices– faster than hard disks, nonvolatile
 - Various technologies
 - Becoming more popular as capacity and performance increases, price drops





Storage Definitions and Notation Review

The basic unit of computer storage is the **bit**. A bit can contain one of two values, 0 and 1. All other storage in a computer is based on collections of bits. Given enough bits, it is amazing how many things a computer can represent: numbers, letters, images, movies, sounds, documents, and programs, to name a few. A **byte** is 8 bits, and on most computers it is the smallest convenient chunk of storage. For example, most computers don't have an instruction to move a bit but do have one to move a byte. A less common term is **word**, which is a given computer architecture's native unit of data. A word is made up of one or more bytes. For example, a computer that has 64-bit registers and 64-bit memory addressing typically has 64-bit (8-byte) words. A computer executes many operations in its native word size rather than a byte at a time.

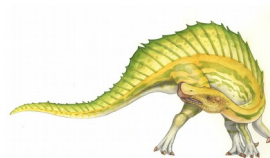
Computer storage, along with most computer throughput, is generally measured and manipulated in bytes and collections of bytes. A **kilobyte**, or KB, is 1,024 bytes; a **megabyte**, or MB, is $1,024^2$ bytes; a **gigabyte**, or GB, is $1,024^3$ bytes; a **terabyte**, or TB, is $1,024^4$ bytes; and a **petabyte**, or PB, is $1,024^5$ bytes. Computer manufacturers often round off these numbers and say that a megabyte is 1 million bytes and a gigabyte is 1 billion bytes. Networking measurements are an exception to this general rule; they are given in bits (because networks move data a bit at a time).





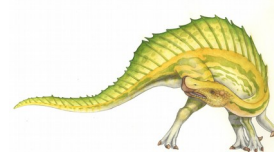
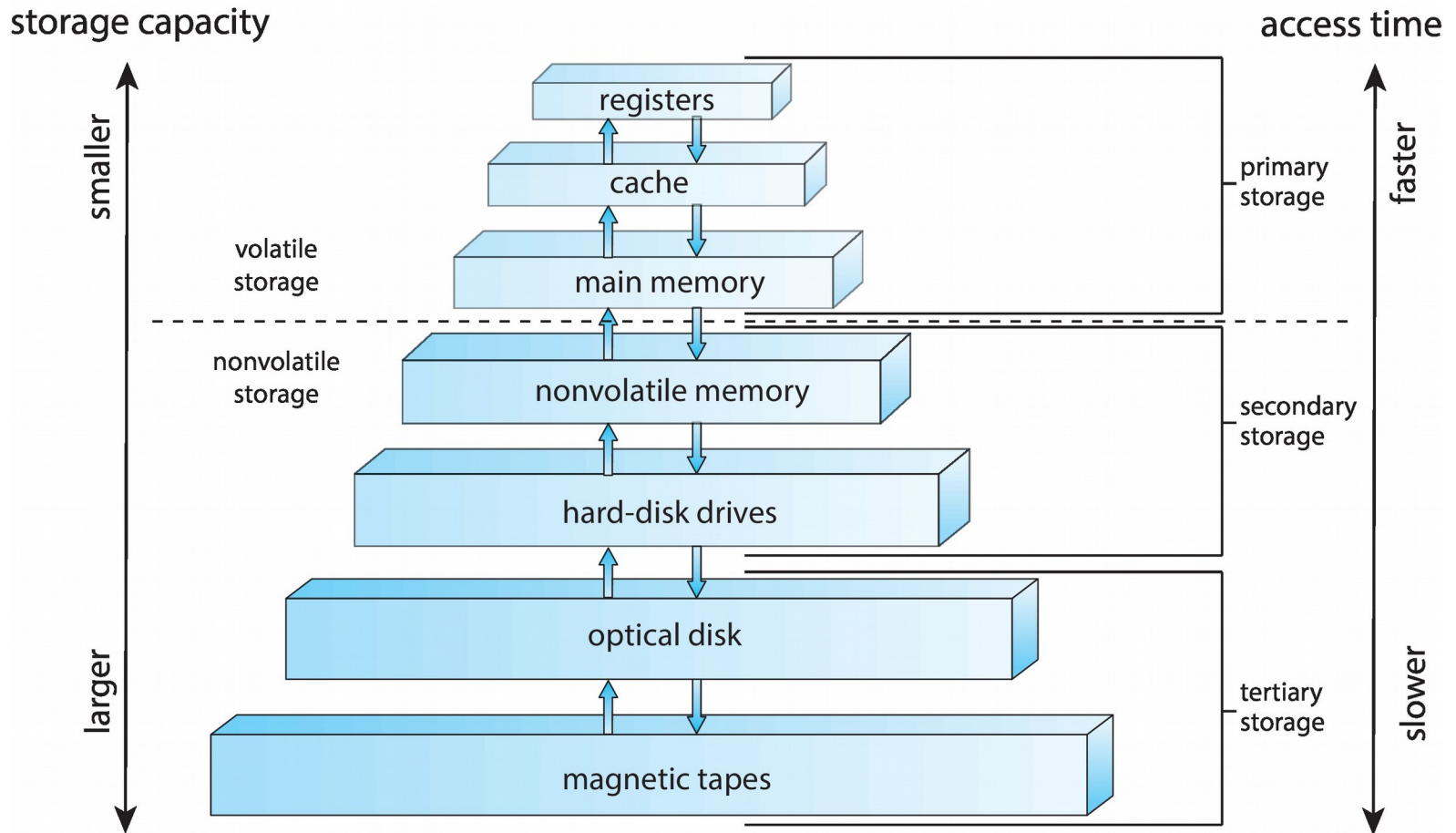
Storage Hierarchy

- Storage systems organized in hierarchy
 - Speed
 - Cost
 - Volatility
- **Caching** – copying information into faster storage system; main memory can be viewed as a cache for secondary storage
- **Device Driver** for each device controller to manage I/O
 - Provides uniform interface between controller and kernel



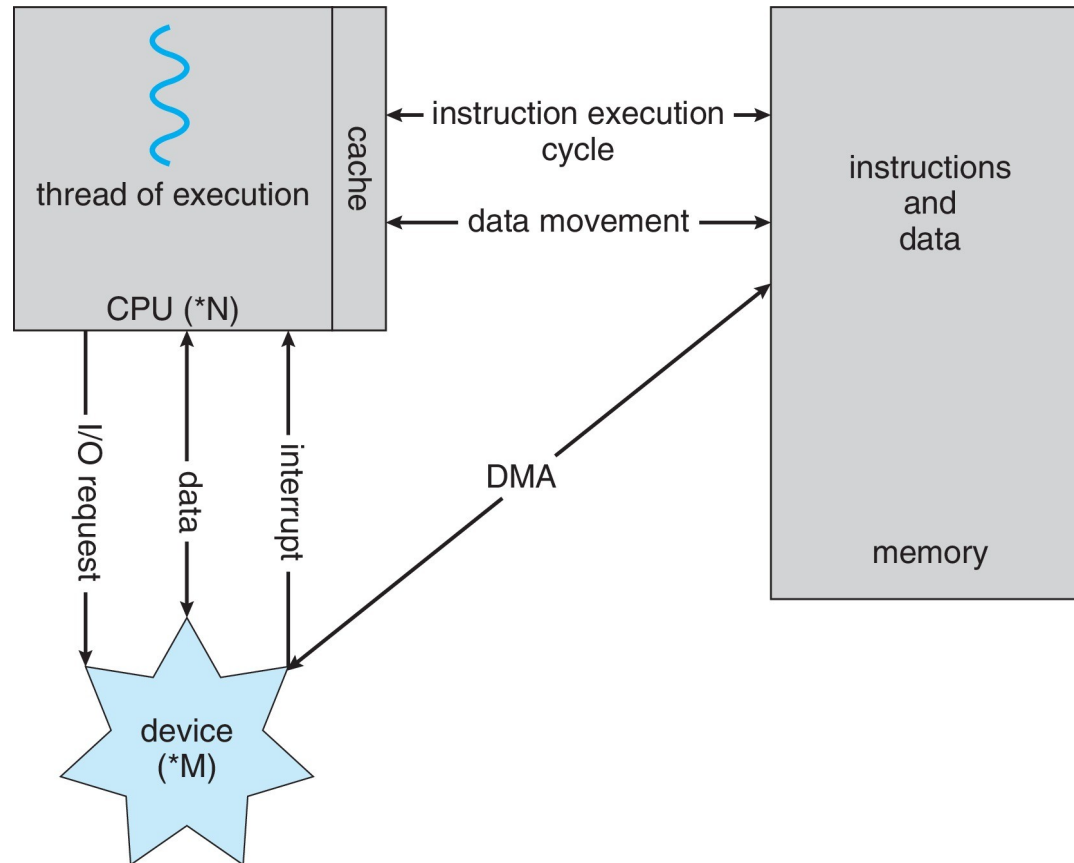


Storage-Device Hierarchy





How a Modern Computer Works



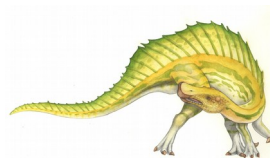
A von Neumann architecture





Direct Memory Access Structure

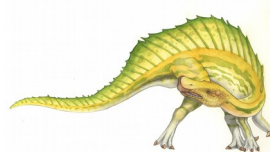
- Used for high-speed I/O devices able to transmit information at close to memory speeds
- Device controller transfers blocks of data from buffer storage directly to main memory without CPU intervention. (CPU needs to set up transfer first.)
- Only one interrupt is generated per block, rather than the one interrupt per byte





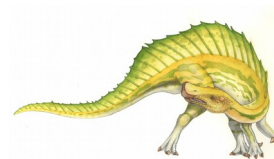
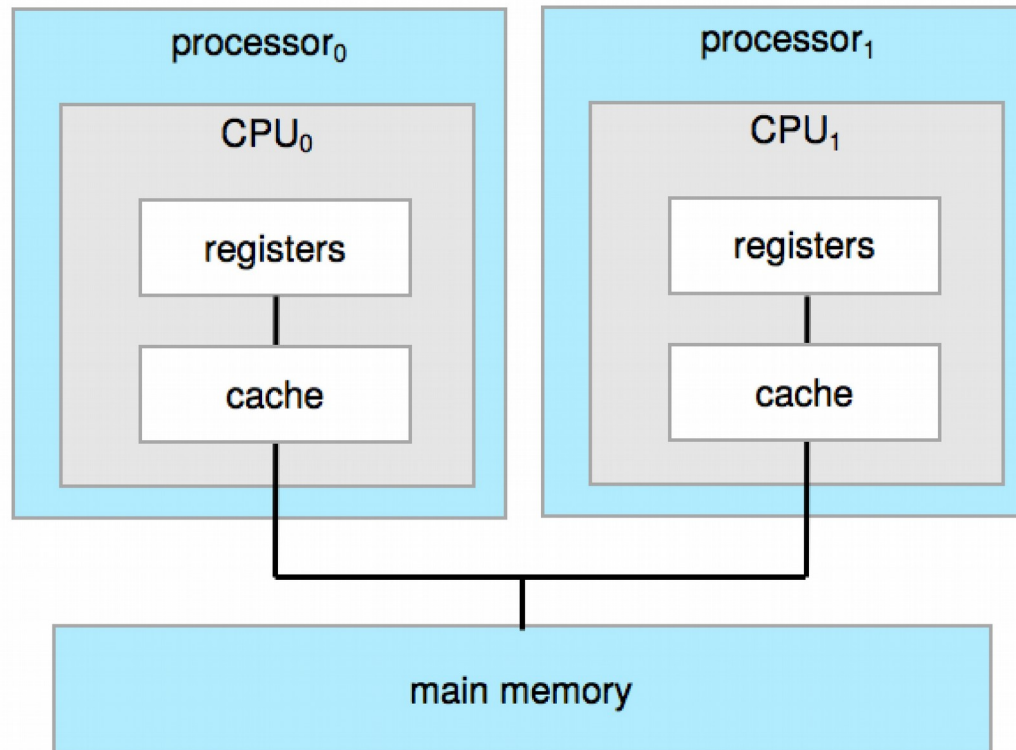
Computer-System Architecture

- Most systems used to use a single general-purpose processor. Now most use more than one.
 - Most systems have special-purpose processors as well
- **Multiprocessors** systems growing in use and importance
 - Also known as **parallel systems**, **tightly-coupled systems**
 - Advantages include:
 1. **Increased throughput**
 2. **Economy of scale**
 3. **Increased reliability** – graceful degradation or fault tolerance
 - Two types:
 1. **Asymmetric Multiprocessing** – each processor is assigned a specific task.
 2. **Symmetric Multiprocessing** – each processor performs all tasks





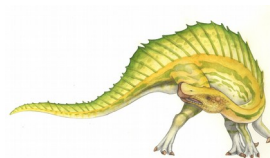
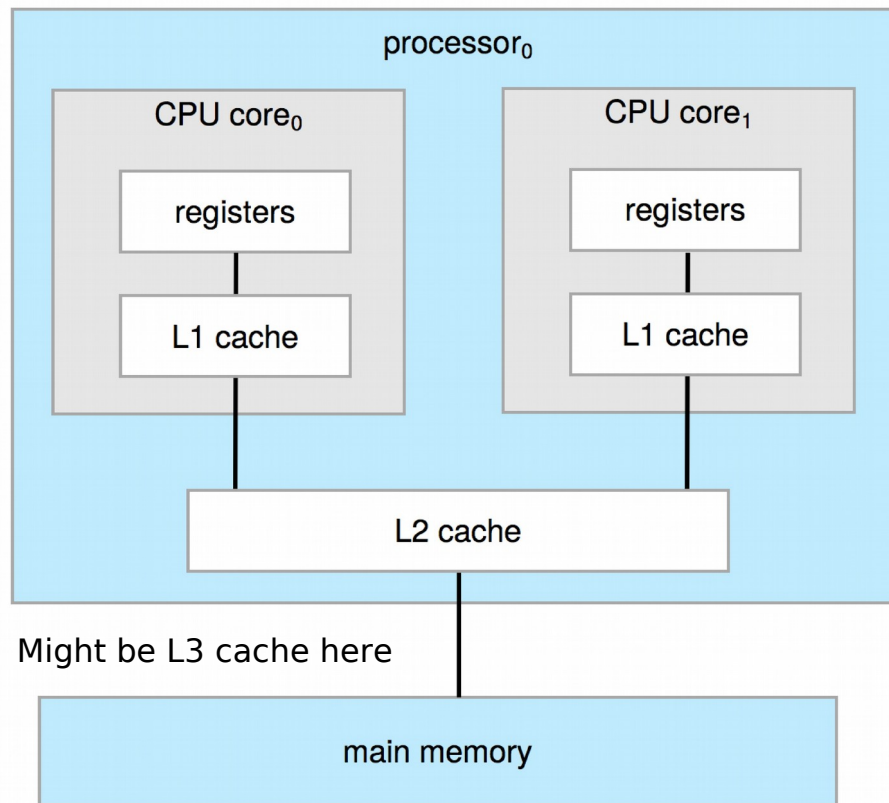
2-Processor Symmetric Multiprocessing Architecture





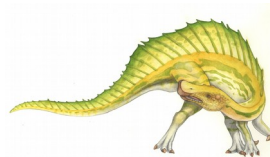
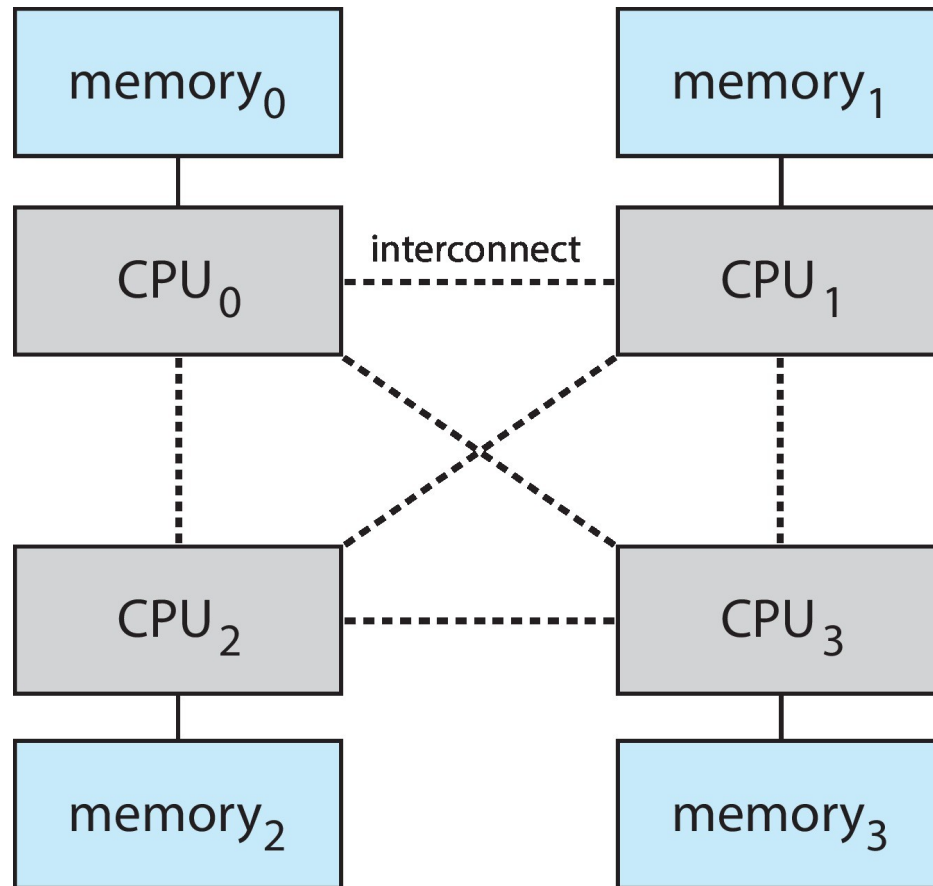
A Dual-Core Design

- Multi-chip and **multicore**
- Systems containing all chips
 - Chassis containing multiple separate systems





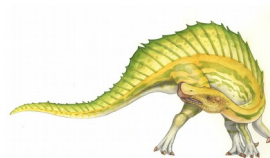
Non-Uniform Memory (NUMA) Access System





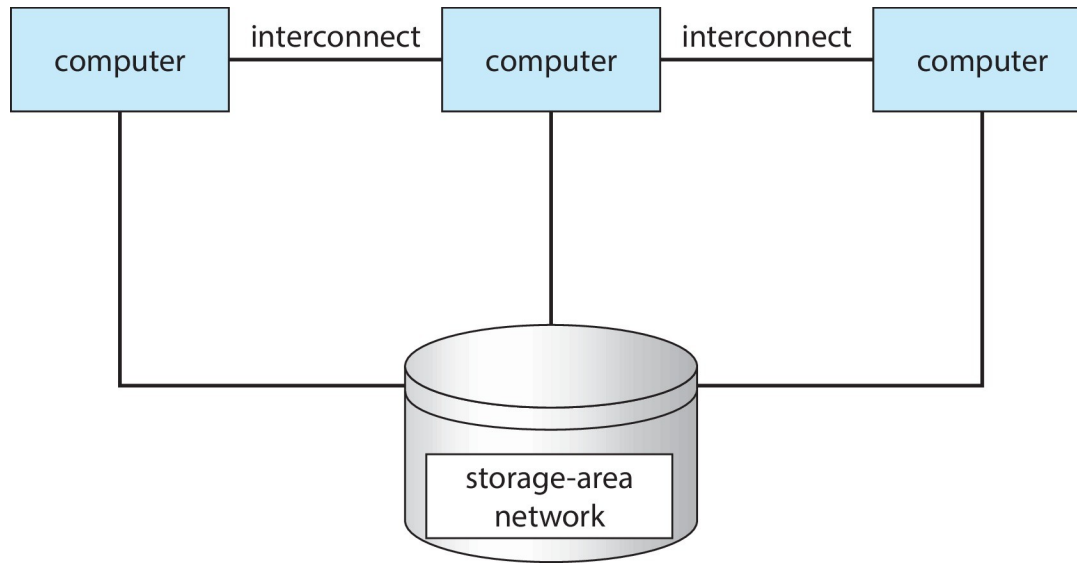
Clustered Systems

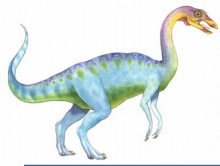
- Like multiprocessor systems, but multiple systems working together
 - Usually sharing storage via a **storage-area network (SAN)**
 - Provides a **high-availability** service which survives failures
 - ▶ **Asymmetric clustering** has one machine in hot-standby mode
 - ▶ **Symmetric clustering** has multiple nodes running applications, monitoring each other
 - Some clusters are for **high-performance computing (HPC)**
 - ▶ Applications must be written to use **parallelization**
 - Some have **distributed lock manager (DLM)** to avoid conflicting operations





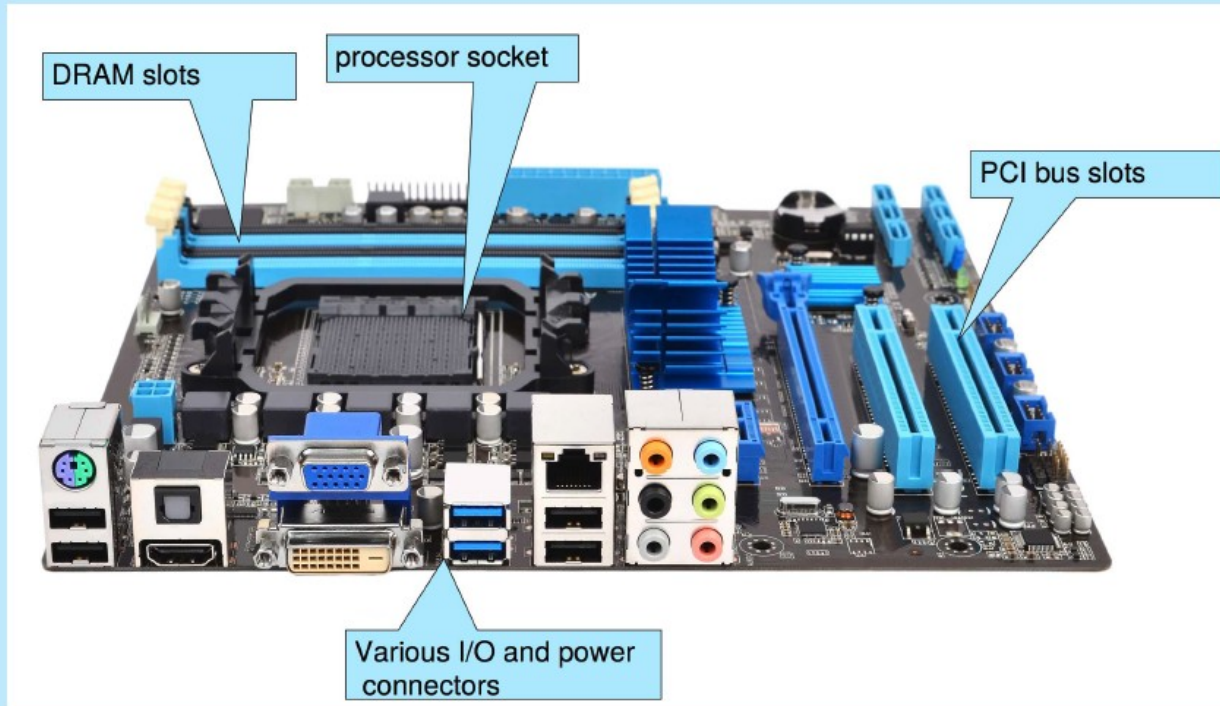
A Clustered System



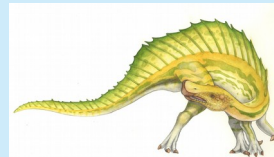


PC Motherboard

Consider the desktop PC motherboard with a processor socket shown below:



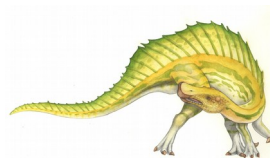
This board is a fully-functioning computer, once its slots are populated. It consists of a processor socket containing a CPU, DRAM sockets, PCIe bus slots, and I/O connectors of various types. Even the lowest-cost general-purpose CPU contains multiple cores. Some motherboards contain multiple processor sockets. More advanced computers allow more than one system board, creating NUMA systems.





Operating-System Operations

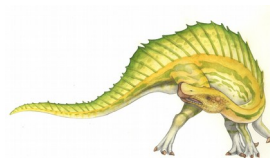
- Bootstrap program – simple code to initialize the system, load the kernel
- Kernel loads
- Starts **system daemons** (services provided outside of the kernel)
- Kernel **interrupt driven** (hardware and software)
 - Hardware interrupt by one of the devices
 - Software interrupt (**exception** or **trap**):
 - ▶ Software error (e.g., division by zero)
 - ▶ Request for operating system service – **system call**
 - ▶ Other process problems include infinite loop, processes modifying each other or the operating system





Multiprogramming

- **Multiprogramming** means keeping more than one program in memory at a time.
 - A single program cannot keep CPU and I/O devices busy at all times.
 - Allows several programs to be run simultaneously.
 - Multiprogramming allows OS to switch programs so that CPU always has one to execute.
 - When running process requests I/O, CPU is given to a process that can make progress.
 - Increases **CPU utilization** (*fraction of time CPU is doing useful work*).





Batch Processing and Multitasking

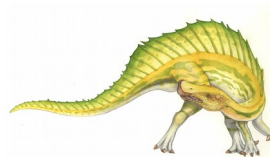
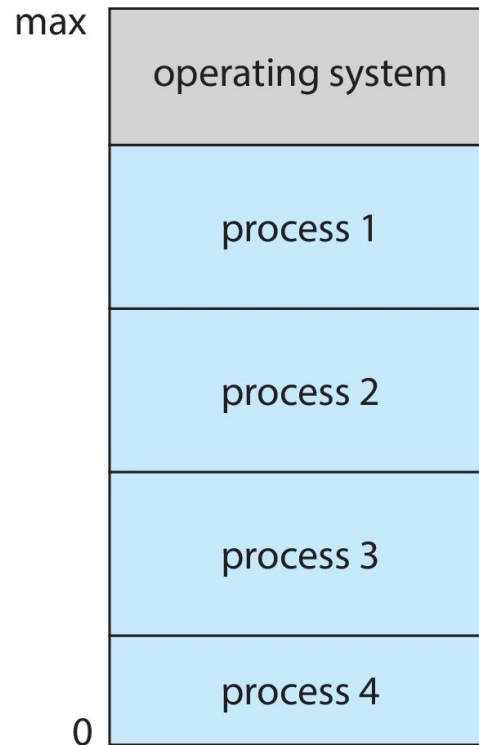
- **Batch processing** is one kind of multiprogramming system:
 - Processes jobs in bulk, with predetermined input from files or other data sources.
 - A subset of total jobs in system is kept in memory
 - One job selected and run via **job scheduling**
 - Each job is *run to completion*.

- **Multitasking** is a kind of multiprogramming system in which processes get small time slices.
 - It is a form of **timesharing system**.
 - CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing.
 - **Response time** should be fast
 - If several jobs ready to run at the same time **CPU scheduling** is used
 - If processes don't fit in memory, **swapping** moves them in and out to run
 - **Virtual memory** allows execution of processes not completely in memory





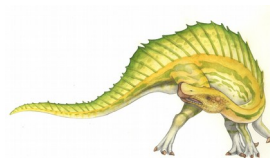
Memory Layout for Multiprogrammed System





Dual-mode and Multimode Operation

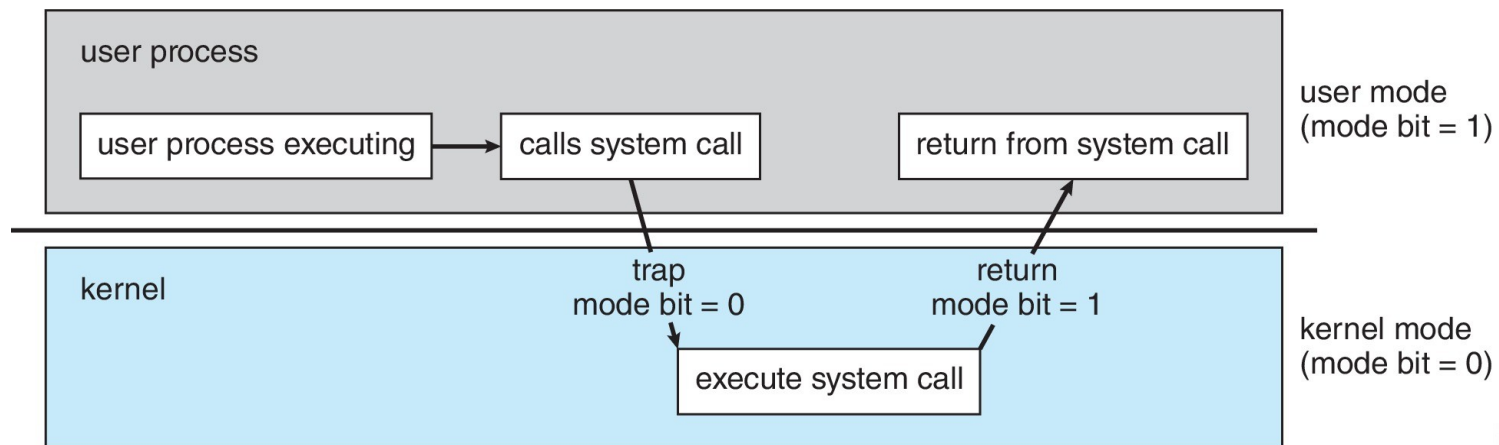
- **Dual-mode** operation allows OS to protect itself and other system components. Two modes:
 - **User mode** and **kernel mode (aka privileged mode, supervisor mode)**
 - **Mode bit** provided by hardware
 - ▶ Provides ability to distinguish when system is running user code or kernel code
 - ▶ Some instructions designated as **privileged**, only executable in **kernel mode**
 - ▶ System call changes mode to kernel, return from call resets it to user
- Increasingly CPUs support multi-mode operations
 - i.e. **virtual machine manager (VMM)** mode for guest **VMs**





Transition from User to Kernel Mode

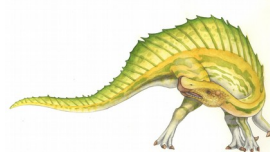
- **Timer** to prevent infinite loop / process hogging resources
 - Timer is set to interrupt the computer after some time period
 - Keep a counter that is decremented by the physical clock
 - Operating system set the counter (privileged instruction)
 - When counter is zero, generate an interrupt
 - Set up before scheduling process to regain control or terminate program that exceeds allotted time





Process Management

- ***A process is a program in execution.*** It is a unit of work within the system. Program is a **passive entity**, process is an **active entity**.
- Process needs resources to accomplish its task
 - CPU, memory, I/O, files
 - Initialization data
- Process termination requires reclaiming any reusable resources
- Single-threaded process has one **program counter** specifying location of next instruction to execute
 - Process executes instructions sequentially, one at a time, until completion
- Multi-threaded process has one program counter per thread
- Typically system has many processes, some user, some operating system running concurrently on one or more CPUs
 - Concurrency by multiplexing the CPUs among the processes / threads

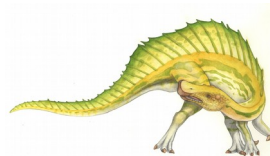




Process Management Activities

The operating system is responsible for the following activities in connection with process management:

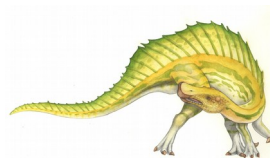
- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication
- Providing mechanisms for deadlock handling





Memory Management

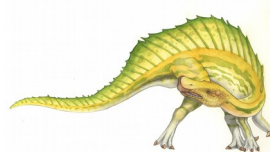
- To execute a program all (or part) of the instructions must be in memory
- All (or part) of the data that is needed by the program must be in memory
- Memory management determines what is in memory and when
 - Optimizing CPU utilization and computer response to users
- Memory management activities
 - Keeping track of which parts of memory are currently being used and by whom
 - Deciding which processes (or parts thereof) and data to move into and out of memory
 - Allocating and deallocating memory space as needed





File-system Management

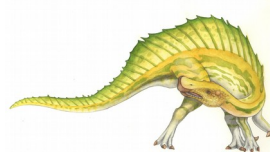
- OS provides uniform, logical view of information storage
 - Abstracts physical properties to logical storage unit - **file**
 - Each medium is controlled by device (i.e., disk drive, tape drive)
 - ▶ Varying properties include access speed, capacity, data-transfer rate, access method (sequential or random)
- File-System management
 - Files usually organized into directories
 - Access control on most systems to determine who can access what
 - OS activities include
 - ▶ Creating and deleting files and directories
 - ▶ Primitives to manipulate files and directories
 - ▶ Mapping files onto secondary storage
 - ▶ Backup files onto stable (non-volatile) storage media





Mass-Storage Management

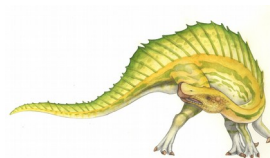
- Usually disks used to store data that does not fit in main memory or data that must be kept for a “long” period of time
- Proper management is of central importance
- Entire speed of computer operation hinges on disk subsystem and its algorithms
- OS activities
 - Mounting and unmounting filesystems to file hierarchy
 - Free-space management
 - Storage allocation
 - Disk scheduling
 - Partitioning disks
 - Protection
- Some storage need not be fast
 - Tertiary storage includes optical storage, magnetic tape
 - Still must be managed – by OS or applications





Caching

- Important principle, performed at many levels in a computer (in hardware, operating system, software)
- Information in use copied from slower to faster storage temporarily
- Faster storage (cache) checked first to determine if information is there
 - If it is, information used directly from the cache (fast)
 - If not, data from slower storage copied to cache and then used there
- Cache smaller than storage being cached
 - Cache management important design problem
 - Cache size and replacement policy

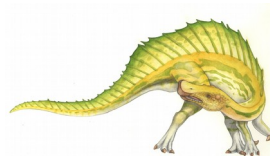




Characteristics of Various Types of Storage

Level	1	2	3	4	5
Name	registers	cache	main memory	solid-state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25-0.5	0.5-25	80-250	25,000-50,000	5,000,000
Bandwidth (MB/sec)	20,000-100,000	5,000-10,000	1,000-5,000	500	20-150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

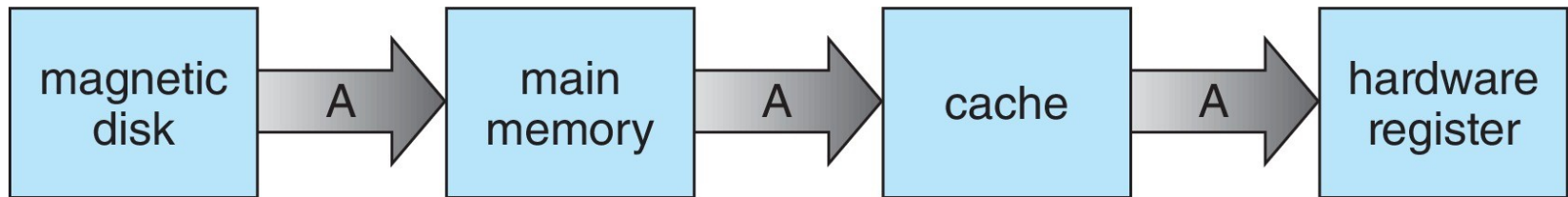
- Movement between levels of storage hierarchy can be explicit or implicit.
- Values based on data from around 2017



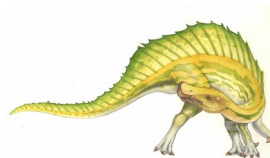


Migration of data “A” from Disk to Register

- Multitasking environments must be careful to use most recent value, no matter where it is stored in the storage hierarchy



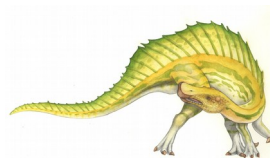
- Multiprocessor environment must provide **cache coherency** in hardware such that all CPUs have the most recent value in their cache
- Distributed environment situation even more complex
 - Several copies of a datum can exist
 - Various solutions covered in Chapter 19





I/O Subsystem

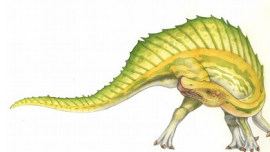
- One purpose of OS is to hide peculiarities of hardware devices from the user
- I/O subsystem responsible for
 - Memory management of I/O including buffering (storing data temporarily while it is being transferred), caching (storing parts of data in faster storage for performance), spooling (the overlapping of output of one job with input of other jobs)
 - General device-driver interface
 - Drivers for specific hardware devices





Protection and Security

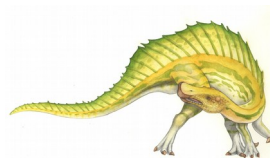
- **Protection** – any mechanism for controlling access of processes or users to resources defined by the OS
- **Security** – defense of the system against internal and external attacks
 - Huge range, including denial-of-service, worms, viruses, identity theft, theft of service
- Systems generally first distinguish among users, to determine who can do what
 - User identities (**user IDs**, security IDs) include name and associated number, one per user
 - User ID then associated with all files, processes of that user to determine access control
 - Group identifier (**group ID**) allows set of users to be defined and controls managed, then also associated with each process, file
 - **Privilege escalation** allows user to change to effective ID with more rights





Computing Environments - Traditional

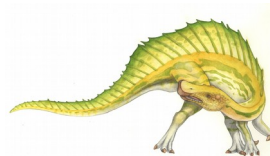
- Stand-alone general purpose machines
- But blurred as most systems interconnect with others (i.e., the Internet)
- **Portals** provide web access to internal systems
- **Network computers** (**thin clients**) are like Web terminals
- Mobile computers interconnect via **wireless networks**
- Networking becoming ubiquitous – even home systems use **firewalls** to protect home computers from Internet attacks





Computing Environments - Mobile

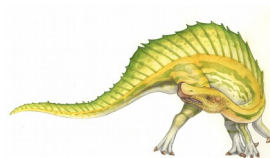
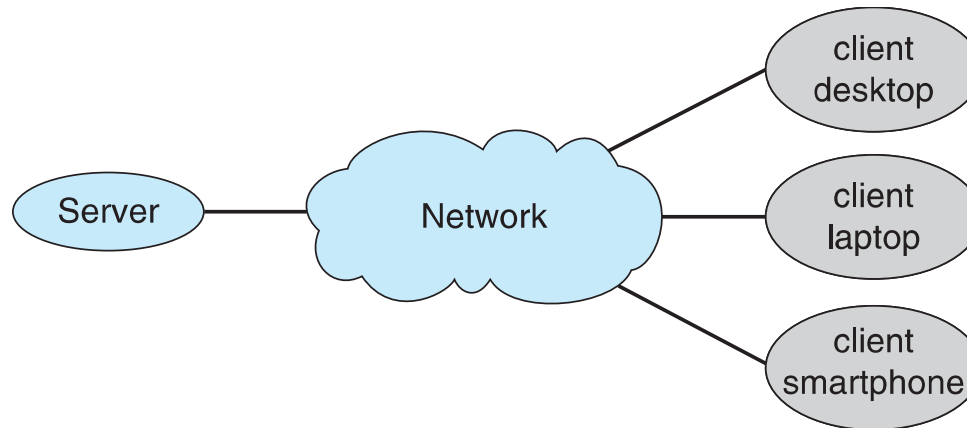
- Handheld smartphones, tablets, etc
- What is the functional difference between them and a “traditional” laptop?
- Extra feature – more OS features (GPS, gyroscope)
- Allows new types of apps like ***augmented reality***
- Use IEEE 802.11 wireless, or cellular data networks for connectivity
- Most common are **Apple iOS** and **Google Android**





Computing Environments – Client-Server

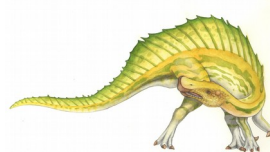
- Client-Server Computing
 - Dumb terminals supplanted by smart PCs
 - Many systems now **servers**, responding to requests generated by **clients**
 - ▶ **Compute-server system** provides an interface to client to request services (i.e., database)
 - ▶ **File-server system** provides interface for clients to store and retrieve files





Free and Open-Source Operating Systems

- Operating systems made available in source-code format rather than just binary and free to modify, as opposed to **closed-source** and **proprietary**
- Counter to the **copy protection** and **Digital Rights Management (DRM)** movement
- UNIX was the first one.
- Promoted by **Free Software Foundation (FSF)**, which invented the first “copyleft” **GNU Public License (GPL)**
 - Free software and open-source software are two different ideas championed by different groups of people
- Examples include **GNU/Linux** and **BSD UNIX** (including core of **Mac OS X**), and many more
- Can use VMM like VMware Player (Free on Windows), Virtualbox (open source on many platforms - <http://www.virtualbox.com>)
 - Use to run guest operating systems for exploration





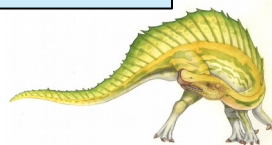
The Study of Operating Systems

There has never been a more interesting time to study operating systems, and it has never been easier. The open-source movement has overtaken operating systems, causing many of them to be made available in both source and binary (executable) format. The list of operating systems available in both formats includes Linux, BSD UNIX, Solaris, and part of macOS. The availability of source code allows us to study operating systems from the inside out. Questions that we could once answer only by looking at documentation or the behavior of an operating system we can now answer by examining the code itself.

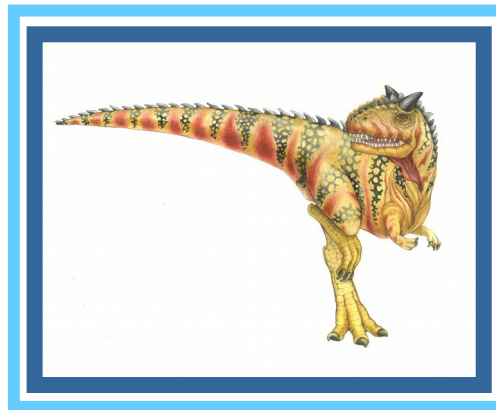
Operating systems that are no longer commercially viable have been open-sourced as well, enabling us to study how systems operated in a time of fewer CPU, memory, and storage resources. An extensive but incomplete list of open-source operating-system projects is available from https://curlie.org/Computers/Software/Operating_Systems/Open_Source/

In addition, the rise of virtualization as a mainstream (and frequently free) computer function makes it possible to run many operating systems on top of one core system. For example, VMware (<http://www.vmware.com>) provides a free “player” for Windows on which hundreds of free “virtual appliances” can run. Virtualbox (<http://www.virtualbox.com>) provides a free, open-source virtual machine manager on many operating systems. Using such tools, students can try out hundreds of operating systems without dedicated hardware.

The advent of open-source operating systems has also made it easier to make the move from student to operating-system developer. With some knowledge, some effort, and an Internet connection, a student can even create a new operating-system distribution. Just a few years ago, it was difficult or impossible to get access to source code. Now, such access is limited only by how much interest, time, and disk space a student has.



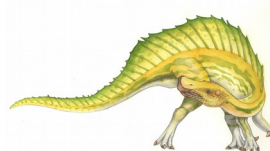
Chapter 2: Operating-System Structures





Operating-System Structures

- Operating System Services
- User and Operating System-Interface
- System Calls
- System Services
- Linkers and Loaders
- Why Applications are Operating System Specific
- Operating-System Design and Implementation
- Operating System Structure





Chapter Objectives

- Identify services provided by an operating system
- Illustrate how system calls are used to provide operating system services
- Compare and contrast monolithic, layered, microkernel, modular, and hybrid strategies for designing operating systems
- Apply tools for monitoring operating system performance





Operating System Services 1

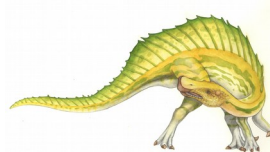
- Operating systems provide an environment for execution of programs and services to programs and users
- One set of operating-system services provides functions that are helpful to the user:
 - **User interface** - Almost all operating systems have a user interface (**UI**).
 - ▶ Varies between **Command-Line (CLI)**, **Graphical User Interface (GUI)**, **touch-screen**, **Batch**
 - **Program execution** - The system must be able to load a program into memory and to run that program, terminate execution, either normally or abnormally (indicating error).
 - **I/O operations** - A running program may require I/O, which may involve a file or an I/O device; OS must provide interface to I/O subsystems.





Operating System Services 2

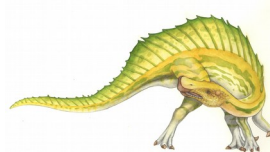
- More operating-system services that are helpful to the user :
 - **File-system manipulation** - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file information, permission management.
 - **Communications** – Processes may exchange information, on the same computer or between computers over a network
 - ▶ Communications may be via shared memory or through message passing (packets moved by the OS)
 - **Error detection** – OS needs to be constantly aware of possible errors
 - ▶ May occur in the CPU and memory hardware, in I/O devices, in user program
 - ▶ For each type of error, OS should take the appropriate action to ensure correct and consistent computing
 - ▶ Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system





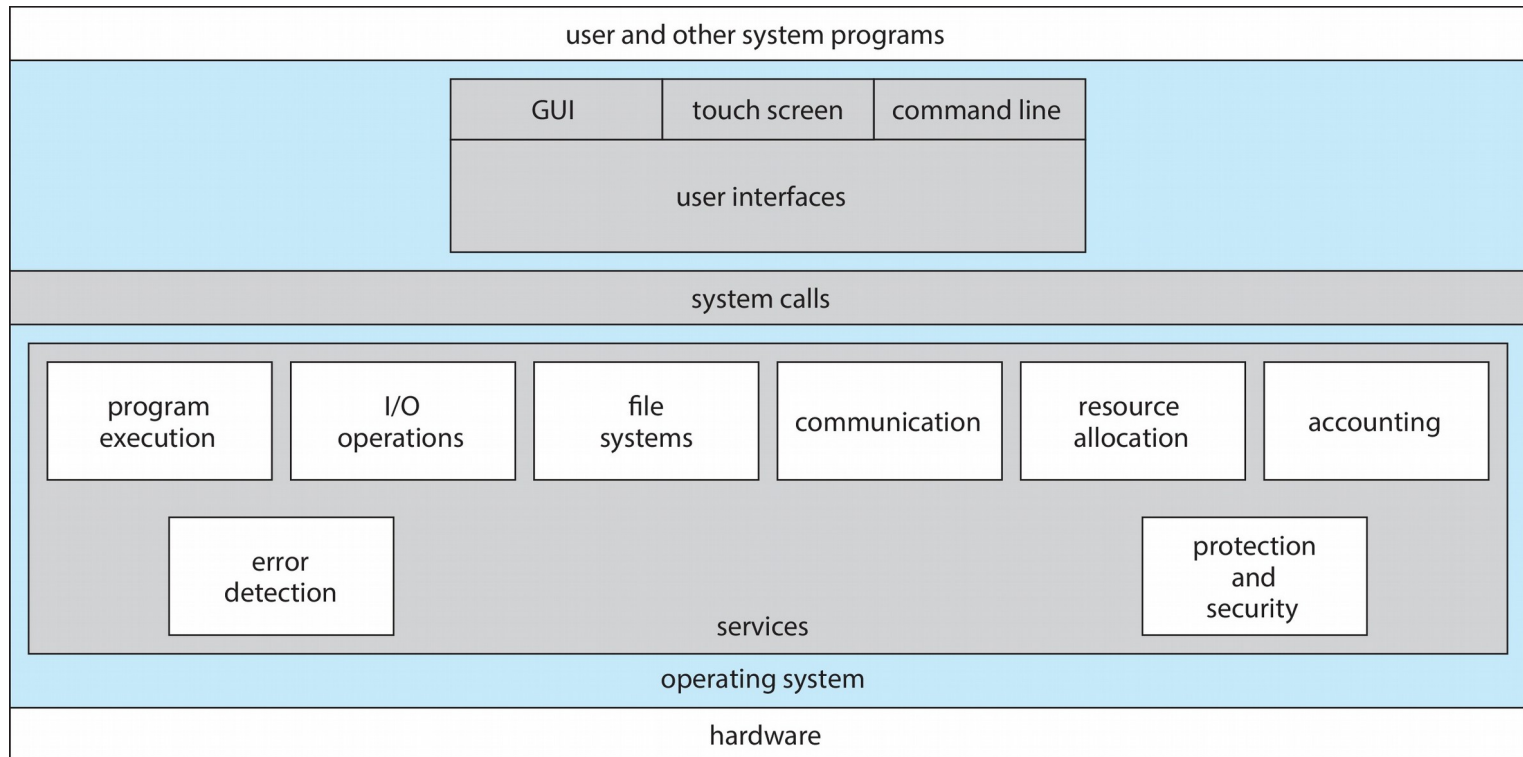
Operating System Services 3

- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing
 - **Resource allocation** - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
 - ▶ Many types of resources - CPU cycles, main memory, file storage, I/O devices.
 - **Logging** - To keep track of which users use how much and what kinds of computer resources
 - **Protection and security** - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
 - ▶ **Protection** involves ensuring that all access to system resources is controlled
 - ▶ **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts





A View of Operating System Services

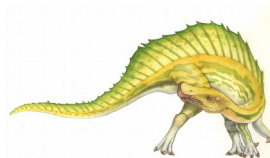




User Operating System Interface - CLI

CLI or **command line interpreter** allows direct command entry

- Sometimes implemented in kernel, sometimes by systems program
- Sometimes multiple programs implemented – **shells (sh, bash, csh, tcsh, zsh)**
- Primarily reads a command entered by user and executes it
- Sometimes commands are built-in, sometimes just names of programs
 - ▶ If the latter, adding new features doesn't require shell modification

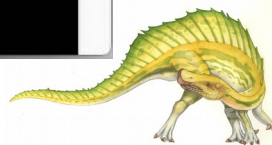




Bourne Shell Command Interpreter

```
1. root@r6181-d5-us01:~ (ssh)
root@r6181-d5-u... 1
ssh 2
root@r6181-d5-us01... 3

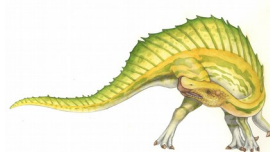
Last login: Thu Jul 14 08:47:01 on ttys002
iMacPro:~ pbg$ ssh root@r6181-d5-us01
root@r6181-d5-us01's password:
Last login: Thu Jul 14 06:01:11 2016 from 172.16.16.162
[root@r6181-d5-us01 ~]# uptime
 06:57:48 up 16 days, 10:52,  3 users,  load average: 129.52, 80.33, 56.55
[root@r6181-d5-us01 ~]# df -kh
Filesystem                Size      Used Avail Use% Mounted on
/dev/mapper/vg_ks-lv_root    50G       19G   28G  41% /
tmpfs                      127G      520K   127G   1% /dev/shm
/dev/sda1                   477M       71M   381M  16% /boot
/dev/dssd0000               1.0T     480G   545G  47% /dssd_xfs
tcp://192.168.150.1:3334/orangeifs
                          12T     5.7T   6.4T  47% /mnt/orangeifs
/dev/gpfs-test              23T     1.1T    22T   5% /mnt/gpfs
[root@r6181-d5-us01 ~]#
[root@r6181-d5-us01 ~]# ps aux | sort -nrk 3,3 | head -n 5
root      97653 11.2  6.6 42665344 17520636 ?    S<Ll  Jul13 166:23 /usr/lpp/mmfs/bin/mmfsd
root      69849  6.6  0.0      0      0 ?        S    Jul12 181:54 [vpthread-1-1]
root      69850  6.4  0.0      0      0 ?        S    Jul12 177:42 [vpthread-1-2]
root       3829  3.0  0.0      0      0 ?        S    Jun27 730:04 [rp_thread 7:0]
root       3826  3.0  0.0      0      0 ?        S    Jun27 728:08 [rp_thread 6:0]
[root@r6181-d5-us01 ~]# ls -l /usr/lpp/mmfs/bin/mmfsd
-r-x----- 1 root root 20667161 Jun  3  2015 /usr/lpp/mmfs/bin/mmfsd
[root@r6181-d5-us01 ~]#
```





User Operating System Interface - GUI

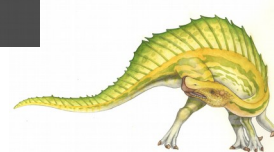
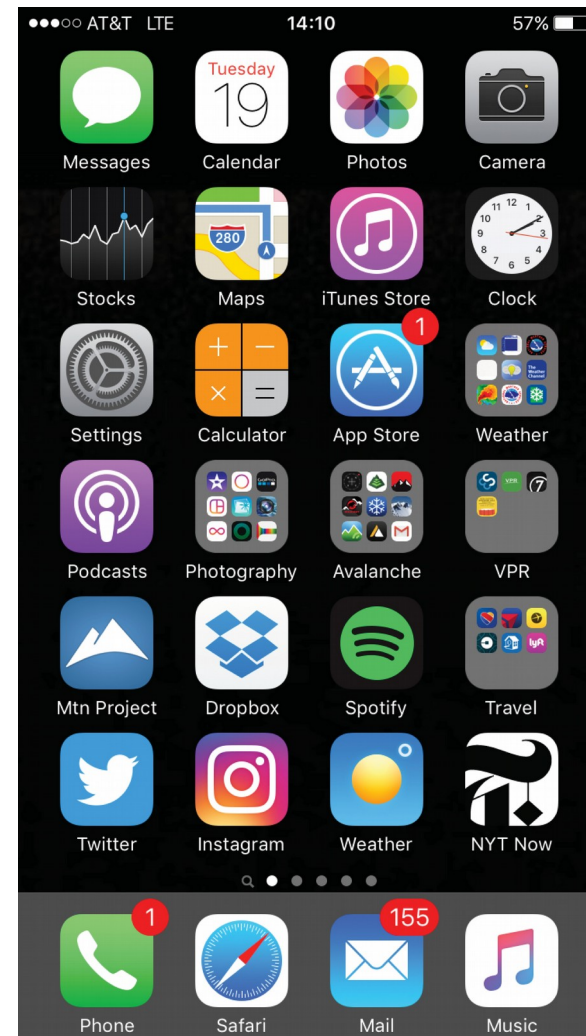
- User-friendly **desktop** metaphor interface
 - Usually **mouse** (or **touchpad**), **keyboard**, and **monitor**
 - **Icons** represent files, programs, actions, etc
 - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (also known as a **folder**))
 - Invented at Xerox PARC
- Many systems now include both CLI and GUI interfaces
 - Microsoft Windows is GUI with CLI “command” shell
 - Apple Mac OS X is “Aqua” GUI interface with UNIX kernel underneath and shells available
 - Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME)

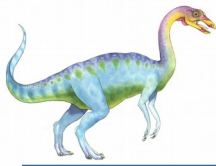




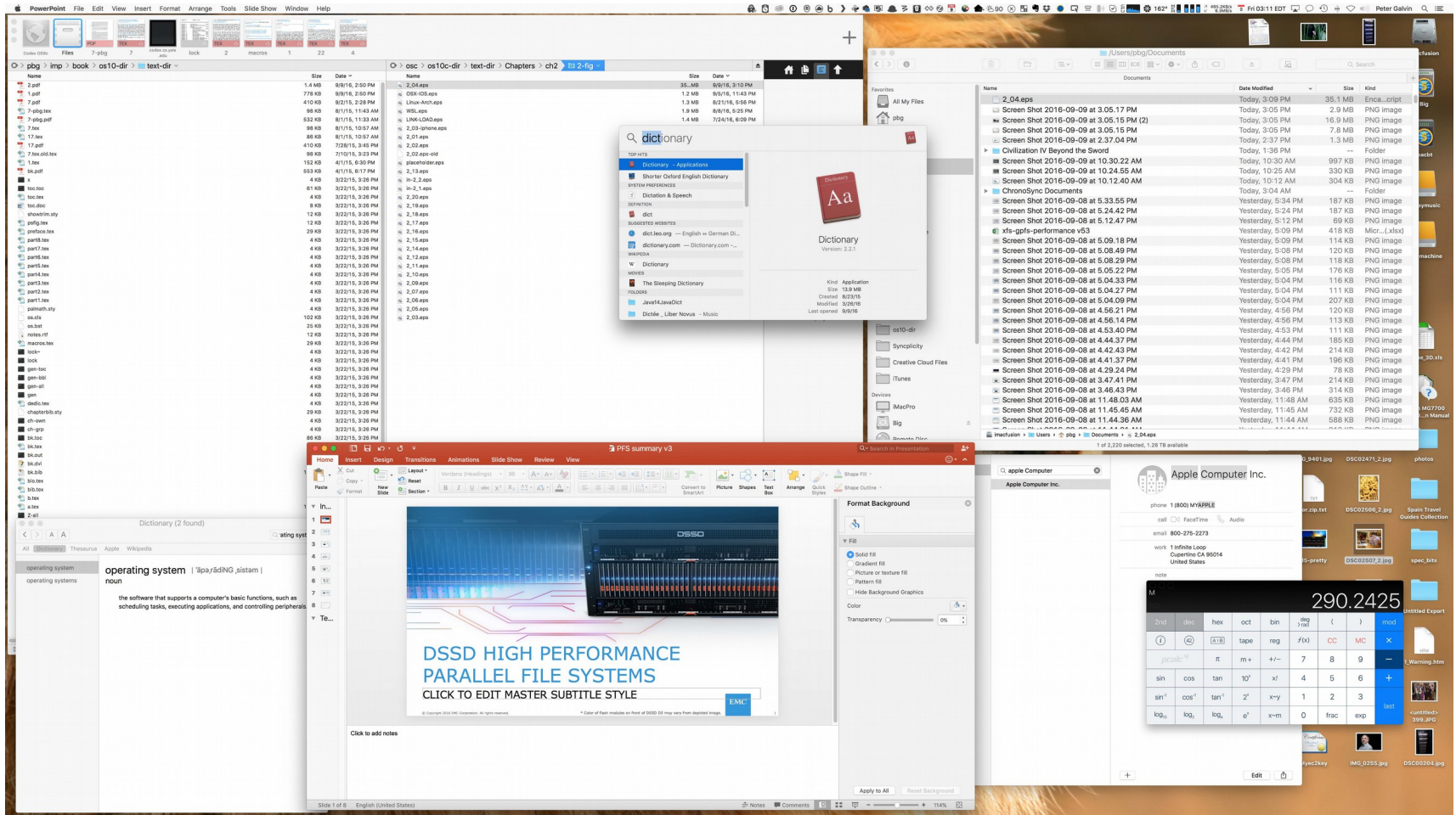
Touchscreen Interfaces

- Touchscreen devices require new interfaces
 - Mouse not possible or not desired
 - Actions and selection based on gestures
 - Virtual keyboard for text entry
- Voice commands





The Mac OS X GUI

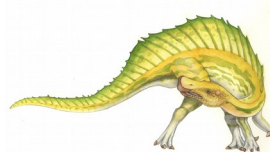




System Calls

- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level **Application Programming Interface (API)** rather than direct system call use
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)

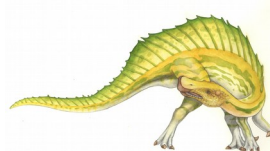
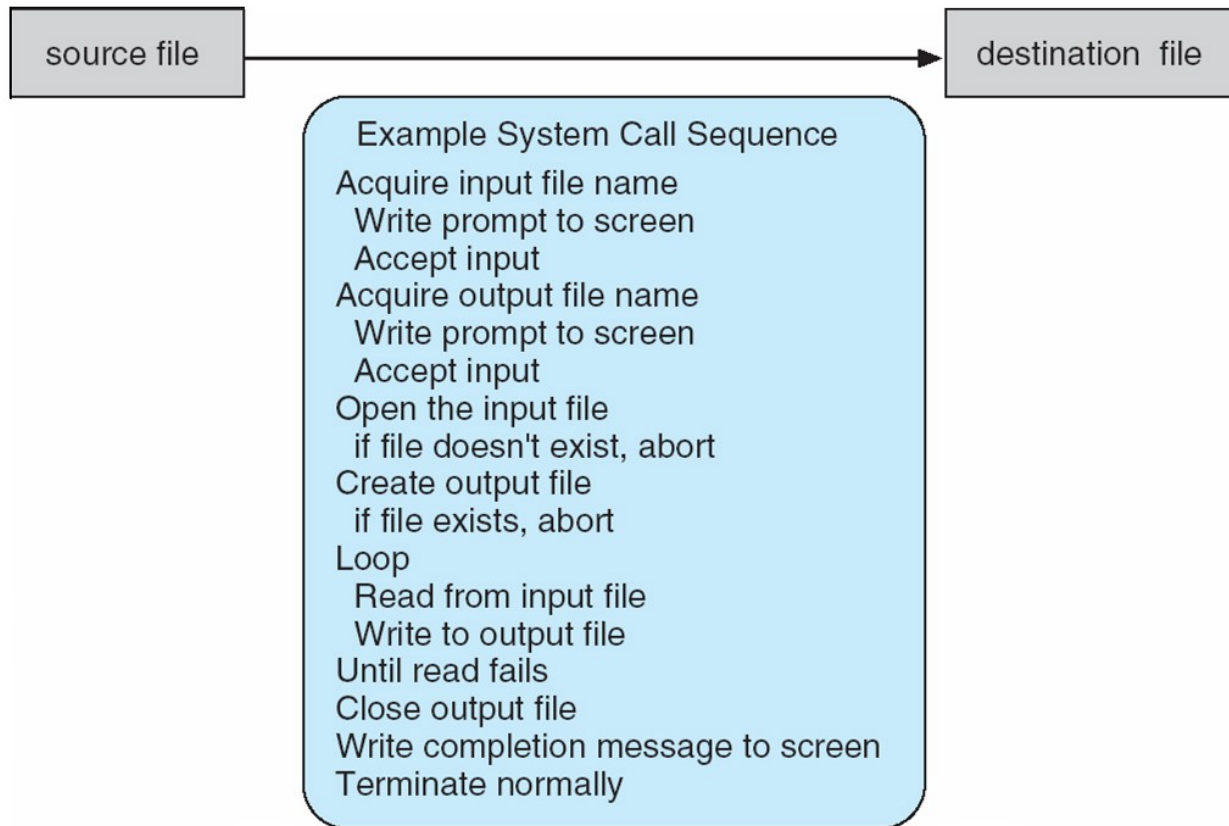
Note that the system-call names used throughout this text are generic





Example of System Calls

- System call sequence to copy the contents of one file to another file





Example of Standard API

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the man page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

<pre>#include <unistd.h></pre>		
<pre>ssize_t</pre>	<pre>read(int fd, void *buf, size_t count)</pre>	
return	function	parameters
value	name	

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer into which the data will be read
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns `-1`.





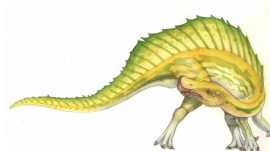
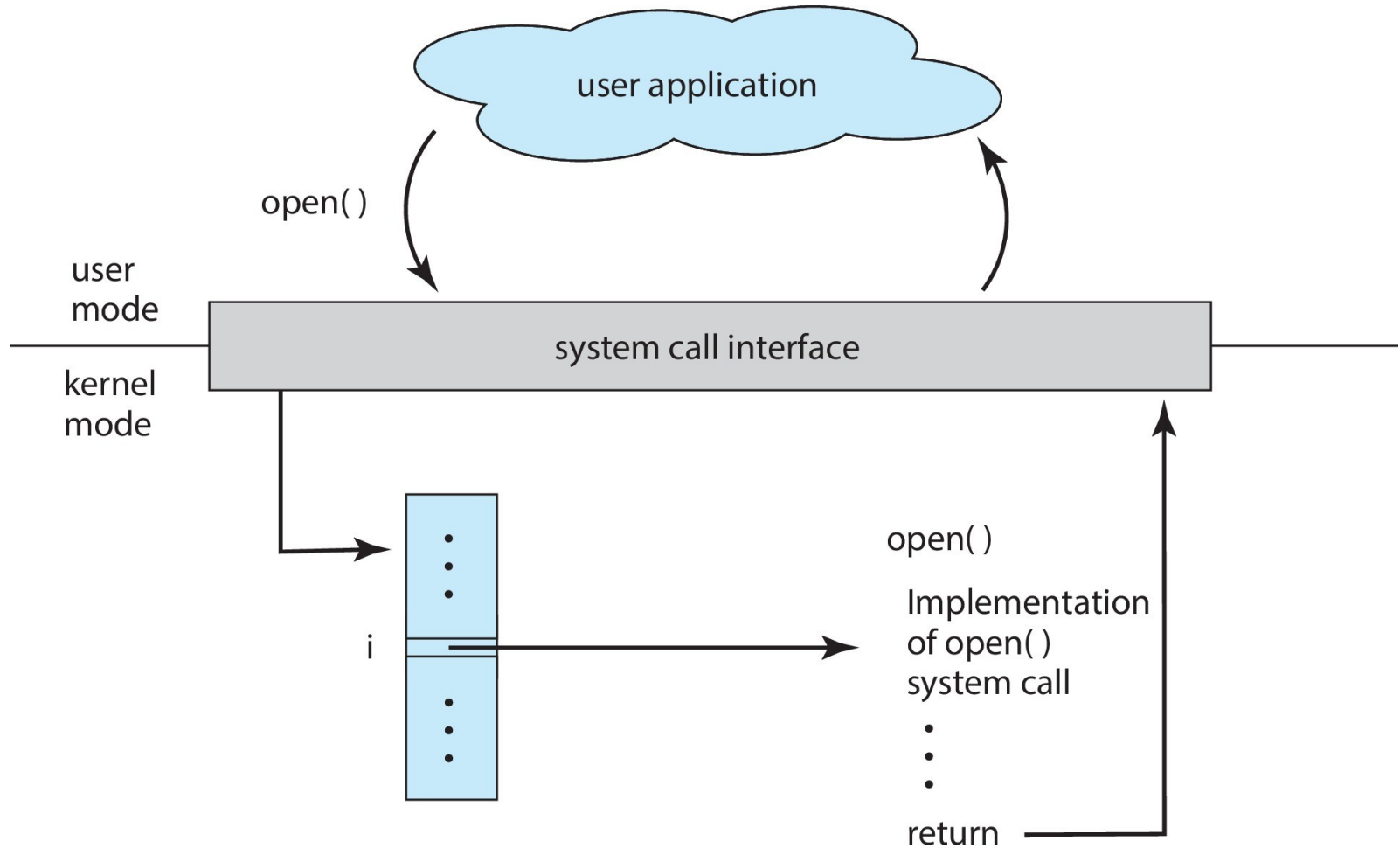
System Call Implementation

- Typically, a number associated with each system call
 - **System-call interface** maintains a table indexed according to these numbers
- The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
 - Just needs to obey API and understand what OS will do as a result call
 - Most details of OS interface hidden from programmer by API
 - ▶ Managed by run-time support library (set of functions built into libraries included with compiler)





API – System Call – OS Relationship





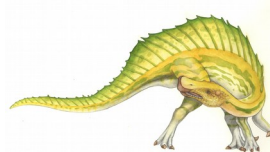
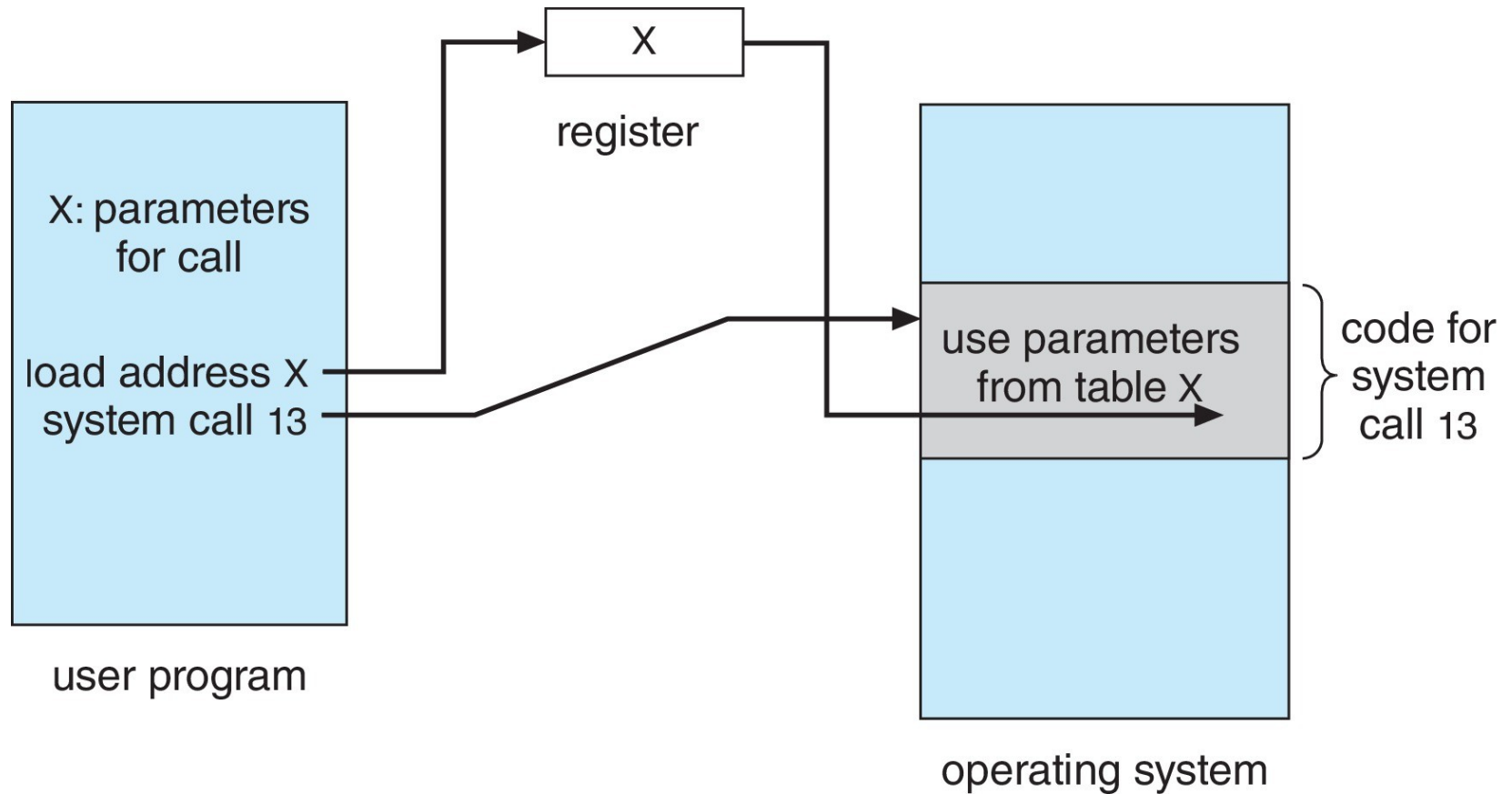
System Call Parameter Passing

- Often, more information is required than simply identity of desired system call
 - Exact type and amount of information vary according to OS and call
- Three general methods used to pass parameters to the OS
 - Simplest: pass the parameters in registers
 - ▶ In some cases, may be more parameters than registers
 - Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
 - ▶ This approach taken by Linux and Solaris
 - Parameters placed, or **pushed**, onto the **stack** by the program and **popped** off the stack by the operating system
 - Block and stack methods do not limit the number or length of parameters being passed





Parameter Passing via Table

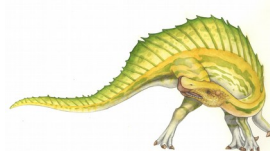




Types of System Calls

■ Process control

- create process, terminate process
- end, abort
- load, execute
- get process attributes, set process attributes
- wait for time
- wait event, signal event
- allocate and free memory
- Dump memory if error
- **Debugger** for determining **bugs, single step** execution
- **Locks** for managing access to shared data between processes





Types of System Calls (cont.)

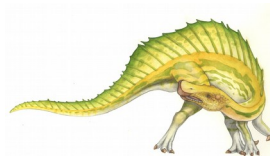
- File management
 - create file, delete file
 - open, close file
 - read, write, reposition
 - get and set file attributes
- Device management
 - request device, release device
 - read, write, reposition
 - get device attributes, set device attributes
 - logically attach or detach devices





Types of System Calls (Cont.)

- Information maintenance
 - get time or date, set time or date
 - get system data, set system data
 - get and set process, file, or device attributes
- Communications
 - create, delete communication connection
 - send, receive messages if **message passing model** to **host name** or **process name**
 - ▶ From **client** to **server**
 - **Shared-memory model** create and gain access to memory regions
 - transfer status information
 - attach and detach remote devices





Types of System Calls (Cont.)

- Protection
 - Control access to resources
 - Get and set permissions
 - Allow and deny user access



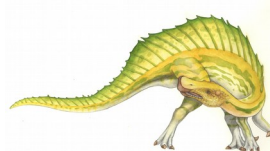


Examples of Windows and Unix System Calls

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

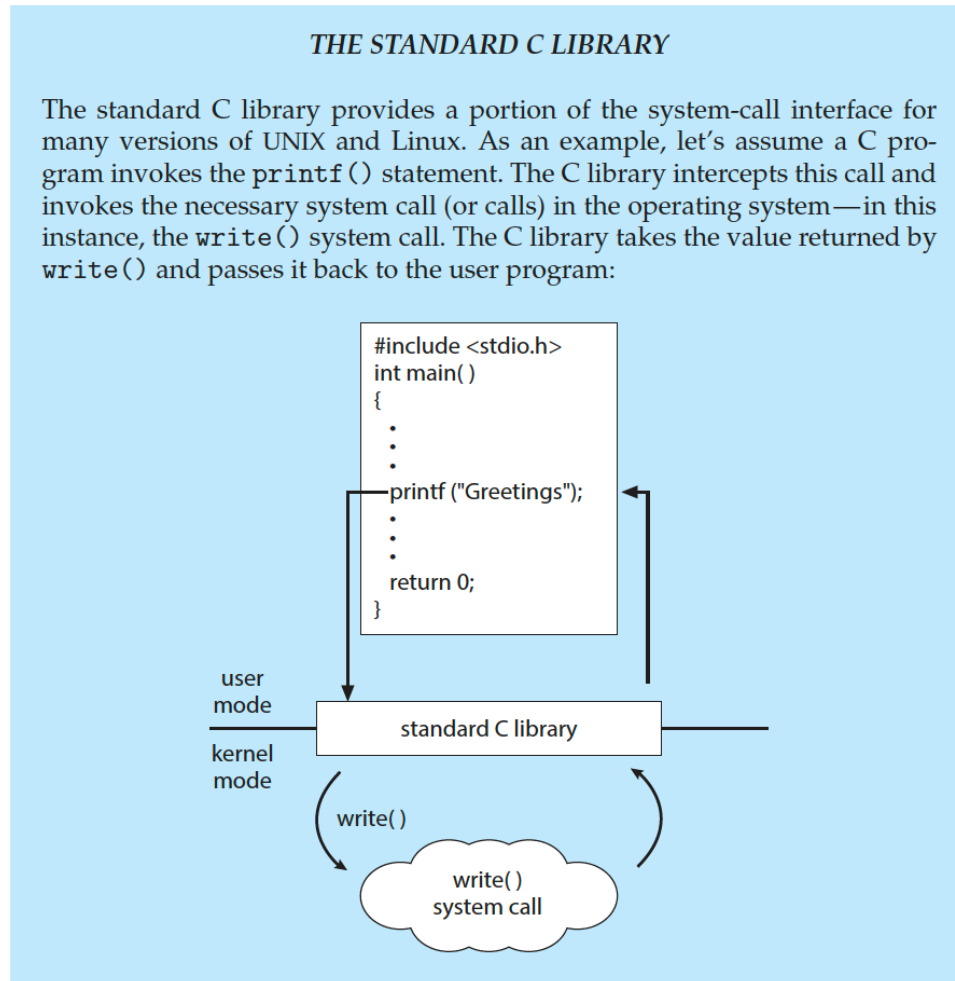
	Windows	Unix
Process control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communications	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()





Standard C Library Example

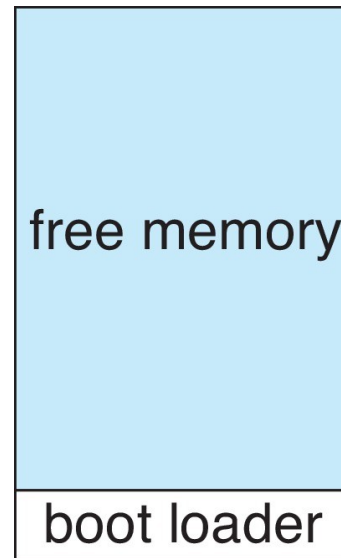
- C program invoking `printf()` library call, which calls `write()` system call





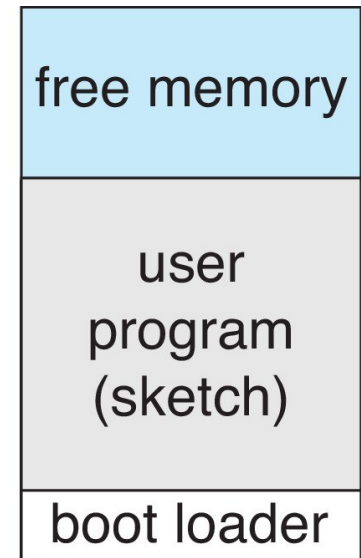
Example: Arduino

- Single-tasking
- No operating system
- Programs (sketch) loaded via USB into flash memory
- Single memory space
- Boot loader loads program
- Program exit -> shell reloaded



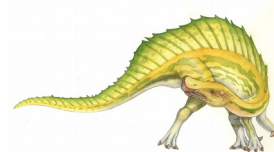
(a)

At system startup



(b)

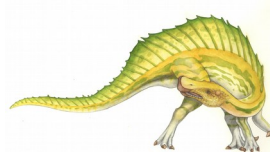
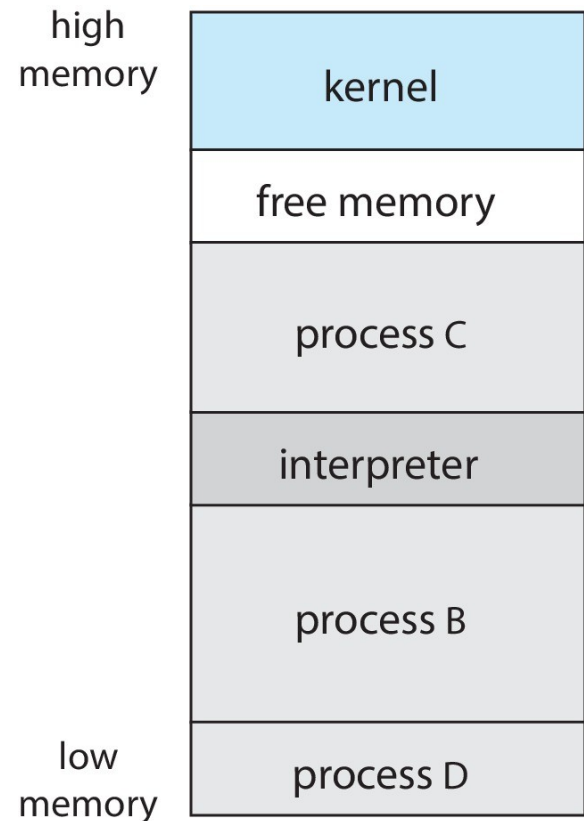
running a program





Example: FreeBSD

- Unix variant
- Multitasking
- User login -> invoke user's choice of shell
- Shell executes `fork()` system call to create process
 - Executes `exec()` to load program into process
 - Shell waits for process to terminate or continues with user commands
- Process exits with:
 - `code = 0` – no error
 - `code > 0` – error code





System Services

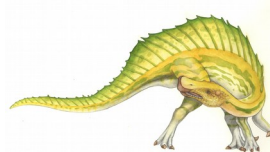
- System programs provide a convenient environment for program development and execution. They can be divided into categories:
 - File manipulation
 - Status information sometimes stored in a file
 - Programming language support
 - Program loading and execution
 - Communications
 - Background services
 - Application programs
- Most users' view of the operation system is defined by system programs, not the actual system calls





System Services (cont.)

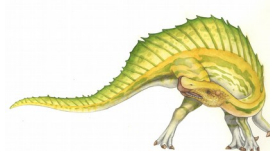
- Provide a convenient environment for program development and execution
 - Some of them are simply user interfaces to system calls; others are considerably more complex
- **File management** - Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories
- **Status information**
 - Some ask the system for info - date, time, amount of available memory, disk space, number of users
 - Others provide detailed performance, logging, and debugging information
 - Typically, these programs format and print the output to the terminal or other output devices
 - Some systems implement a **registry** - used to store and retrieve configuration information





System Services (Cont.)

- **File modification**
 - Text editors to create and modify files
 - Special commands to search contents of files or perform transformations of the text
- **Programming-language support** - Compilers, assemblers, debuggers and interpreters sometimes provided
- **Program loading and execution**- Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language
- **Communications** - Provide the mechanism for creating virtual connections among processes, users, and computer systems
 - Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another





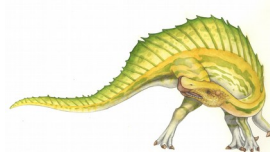
System Services (Cont.)

■ Background Services

- Launch at boot time
 - ▶ Some for system startup, then terminate
 - ▶ Some from system boot to shutdown
- Provide facilities like disk checking, process scheduling, error logging, printing
- Run in user context not kernel context
- Known as **services**, **subsystems**, **daemons**

■ Application programs

- Don't pertain to system
- Run by users
- Not typically considered part of OS
- Launched by command line, mouse click, finger poke





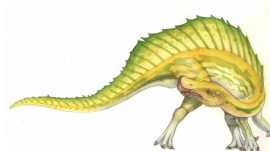
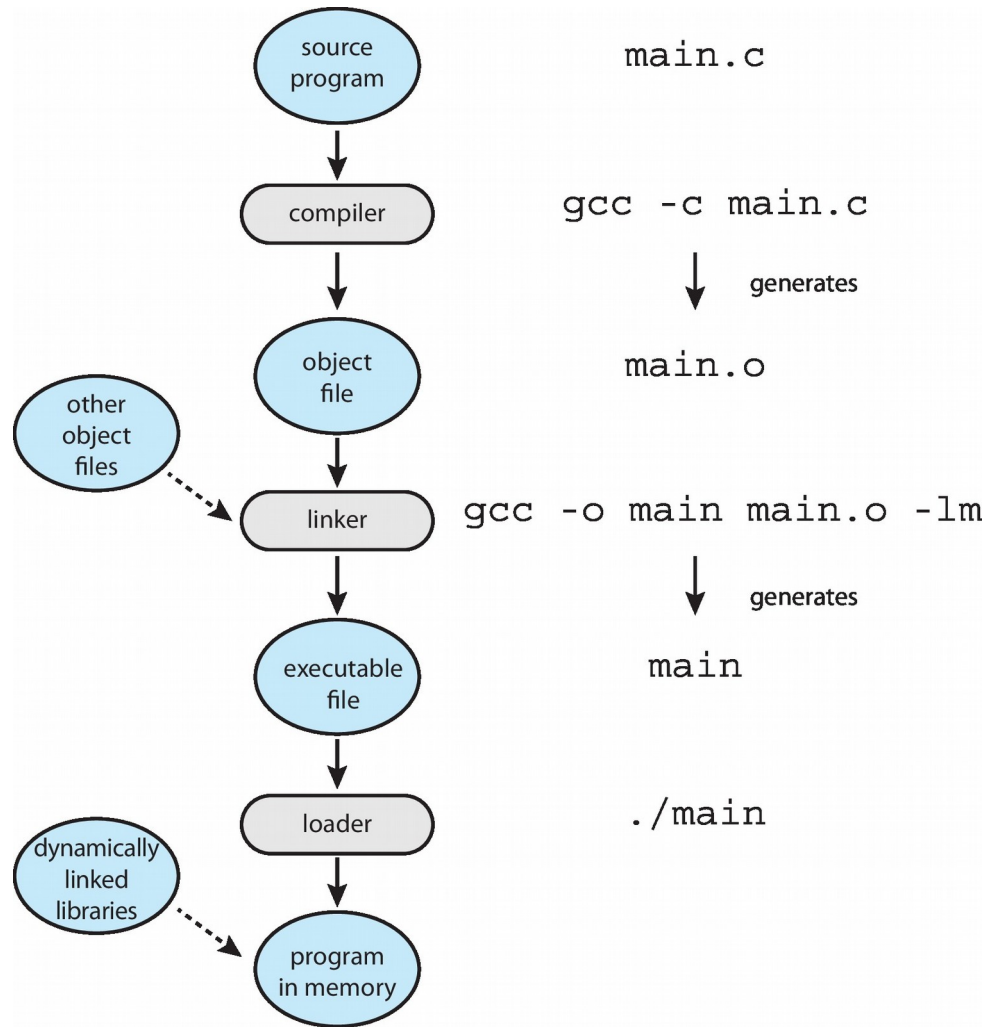
Linkers and Loaders

- Source code compiled into object files designed to be loaded into any physical memory location – **relocatable object file**
- **Linker** combines these into single binary **executable** file
 - Also brings in libraries
- Program resides on secondary storage as binary executable
- Must be brought into memory by **loader** to be executed
 - **Relocation** assigns final addresses to program parts and adjusts code and data in program to match those addresses
- Modern general purpose systems don't link libraries into executables
 - Rather, **dynamically linked libraries** (in Windows, **DLLs**) are loaded as needed, shared by all that use the same version of that same library (loaded once)
- Object, executable files have standard formats, so operating system knows how to load and start them





The Role of the Linker and Loader





Why Applications are Operating System Specific

- Apps compiled on one system usually not executable on other operating systems
- Each operating system provides its own unique system calls
 - Own file formats, etc
- Apps can be multi-operating system
 - Written in interpreted language like Python, Ruby, and interpreter available on multiple operating systems
 - App written in language that includes a VM containing the running app (like Java)
 - Use standard language (like C), compile separately on each operating system to run on each
- **Application Binary Interface (ABI)** is architecture equivalent of API, defines how different components of binary code can interface for a given operating system on a given architecture, CPU, etc





Operating System Design and Implementation

- Design and Implementation of OS not “solvable”, but some approaches have proven successful
- Internal structure of different Operating Systems can vary widely
- Start the design by defining goals and specifications
- Affected by choice of hardware, type of system
- **User** goals and **System** goals
 - User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast
 - System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient





Operating System Design and Implementation (Cont.)

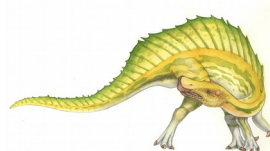
- Important principle to separate
Policy: *What* will be done?
Mechanism: *How* to do it?
- Mechanisms determine how to do something, policies decide what will be done
- The separation of policy from mechanism is a very important principle, it allows maximum flexibility if policy decisions are to be changed later (example – timer)
- Specifying and designing an OS is highly creative task of **software engineering**





Implementation

- Much variation
 - Early OSes in assembly language
 - Then system programming languages like Algol, PL/1
 - Now C, C++
- Actually usually a mix of languages
 - Lowest levels in assembly
 - Main body in C
 - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts
- More high-level language easier to **port** to other hardware
 - But slower
- **Emulation** can allow an OS to run on non-native hardware





Operating System Structure

- General-purpose OS is very large program
- Various ways to structure ones
 - Simple structure – MS-DOS
 - More complex -- UNIX
 - Layered – an abstraction
 - Microkernel -Mach

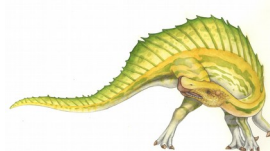




Monolithic Structure – Original UNIX

UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts

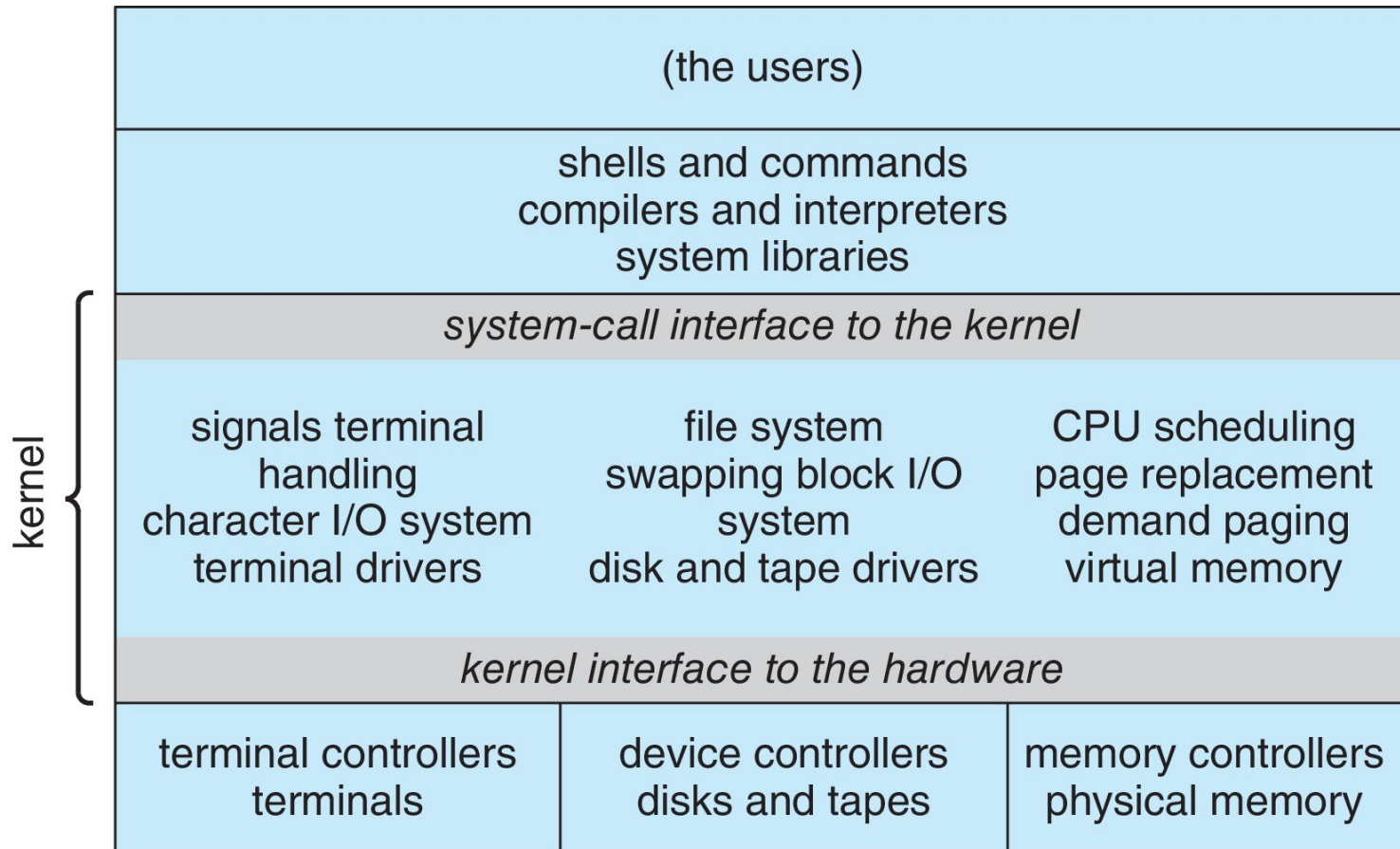
- Systems programs
- The kernel
 - ▶ Consists of everything below the system-call interface and above the physical hardware
 - ▶ Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level





Traditional UNIX System Structure

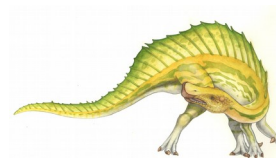
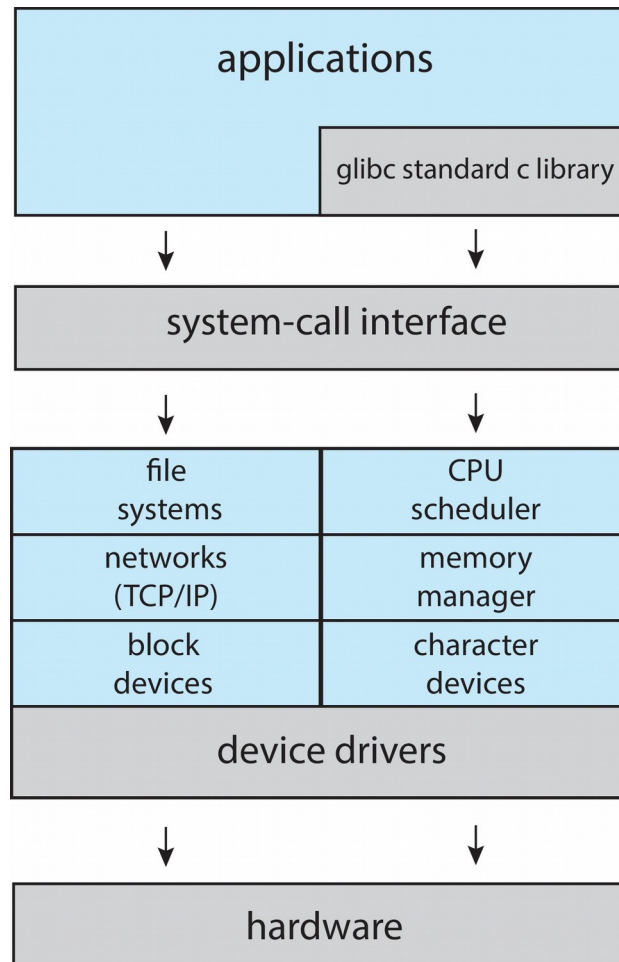
Beyond simple but not fully layered





Linux System Structure

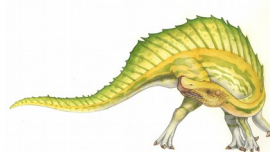
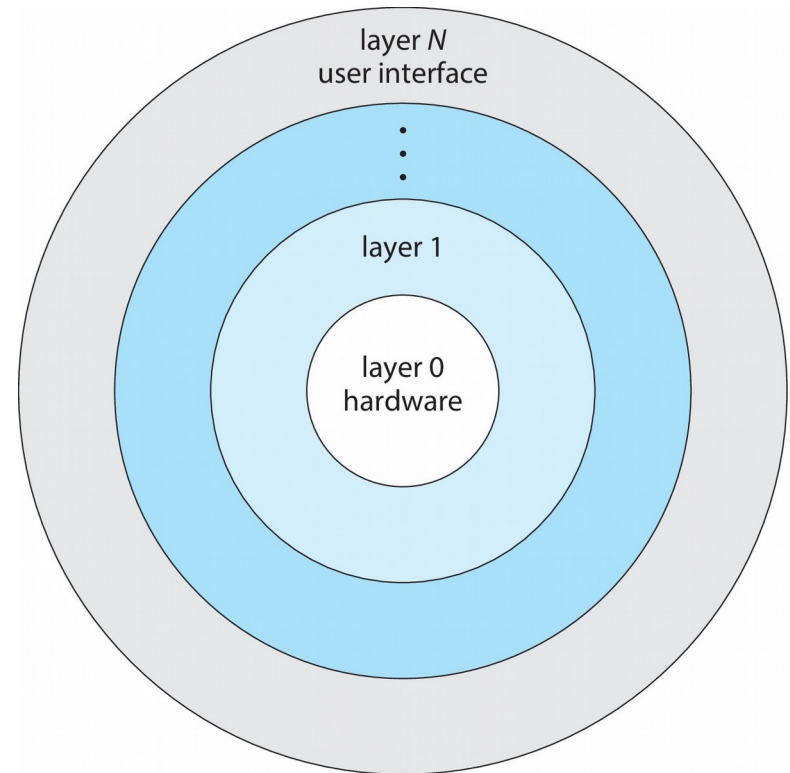
Monolithic plus modular design





Layered Approach

- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers





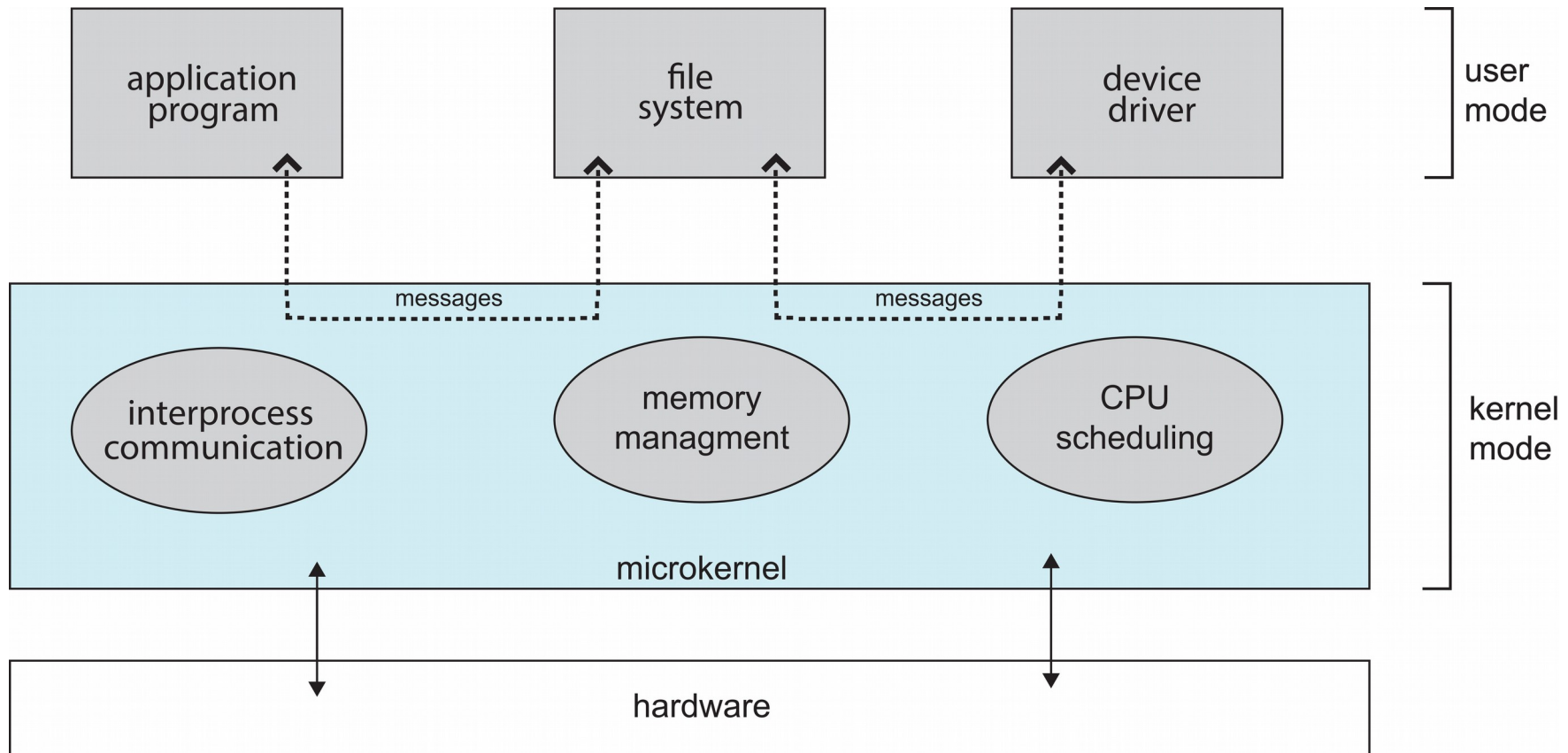
Microkernels

- Moves as much from the kernel into user space
- **Mach** example of **microkernel**
 - Mac OS X kernel (**Darwin**) partly based on Mach
- Communication takes place between user modules using **message passing**
- Benefits:
 - Easier to extend a microkernel
 - Easier to port the operating system to new architectures
 - More reliable (less code is running in kernel mode)
 - More secure
- Detriments:
 - Performance overhead of user space to kernel space communication





Microkernel System Structure

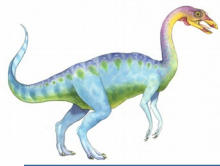




Modules

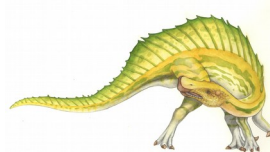
- Many modern operating systems implement **loadable kernel modules (LKMs)**
 - Uses object-oriented approach
 - Each core component is separate
 - Each talks to the others over known interfaces
 - Each is loadable as needed within the kernel
- Overall, similar to layers but with more flexible
 - Linux, Solaris, etc





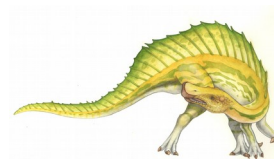
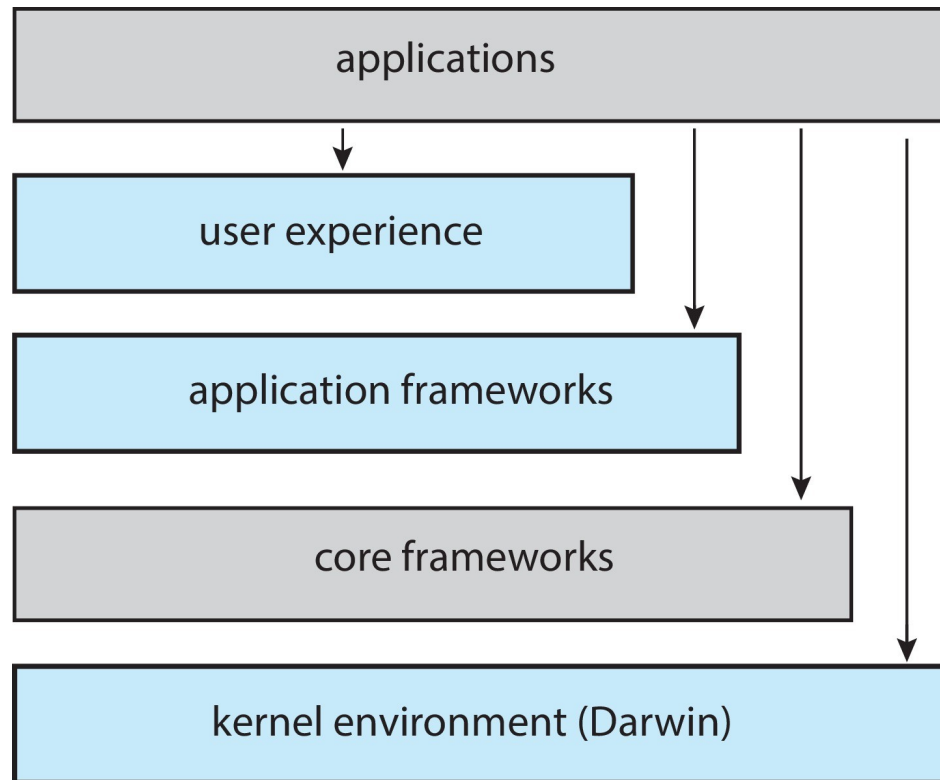
Hybrid Systems

- Most modern operating systems are actually not one pure model
 - Hybrid combines multiple approaches to address performance, security, usability needs
 - Linux and Solaris kernels in kernel address space, so monolithic, plus modular for dynamic loading of functionality
 - Windows mostly monolithic, plus microkernel for different subsystem ***personalities***
- Apple Mac OS X hybrid, layered, **Aqua** UI plus **Cocoa** programming environment
 - Below is kernel consisting of Mach microkernel and BSD Unix parts, plus I/O kit and dynamically loadable modules (called **kernel extensions**)



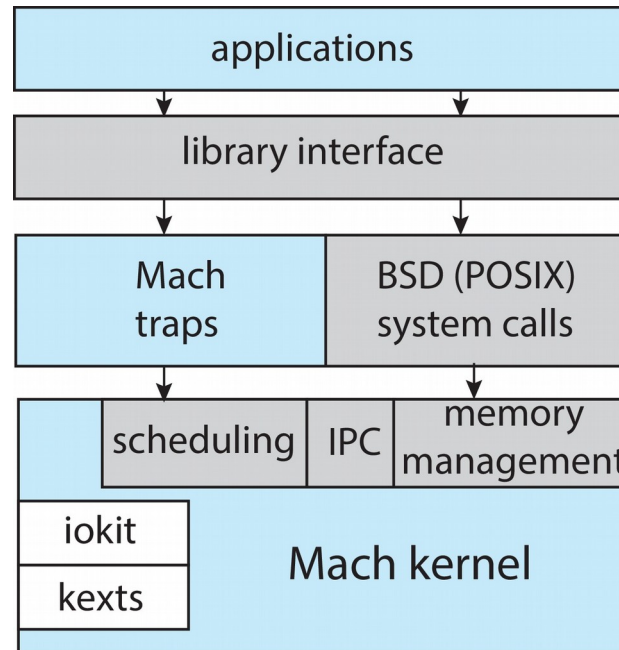


macOS and iOS Structure





Darwin





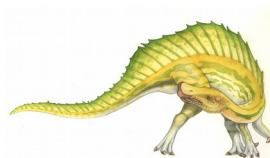
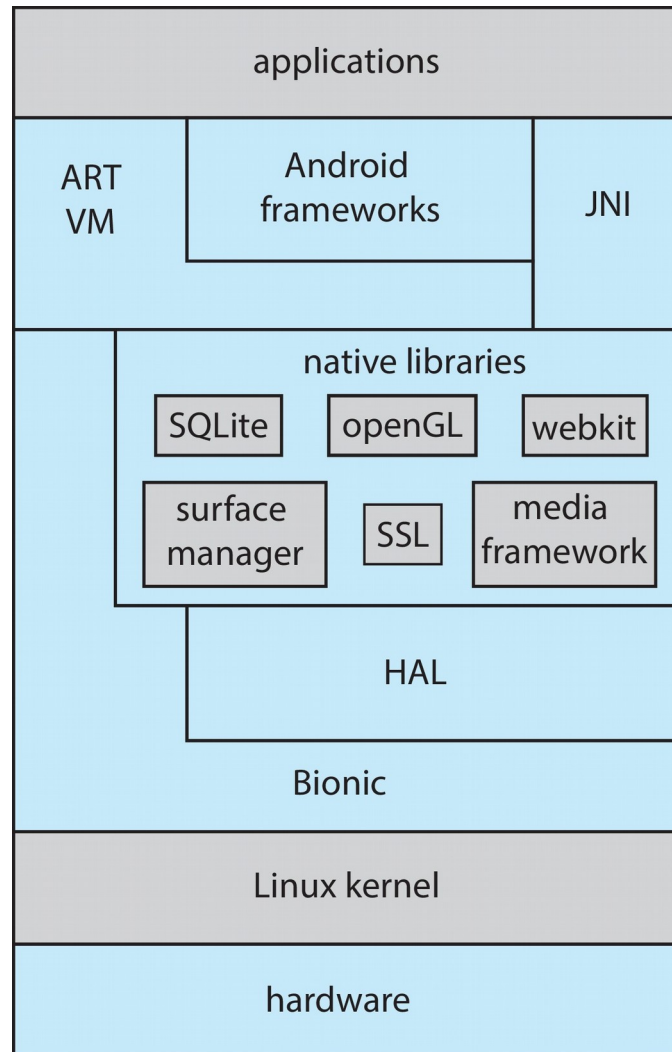
Android

- Developed by Open Handset Alliance (mostly Google)
 - Open Source
- Similar stack to IOS
- Based on Linux kernel but modified
 - Provides process, memory, device-driver management
 - Adds power management
- Runtime environment includes core set of libraries and Dalvik virtual machine
 - Apps developed in Java plus Android API
 - ▶ Java class files compiled to Java bytecode then translated to executable than runs in Dalvik VM
- Libraries include frameworks for web browser (webkit), database (SQLite), multimedia, smaller libc





Android Architecture





System Boot

- When power initialized on system, execution starts at a fixed memory location
- Operating system must be made available to hardware so hardware can start it
 - Small piece of code – **bootstrap loader**, **BIOS**, stored in **ROM** or **EEPROM** locates the kernel, loads it into memory, and starts it
 - Sometimes two-step process where **boot block** at fixed location loaded by ROM code, which loads bootstrap loader from disk
 - Modern systems replace BIOS with **Unified Extensible Firmware Interface (UEFI)**
- Common bootstrap loader, **GRUB**, allows selection of kernel from multiple disks, versions, kernel options
- Kernel loads and system is then **running**
- Boot loaders frequently allow various boot states, such as single user mode





Operating-System Debugging

- **Debugging** is finding and fixing errors, or **bugs**
- Also **performance tuning**
- OS generate **log files** containing error information
- Failure of an application can generate **core dump** file capturing memory of the process
- Operating system failure can generate **crash dump** file containing kernel memory
- Beyond crashes, performance tuning can optimize system performance
 - Sometimes using ***trace listings*** of activities, recorded for analysis
 - **Profiling** is periodic sampling of instruction pointer to look for statistical trends

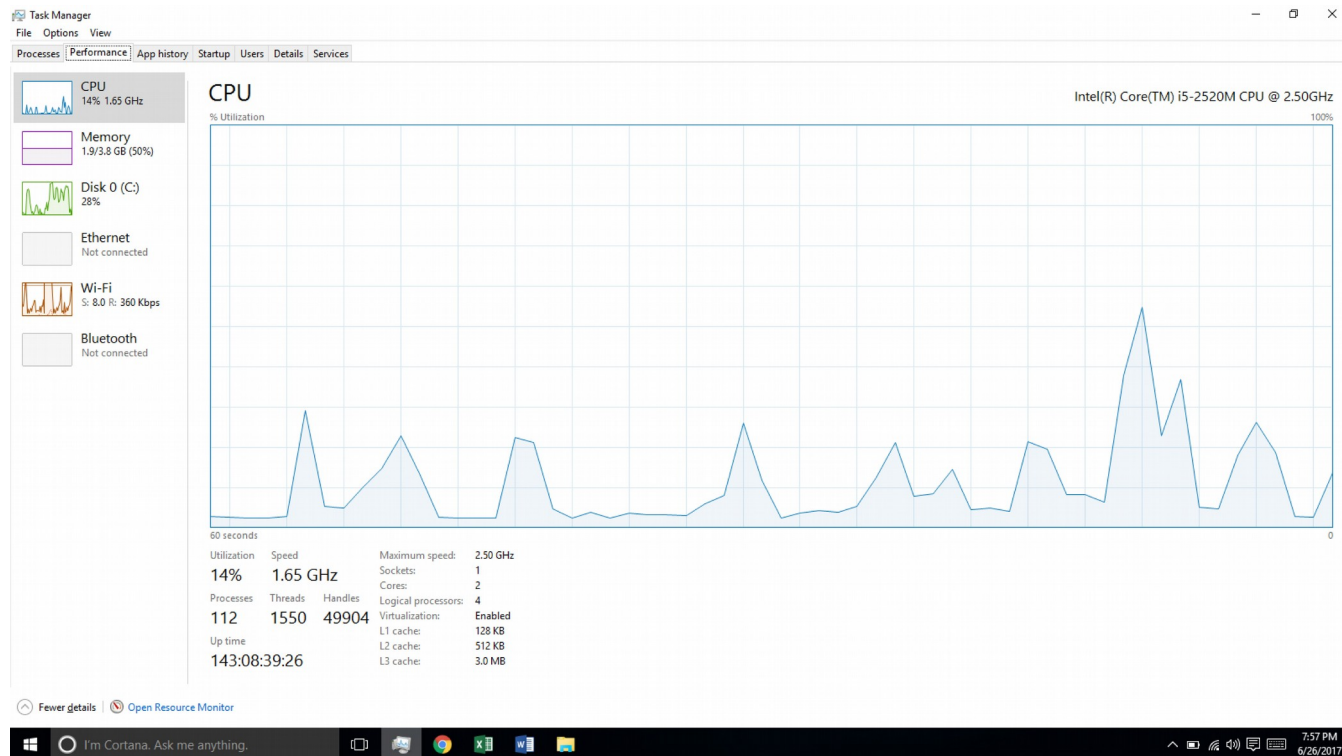
Kernighan's Law: "Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."





Performance Tuning

- Improve performance by removing bottlenecks
- OS must provide means of computing and displaying measures of system behavior
- For example, “top” program or Windows Task Manager





Tracing

- Collects data for a specific event, such as steps involved in a system call invocation
- Tools include
 - strace – trace system calls invoked by a process
 - gdb – source-level debugger
 - perf – collection of Linux performance tools
 - tcpdump – collects network packets





BCC

- Debugging interactions between user-level and kernel code nearly impossible without toolset that understands both and an instrument their actions
- BCC (BPF Compiler Collection) is a rich toolkit providing tracing features for Linux
 - See also the original DTrace
- For example, disksnoop.py traces disk I/O activity

TIME(s)	T	BYTES	LAT(ms)
1946.29186700	R	8	0.27
1946.33965000	R	8	0.26
1948.34585000	W	8192	0.96
1950.43251000	R	4096	0.56
1951.74121000	R	4096	0.35

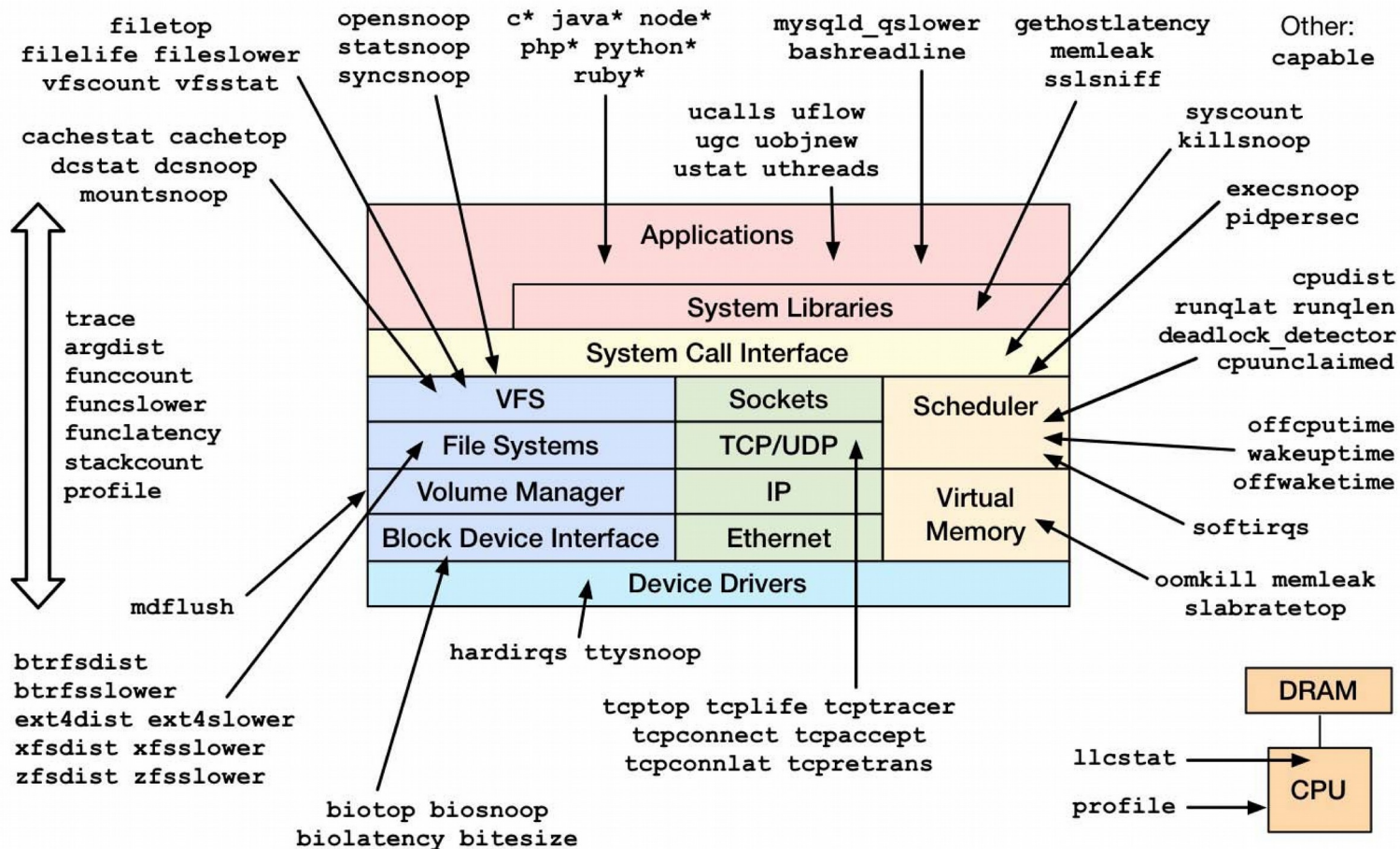
- Many other tools (next slide)





Linux bcc/BPF Tracing Tools

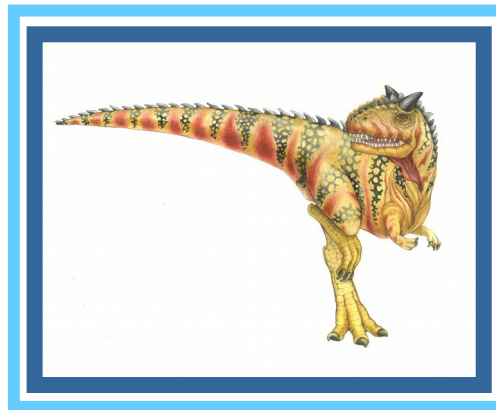
Linux bcc/BPF Tracing Tools



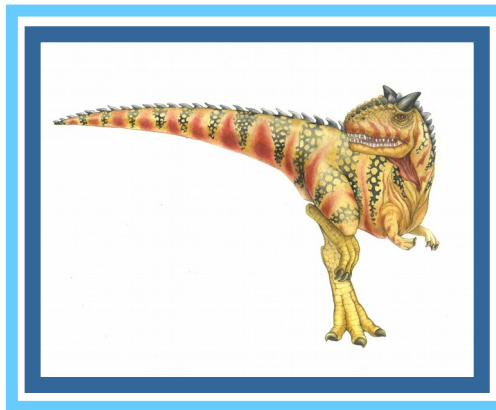
<https://github.com/iovisor/bcc#tools> 2017



End of Chapter 2



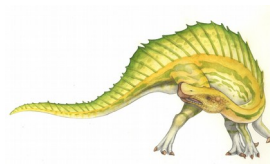
Chapter 3: Processes





Chapter 3: Processes

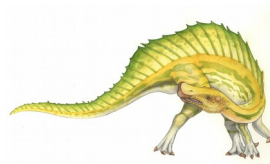
- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- IPC in Shared-Memory Systems
- IPC in Message-Passing Systems
- Examples of IPC Systems





Objectives

- Identify the separate components of a process and illustrate how they are represented and scheduled in an operating system.
- Describe how processes are created and terminated in an operating system, including developing programs using the appropriate system calls that perform these operations.
- Describe and contrast inter-process communication using shared memory and message passing.
- Understand kernel modules that interact with the Linux operating system.





Process Concept

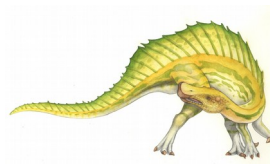
- An operating system executes a variety of programs that run as a **process**.
- **A process is a program in execution.**
- Multiple parts in a **process image**:
 - The program code, also called **text section**
 - Current processor state, including **program counter**, processor registers, including stack pointer, etc.
 - **Stack contents**, containing temporary data
 - ▶ Function parameters, return addresses, local variables
 - **Data section** containing global variables
 - **Heap** containing memory dynamically allocated during run time
 - Other resources such as open files, command-line arguments, environment values, ...





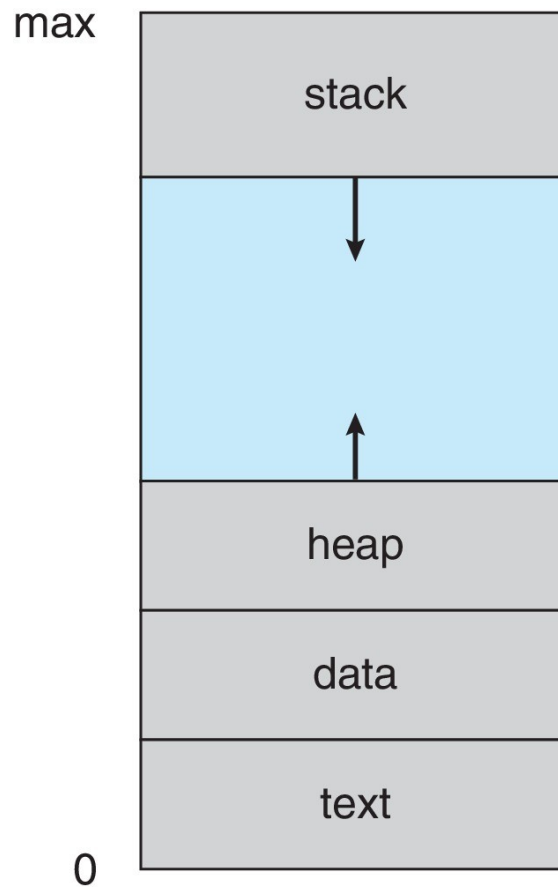
Process Concept (Cont.)

- Program is **passive** entity stored on disk (**executable file**); process is **active** -
 - Program becomes process when executable file loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc
- One program can be executed by several processes
 - e.g.: compiler, shell (bash), browser, etc.



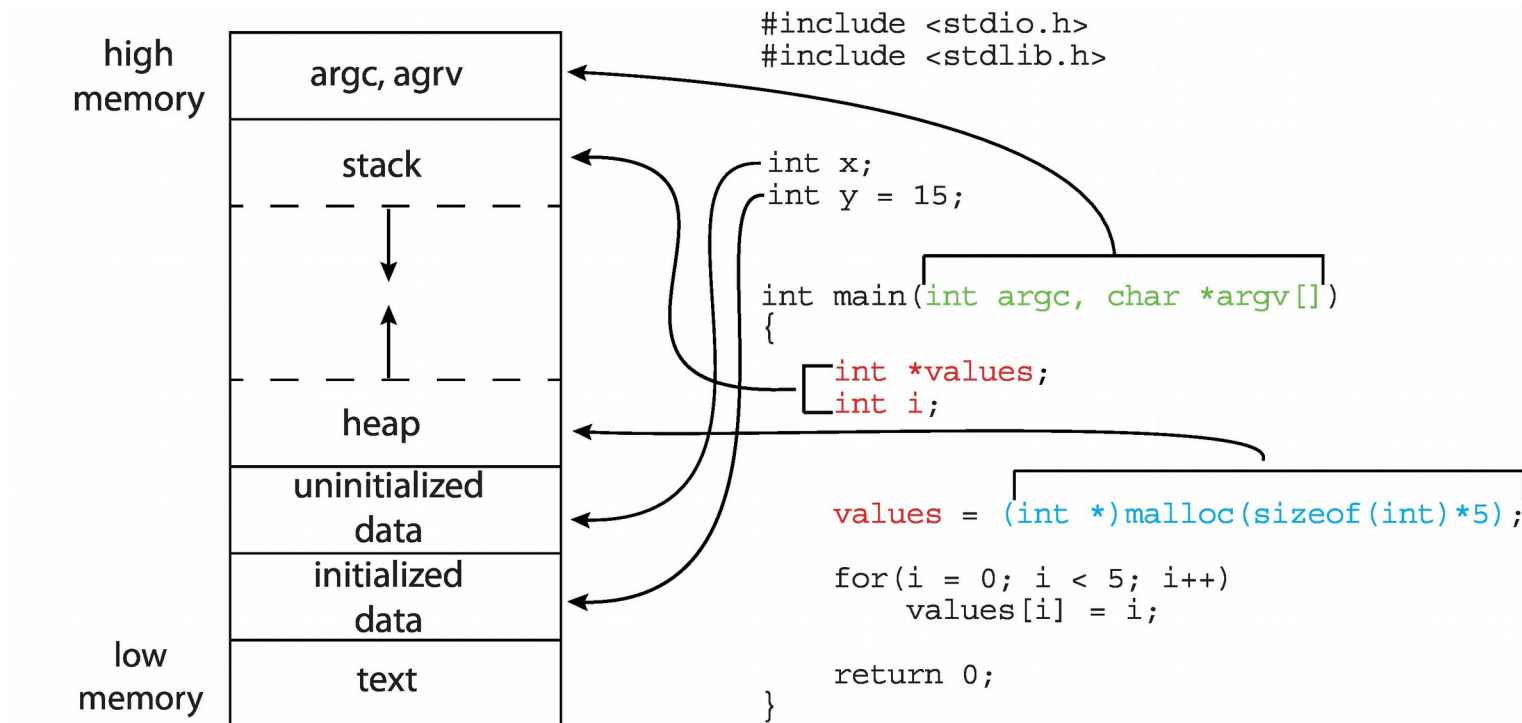


Process in Memory (simplified)



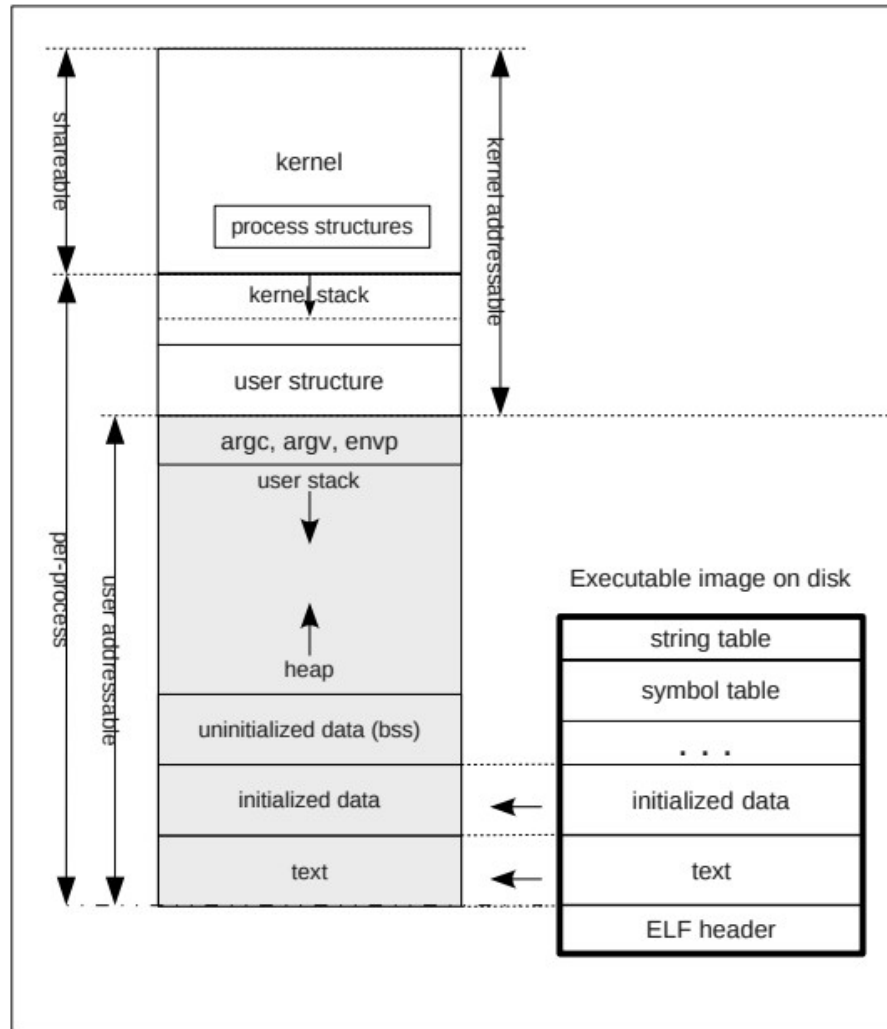


Memory Layout of a C Program





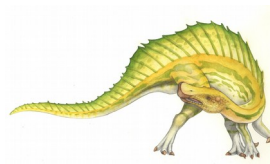
Detailed ELF Memory Layout





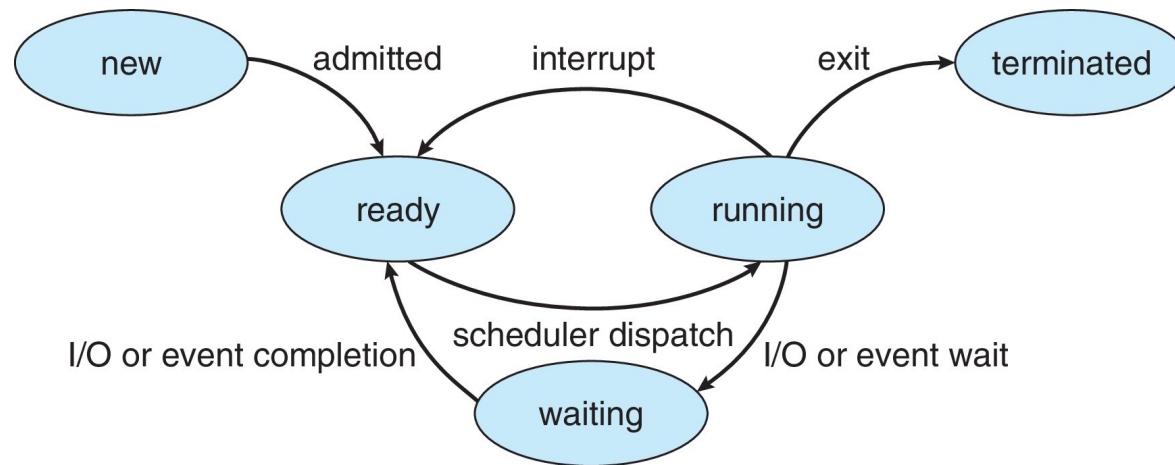
Process State

- As a process executes, it changes **state**
 - **New:** The process was just created
 - **Ready:** The process is ready to run but is waiting to be assigned to a processor
 - **Running:** Instructions are being executed
 - **Waiting:** The process is waiting for some event to occur and is not able to use the processor
 - **Terminated:** The process has finished execution





Process State Transition Diagram



- Edges are transitions.
- Their labels are the actions or events that cause these transitions.





Process Control Block (PCB)

Information associated with each process (also called **task control block**)

- Process state – running, waiting, etc
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files

process state
process number
program counter
registers
memory limits
list of open files
...

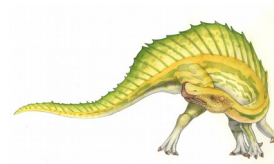
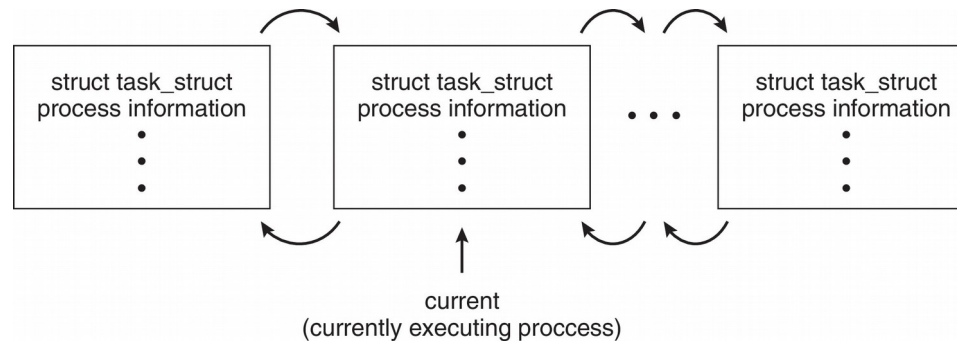




PCB Representation in Linux

Represented by the C structure `task_struct`, part of which is

```
pid t_pid;           /* process identifier */
long state;          /* state of the process */
unsigned int time_slice /* scheduling information */
struct task_struct *parent; /* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files; /* list of open files */
struct mm_struct *mm;   /* address space of this process */
```





Scheduling

- There are three levels of scheduling:
 - **long-term scheduling**: decision about which processes are admitted into system (usually just in batch systems)
 - **medium-term scheduling**: decision about which processes are memory resident
 - **short-term scheduling**: decision about which memory resident process gets the CPU next
- **Short-term scheduler** is also called **process scheduler**





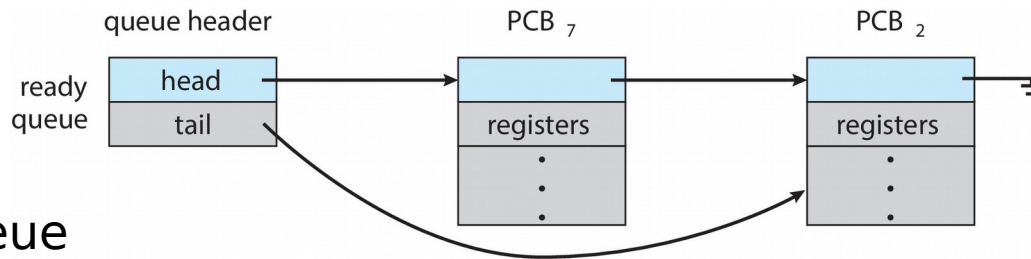
Process Scheduler

- Objective: Maximize CPU utilization, quickly switch processes onto CPU core
- Maintains **scheduling queues** of processes
 - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
 - **Wait queues** – set of processes waiting for an event (i.e. I/O)
 - Processes migrate among the various queues
- Scheduler runs most frequently, so it must be very fast

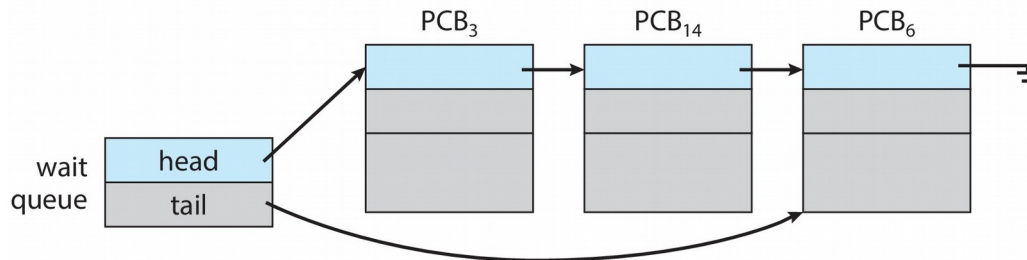




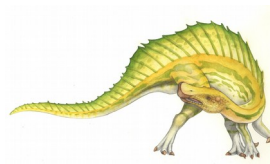
Ready and Wait Queues



One ready queue
for each processor

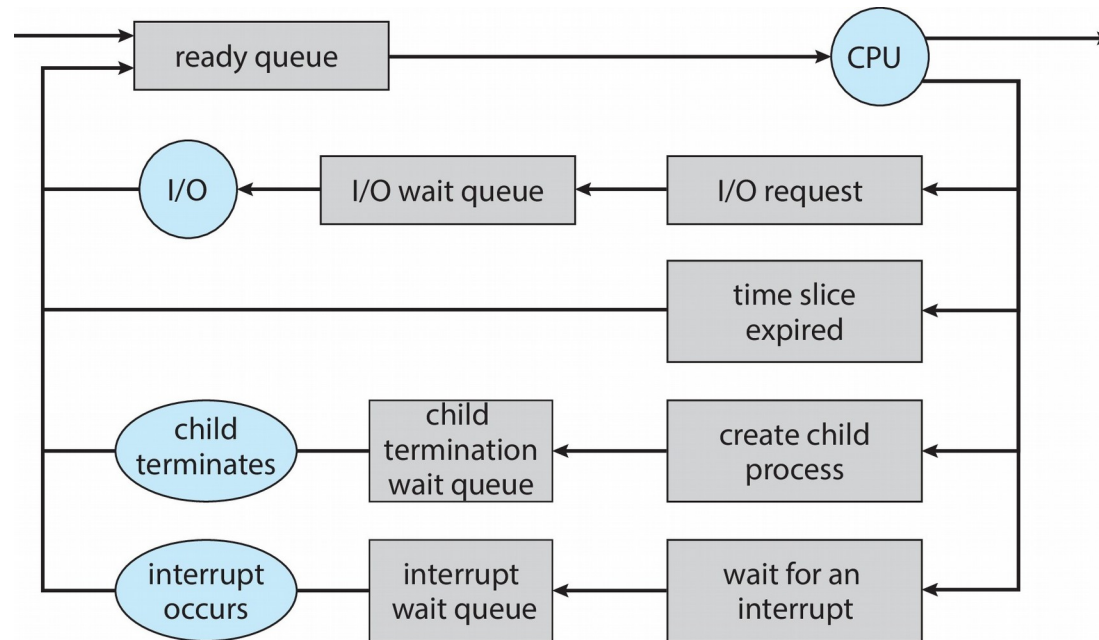


One wait queue for
each device





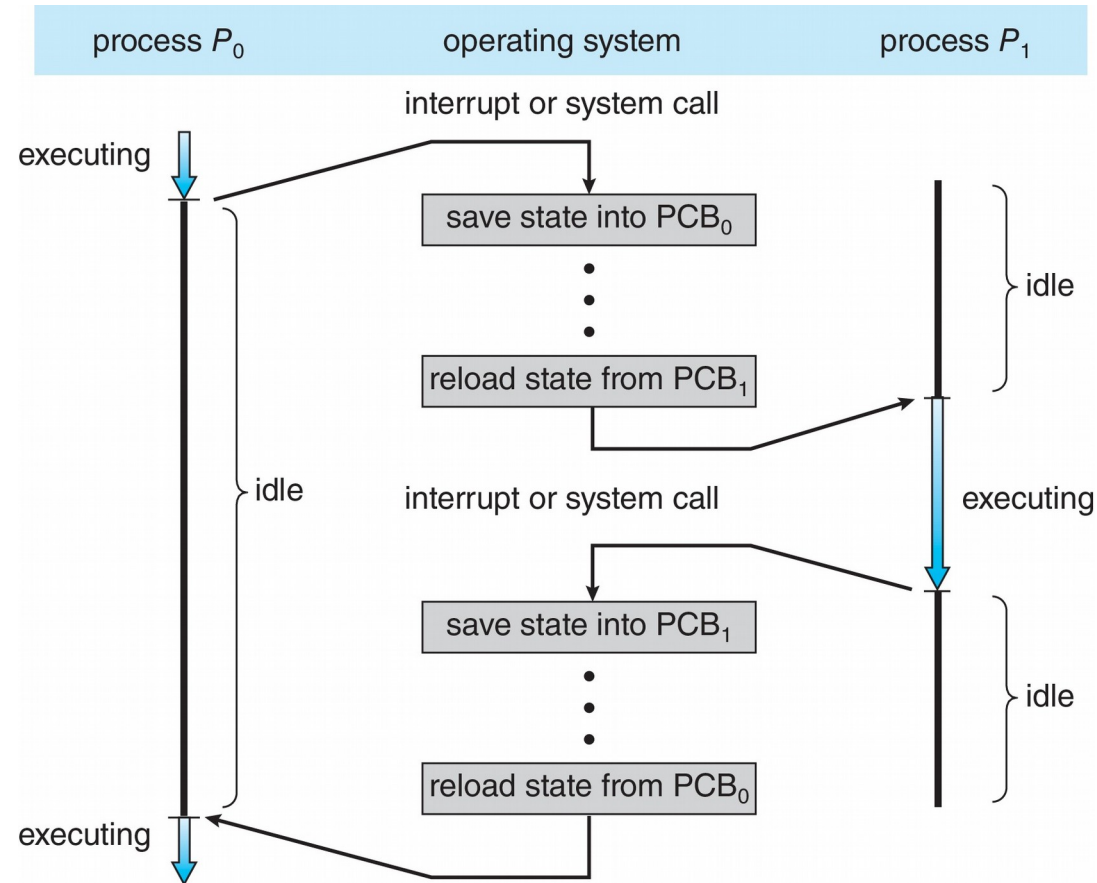
Transitions in Process Scheduling





CPU Switch From Process to Process

A **context switch** occurs when the CPU switches from one process to another.





Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is overhead; the system does no useful work while switching
 - The more complex the OS and the PCB → the longer the context switch
- Time dependent on hardware support
 - Some hardware provides multiple sets of registers per CPU → multiple contexts loaded at once





Operations on Processes

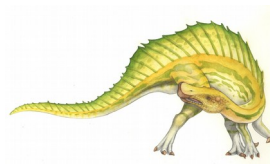
- System must provide mechanisms for:
 - process creation
 - process termination





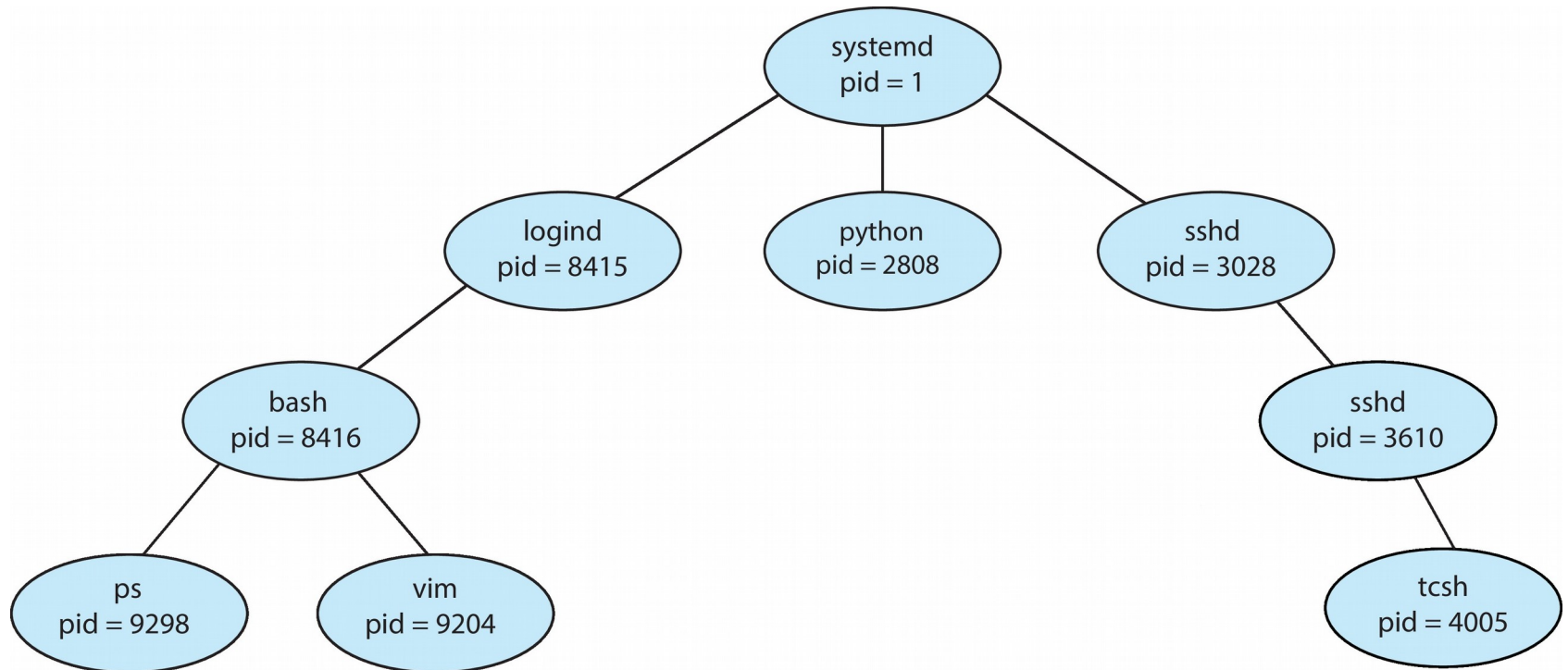
Process Creation

- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, process identified and managed via a **process identifier (pid)**
- Resource sharing options
 - Parent and children share all resources
 - Children share subset of parent's resources
 - Parent and child share no resources
- Execution options
 - Parent and children execute concurrently
 - Parent waits until children terminate





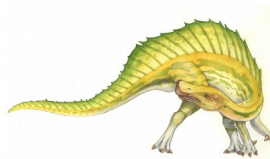
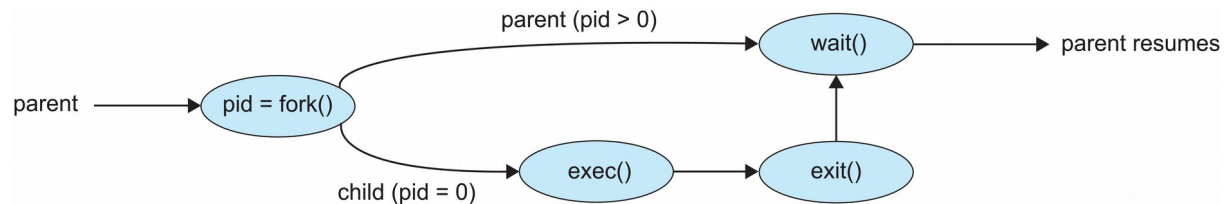
A Tree of Processes in Linux





Process Creation (Cont.)

- Address space
 - Child duplicate of parent
 - Child has a program loaded into it
- UNIX examples
 - **fork()** system call creates new process
 - **exec()** system call used after a **fork()** to replace the process' memory space with a new program
 - Parent process calls **wait()** for the child to terminate





C Program Forking Separate Process

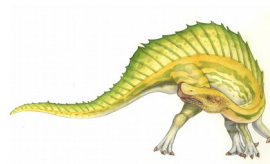
```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }

    return 0;
}
```





Creating a Separate Process via Windows API

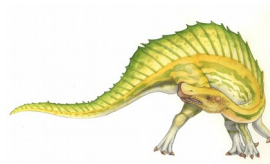
```
#include <stdio.h>
#include <windows.h>

int main(VOID)
{
    STARTUPINFO si;
    PROCESS_INFORMATION pi;

    /* allocate memory */
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    /* create child process */
    if (!CreateProcess(NULL, /* use command line */
        "C:\\WINDOWS\\system32\\mspaint.exe", /* command */
        NULL, /* don't inherit process handle */
        NULL, /* don't inherit thread handle */
        FALSE, /* disable handle inheritance */
        0, /* no creation flags */
        NULL, /* use parent's environment block */
        NULL, /* use parent's existing directory */
        &si,
        &pi))
    {
        fprintf(stderr, "Create Process Failed");
        return -1;
    }
    /* parent will wait for the child to complete */
    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child Complete");

    /* close handles */
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
```





Process Termination

- Process executes last statement and then asks the operating system to delete it using the **exit()** system call.
 - Returns status data from child to parent (via **wait()**)
 - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using signals. Some reasons for doing so:
 - Child has exceeded allocated resources
 - Task assigned to child is no longer required
 - The parent is exiting and the operating systems does not allow a child to continue if its parent terminates



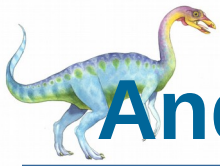


Process Termination 2

- Some operating systems do not allow child to exist if its parent has terminated. If a process terminates, then all its children must also be terminated.
 - **cascading termination.** All children, grandchildren, etc. are terminated.
 - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the **wait()** system call. The call returns status information and the pid of the terminated process

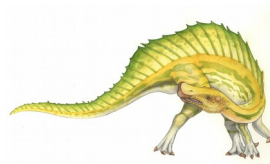
```
pid = wait(&status);
```
- If no parent waiting (did not invoke **wait()**) process is a **zombie**
- If parent terminated without invoking **wait**, process is an **orphan**





Android Process Importance Hierarchy

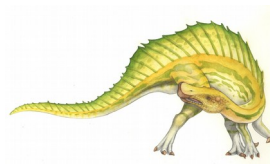
- Mobile operating systems often have to terminate processes to reclaim system resources such as memory. From **most** to **least** important:
 - Foreground process
 - Visible process
 - Service process
 - Background process
 - Empty process
- Android will begin terminating processes that are least important.





Cooperating Processes

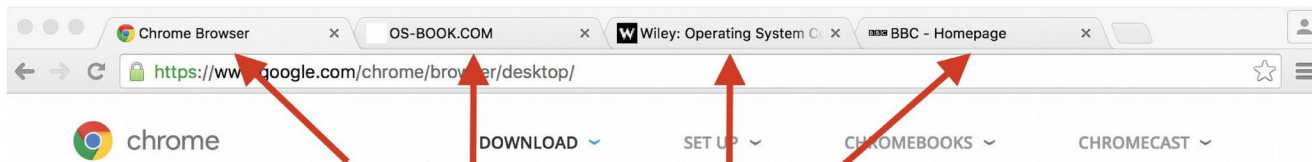
- Two processes can be either *independent* or *cooperating with respect to each other*.
- They are *independent* if neither can affect or be affected by the execution of the other process
- They are *cooperating* if either can affect or be affected by the execution of the other process.
- Various reasons for processes to cooperate:
 - To share information sharing
 - To speed up a computation
 - To increase modularity of an application





Example: Chrome Browser

- Many web browsers run as single process (some still do)
 - If one web site causes trouble, entire browser can hang or crash
- Google Chrome Browser is multiprocess with 3 different types of processes:
 - **Browser** process manages user interface, disk and network I/O
 - **Renderer** process renders web pages, deals with HTML, Javascript. A new renderer created for each website opened
 - ▶ Runs in **sandbox** restricting disk and network I/O, minimizing effect of security exploits
 - **Plug-in** process for each type of plug-in



Each tab represents a separate process.





Interprocess Communication

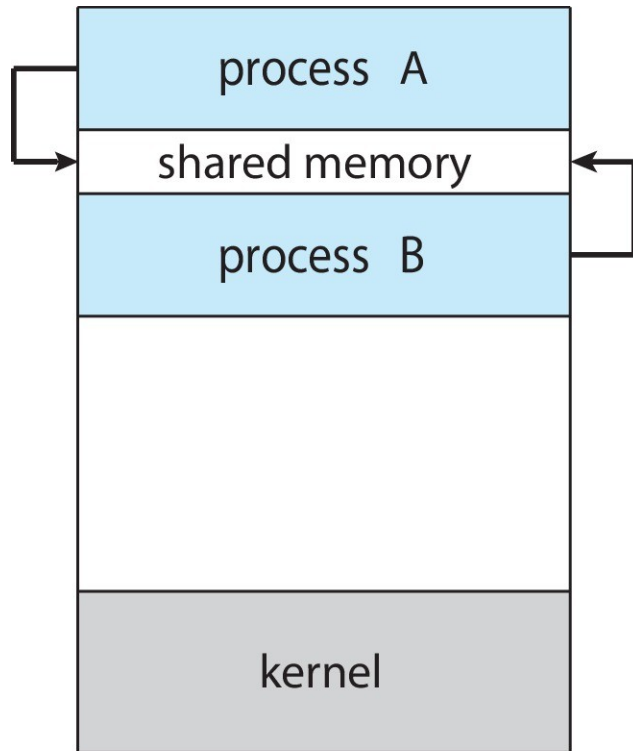
- Cooperating processes need **interprocess communication (IPC)**, a mechanism that allows them to exchange data.
- Two models of IPC:
 - **Shared memory**
 - **Message passing**





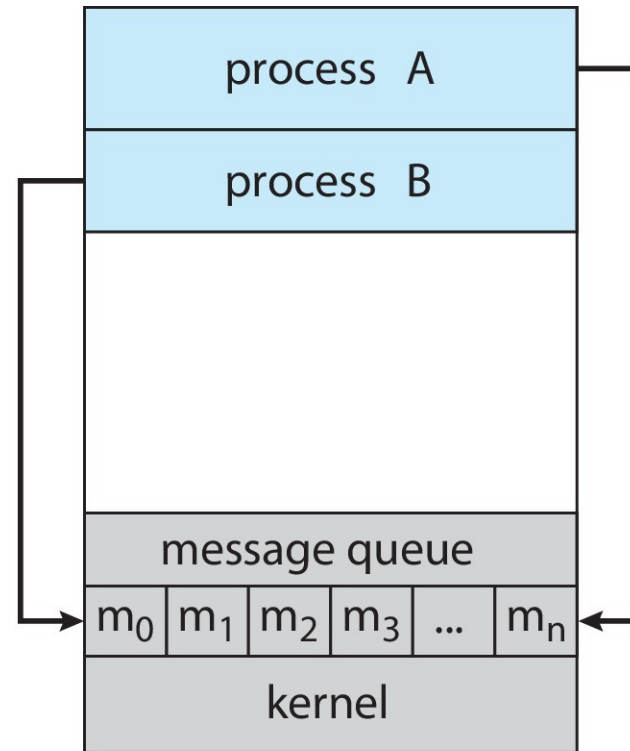
Communications Models

(a) Shared memory.



(a)

(b) Message passing.



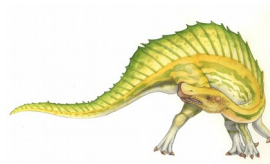
(b)





Interprocess Communication – Shared Memory

- An area of memory shared among the processes that wish to communicate
- With threads (discussed later) sharing memory is easy.
- The communication is under the control of the user processes, not the operating system.
- Processes do not have access to same memory, so operating system must provide mechanism to allow them to create a shared memory region.
- When processes share memory to communicate – **grave danger!** They must synchronize otherwise they risk lots of bad problems (addressed in Chapters 6 and 7).





Producer-Consumer Problem

- Paradigm for cooperating processes, *producer* process produces information that is consumed by a *consumer* process
 - **unbounded-buffer** places no practical limit on the size of the buffer
 - **bounded-buffer** assumes that there is a fixed buffer size
- No buffer means they run in lockstep; as buffer size increases, less need for synchronizing.
- Examples:
 - printer is consumer; word processor is producer
 - compiler produces assembler code; assembler consumes it, producing machine code.
 - pipeline:
 - ▶ `grep expr file | sort | uniq`
 - ▶ command to left is producer for command to right of pipe





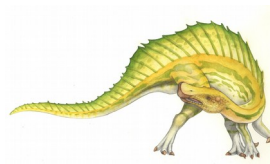
Bounded-Buffer – Shared-Memory Solution

- Shared data

```
#define BUFFER_SIZE 10
typedef struct {
    . . .
} item;

item buffer[BUFFER_SIZE];
/* Initialization */
int in = 0;
int out = 0;
```

- Assumes processes somehow access shared **buffer** and shared variables **in** and **out**
- This solution uses **BUFFER_SIZE-1** elements: treats buffer as a circular queue.
- **in == out** iff **buffer** is empty
- **(in +1) % BUFFER_SIZE == out** iff **buffer** is full

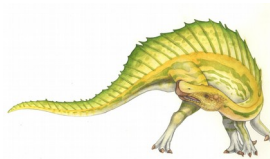


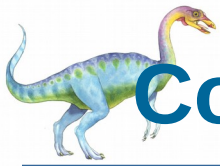


Producer Process – Shared Memory

```
item next_produced; /* local var in producer */

while (true) {
    /* produce an item in next_produced */
    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing (full condition) */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE; /* advance in */
}
```





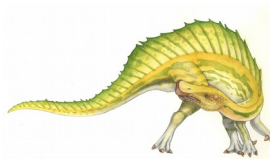
Consumer Process – Shared Memory

```
item next_consumed; /* local var in consumer */

while (true) {
    while (in == out)
        ; /* do nothing (empty condition) */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;

    /* consume the item in next_consumed */
}
```

- Why is this correct?
- What does “correct” mean?
 - No data produced is lost before consumed
 - No data produced is consumed more than once
 - What else?





Interprocess Communication – Message Passing

- Mechanism for processes to communicate and to synchronize their actions
- Message system – processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
 - **send**([destination,] *message*)
 - **receive**([source,] *message*)
- The *message* size is either fixed or variable
- Usually a destination is required by send and usually a source is required by receive.
- Can be used by processes on remote hosts or on same host, so is very general.

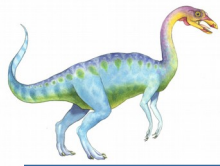




Message Passing (Cont.)

- If processes P and Q wish to communicate, they need to:
 - Establish a **communication link** between them
 - Exchange messages via send/receive
- Implementation issues:
 - How are links established?
 - Can a link be associated with more than two processes?
 - How many links can there be between every pair of communicating processes?
 - What is the capacity of a link?
 - Do links have buffering?
 - Is the size of a message that the link can accommodate fixed or variable?
 - Is a link unidirectional or bi-directional? (Do we need two separate links for messages from P to Q and from Q to P ?)





Message Passing (Cont.)

- Implementation of communication link
 - Physical:
 - ▶ Shared memory
 - ▶ Hardware bus
 - ▶ Network
 - Logical:
 - ▶ Direct or indirect naming of links
 - ▶ Blocking or non-blocking transmission (defined soon)
 - ▶ Symmetric or asymmetric communication (e.g., send is non-blocking but receive is blocking)
 - ▶ Automatic or explicit buffering of link





Direct Communication

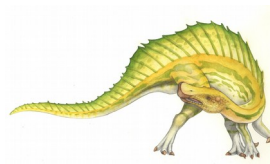
- Processes must name each other explicitly:
 - **send** (P , *message*) – send a message to process P
 - **receive**(Q , *message*) – receive a message from process Q
- Names are bound at compile time
- Properties of communication link
 - Links are established automatically
 - A link is associated with exactly one pair of communicating processes
 - Between each pair there exists exactly one link
 - The link may be unidirectional, but is usually bi-directional
 - Easy to implement
 - Cannot be used for client/server architectures
 - Compile-time binding is very limiting





Indirect Communication

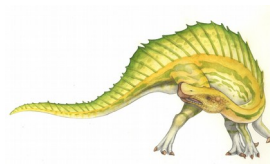
- Messages are directed and received from mailboxes (also referred to as ports)
 - Each mailbox has a unique id
 - Processes can communicate only if they share a mailbox
 - **send** (*A, message*) – send a message to mailbox A
 - **receive**(*A, message*) – receive a message from mailbox A
- Mailboxes might be owned and managed by OS, or by processes.
- Properties of communication link
 - Link established only if processes share a common mailbox
 - A link may be associated with **many** processes
 - Each pair of processes may share several communication links
 - Link may be unidirectional or bi-directional





Indirect Communication

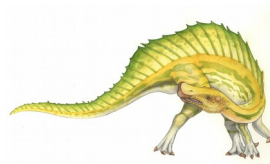
- Operations
 - create a new mailbox (port)
 - send and receive messages through mailbox
 - destroy a mailbox
- Primitives are defined as:
 - send**(*A, message*) – send a message to mailbox *A*
 - receive**(*A, message*) – receive a message from mailbox *A*
- If process creates mailbox, it owns it.
- If process creates child processes they can access sometimes:
 - P creates A
 - P creates child Q
 - Q can receive from or send to A





Indirect Communication

- Mailbox sharing
 - P_1 , P_2 , and P_3 share mailbox A
 - P_1 sends; P_2 and P_3 receive
 - Who gets the message?
- Solutions
 - Allow a link to be associated with at most two processes
 - Allow only one process at a time to execute a receive operation
 - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.
- Many unanswered questions, such as lost messages, faulty communications, process terminations, scrambled messages, etc





Synchronization

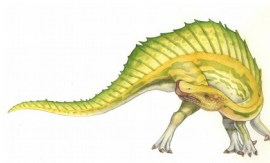
- Message passing may be either blocking or non-blocking
- **Blocking** is considered **synchronous**
 - **Blocking send** -- the sender is blocked until the message is received
 - **Blocking receive** -- the receiver is blocked until a message is available
- **Non-blocking** is considered **asynchronous**
 - **Non-blocking send** -- the sender sends the message and continues without waiting
 - **Non-blocking receive** -- the receiver receives:
 - A valid message, or
 - Null message
- Different combinations possible
 - If both send and receive are blocking, we have a **rendezvous**





Producer – Message Passing

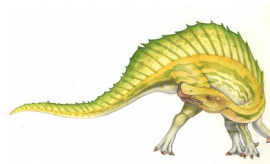
```
message next_produced;  
  
while (true) {  
    /* produce an item in next_produced */  
  
    send(next_produced);  
}
```





Consumer– Message Passing

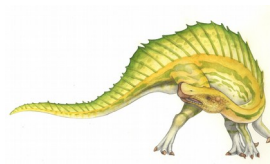
```
message next_consumed;  
  
while (true) {  
    receive(next_consumed)  
  
    /* consume the item in next_consumed */  
}
```





Buffering

- Queue of messages attached to the link.
- Implemented in one of three ways
 1. Zero capacity – no messages are queued on a link.
Sender must wait for receiver (rendezvous)
 2. Bounded capacity – finite length of n messages
Sender must wait if link full
 3. Unbounded capacity – infinite length
Sender never waits

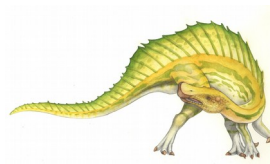


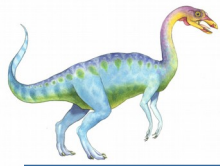


Examples of IPC Systems - POSIX

■ POSIX Shared Memory

- Process first creates shared memory segment
`shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);`
- Also used to open an existing segment
- Set the size of the object
`ftruncate(shm_fd, 4096);`
- Use `mmap()` to memory-map a file pointer to the shared memory object
- Reading and writing to shared memory is done by using the pointer returned by `mmap()`.





IPC POSIX Producer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
    /* the size (in bytes) of shared memory object */
    const int SIZE = 4096;
    /* name of the shared memory object */
    const char *name = "OS";
    /* strings written to shared memory */
    const char *message_0 = "Hello";
    const char *message_1 = "World!";

    /* shared memory file descriptor */
    int shm_fd;
    /* pointer to shared memory object */
    void *ptr;

    /* create the shared memory object */
    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);

    /* configure the size of the shared memory object */
    ftruncate(shm_fd, SIZE);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

    /* write to the shared memory object */
    sprintf(ptr,"%s",message_0);
    ptr += strlen(message_0);
    sprintf(ptr,"%s",message_1);
    ptr += strlen(message_1);

    return 0;
}
```





IPC POSIX Consumer

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
    /* the size (in bytes) of shared memory object */
    const int SIZE = 4096;
    /* name of the shared memory object */
    const char *name = "OS";
    /* shared memory file descriptor */
    int shm_fd;
    /* pointer to shared memory object */
    void *ptr;

    /* open the shared memory object */
    shm_fd = shm_open(name, O_RDONLY, 0666);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    /* read from the shared memory object */
    printf("%s", (char *)ptr);

    /* remove the shared memory object */
    shm_unlink(name);

    return 0;
}
```





Examples of IPC Systems - Mach

- Mach communication is message based
 - Even system calls are messages
 - Each task gets two ports at creation- Kernel and Notify
 - Messages are sent and received using the **mach_msg()** function
 - Ports needed for communication, created via **mach_port_allocate()**
 - Send and receive are flexible, for example four options if mailbox full:
 - ▶ Wait indefinitely
 - ▶ Wait at most n milliseconds
 - ▶ Return immediately
 - ▶ Temporarily cache a message



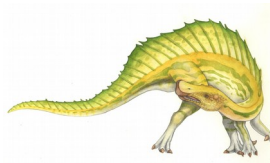


Mach Messages

```
#include<mach/mach.h>

struct message {
    mach_msg_header_t header;
    int data;
};

mach port_t client;
mach port_t server;
```





Mach Message Passing - Client

```
/* Client Code */

struct message message;

// construct the header
message.header.msgh_size = sizeof(message);
message.header.msgh_remote_port = server;
message.header.msgh_local_port = client;

// send the message
mach_msg(&message.header, // message header
        MACH_SEND_MSG, // sending a message
        sizeof(message), // size of message sent
        0, // maximum size of received message - unnecessary
        MACH_PORT_NULL, // name of receive port - unnecessary
        MACH_MSG_TIMEOUT_NONE, // no time outs
        MACH_PORT_NULL // no notify port
    );
```



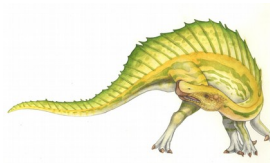


Mach Message Passing - Server

```
    /* Server Code */

    struct message message;

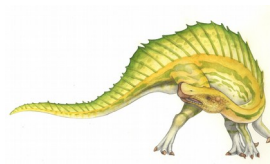
    // receive the message
    mach_msg(&message.header, // message header
             MACH_RCV_MSG, // sending a message
             0, // size of message sent
             sizeof(message), // maximum size of received message
             server, // name of receive port
             MACH_MSG_TIMEOUT_NONE, // no time outs
             MACH_PORT_NULL // no notify port
    );
```





Pipes

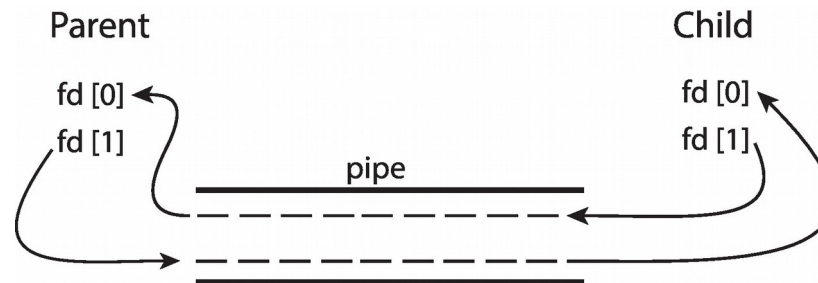
- A **pipe** acts as a conduit allowing two processes to communicate
- Issues:
 - Is communication unidirectional or bidirectional?
 - In the case of two-way communication, is it half or full-duplex?
 - Must there exist a relationship (i.e., **parent-child**) between the communicating processes?
 - Can the pipes be used over a network?
- **Ordinary pipes** – cannot be accessed from outside the process that created it. Typically, a parent process creates a pipe and uses it to communicate with a child process that it created.
- **Named pipes** – can be accessed without a parent-child relationship.



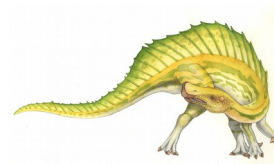


Ordinary Pipes

- **Ordinary Pipes** allow communication in standard producer-consumer style
- Producer writes to one end (the **write-end** of the pipe)
- Consumer reads from the other end (the **read-end** of the pipe)
- Ordinary pipes are therefore unidirectional
- Require parent-child relationship between communicating processes



- Windows calls these **anonymous pipes**





Named Pipes

- **Named Pipes** are more powerful than ordinary pipes
- Communication is bidirectional
- No parent-child relationship is necessary between the communicating processes
- Several processes can use the named pipe for communication
- Provided on both UNIX and Windows systems





Communications in Client-Server Systems

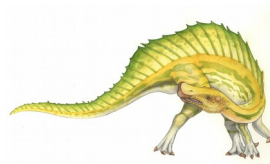
- Sockets
- Remote Procedure Calls





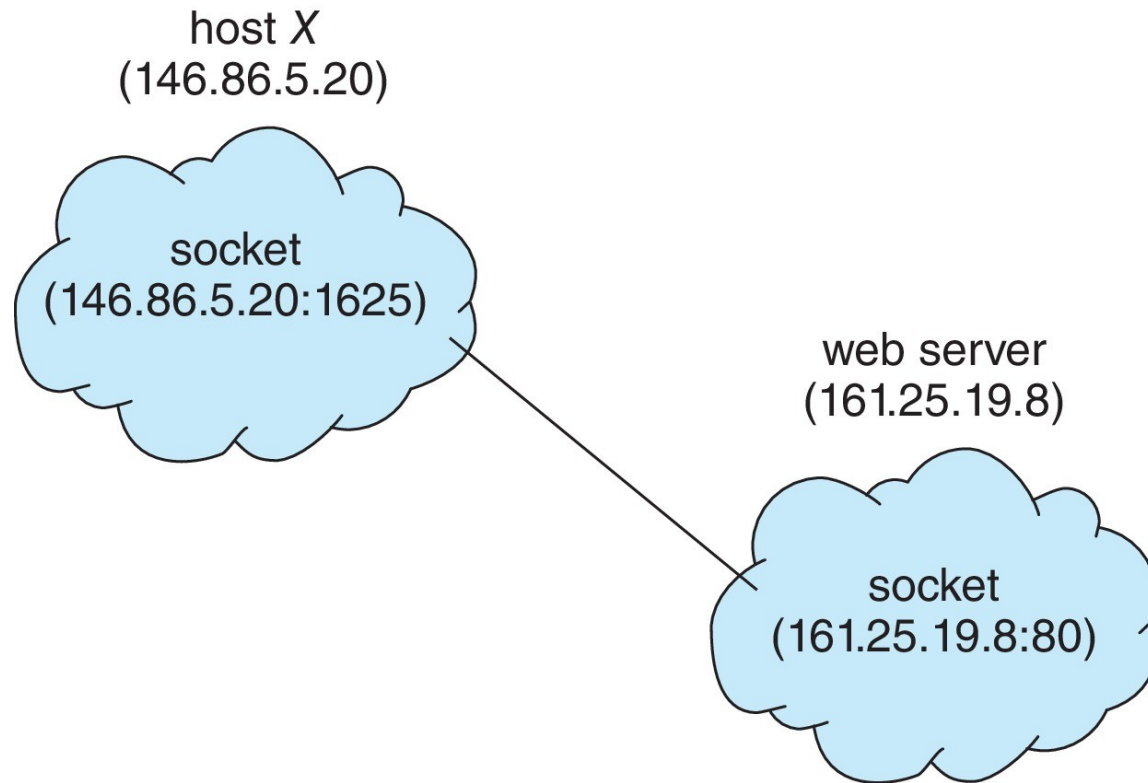
Sockets

- A **socket** is defined as an endpoint for communication
- Concatenation of IP address and **port** – a number included at start of message packet to differentiate network services on a host
- The socket **161.25.19.8:1625** refers to port **1625** on host **161.25.19.8**
- Communication consists between a pair of sockets
- All ports below 1024 are **well known**, used for standard services
- Special IP address 127.0.0.1 (**loopback**) to refer to system on which process is running

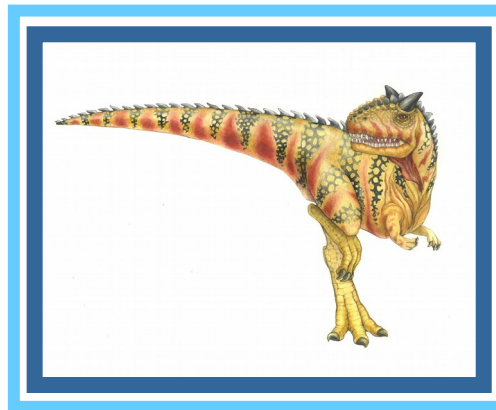




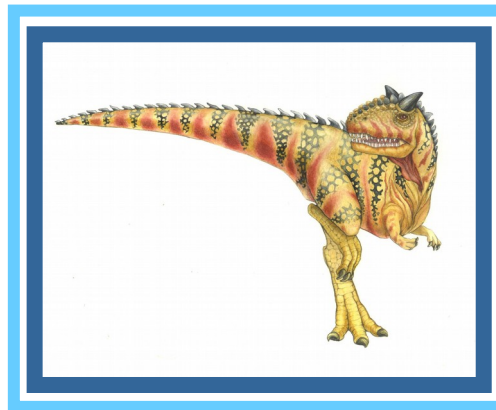
Socket Communication



End of Chapter 3



Chapter 4: Threads & Concurrency





Chapter 4: Threads

- Overview
- Multi-core Programming
- Multi-threading Models
- Thread Libraries
- Implicit Threading
- Threading Issues
- Operating System Examples





Objectives

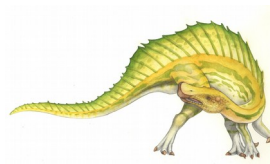
- Identify the basic components of a thread, and contrast threads and processes
- Describe the benefits and challenges of designing multi-threaded applications
- Illustrate different approaches to implicit threading, including thread pools and fork-join
- Describe how the Linux operating system represents threads
- Explore multi-threaded applications using the Pthreads, Java, and Windows threading APIs





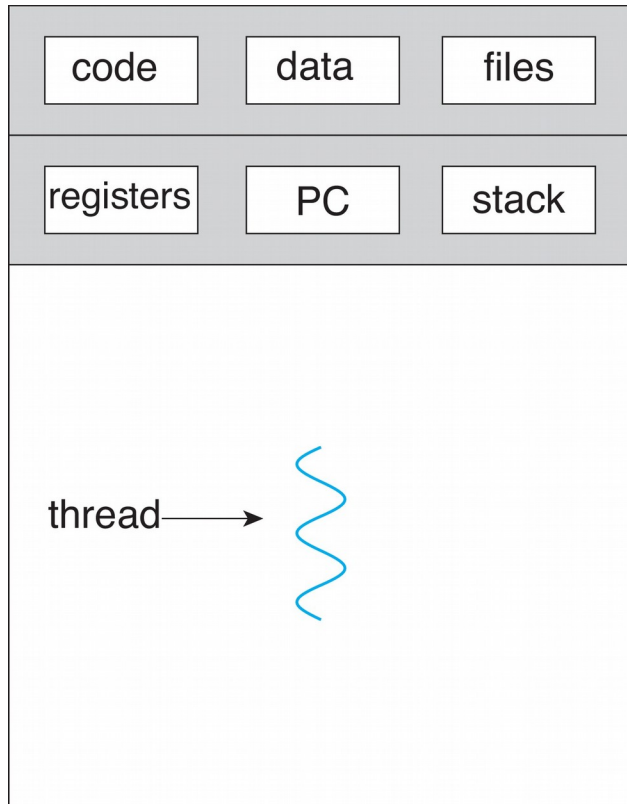
Motivation

- Kernels are generally multi-threaded
- Most modern applications are multi-threaded
- Whereas cooperating processes are independent, cooperating threads run within the same process (think application)
- Multiple functions or tasks within an application can be implemented by separate threads. Example decomposition:
 - A thread to update display
 - A thread to fetch data from a database
 - A thread to run a tool such as a spell-checker
 - A thread to respond to a network request
- Process creation is costly and slow, whereas thread creation is light-weight
- Proper threading can simplify code, increase efficiency

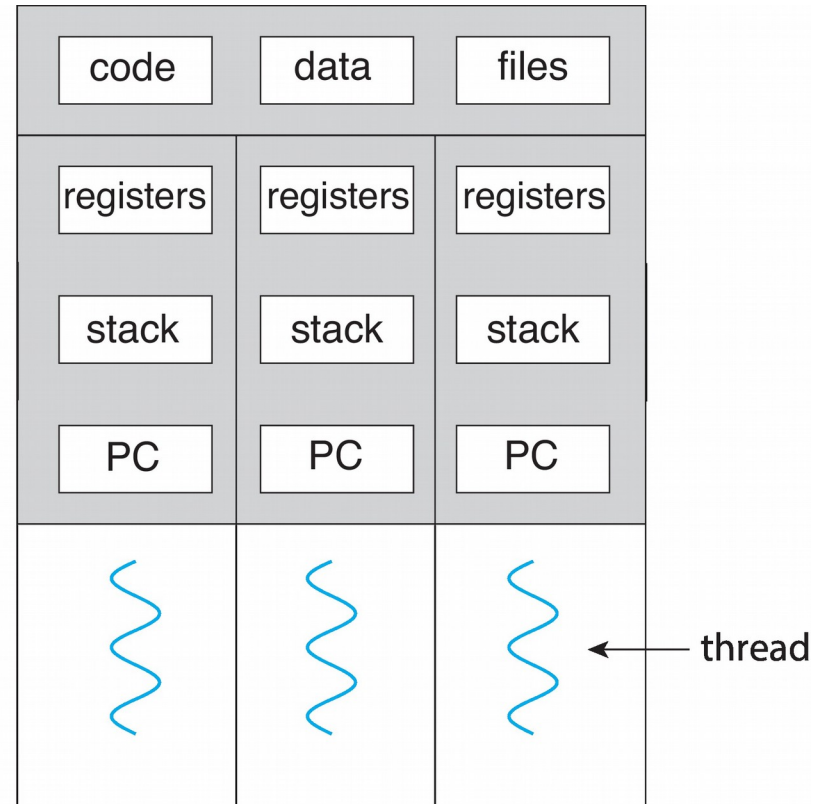




Single versus Multi-threaded Processes



single-threaded process



multithreaded process

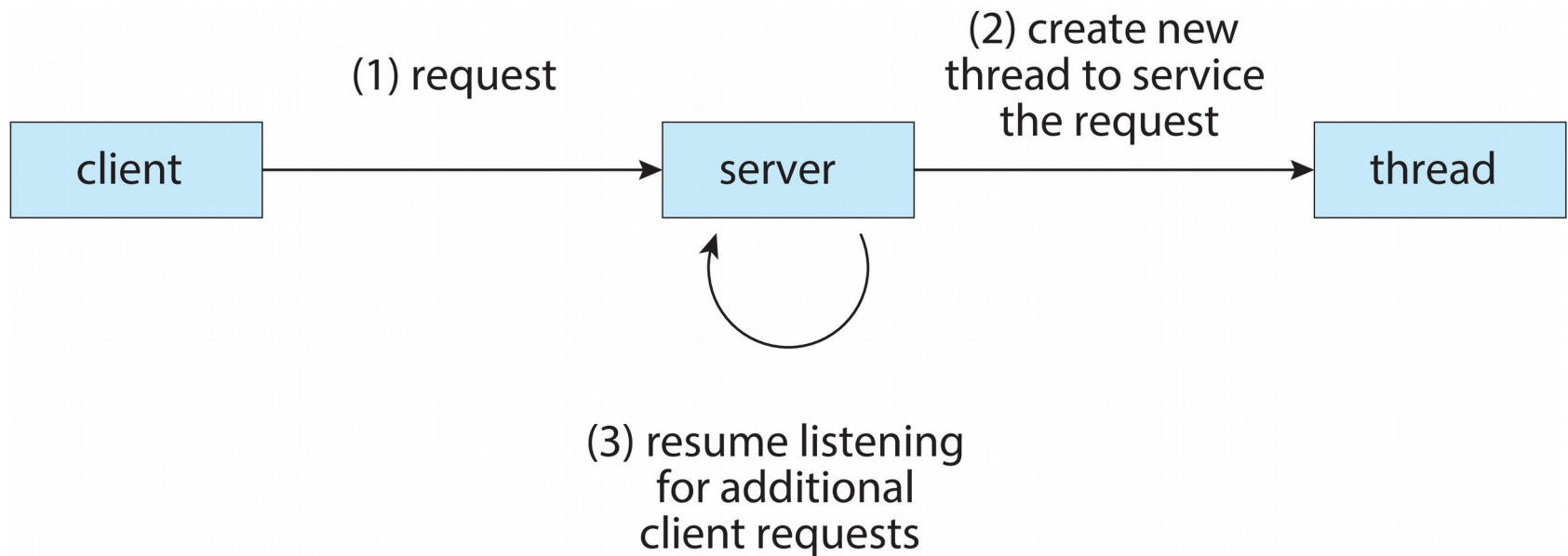
Each thread has its own register set, stack, and PC.





Multi-threaded Server Architecture

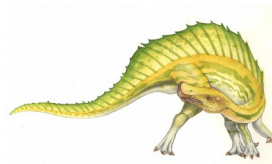
- 1) client sends request to server;
- 2) server creates a thread to process the request, and
- 3) immediately returns to listening for the next request from a client in the same main thread.





Benefits of Threads

- **Responsiveness** – may allow continued execution if part of process is blocked, or some slow operation in a different thread - especially important for user interfaces
- **Resource Sharing** – threads share same address space in single process, easier than processes using shared memory or message passing
- **Cost** – cheaper than process creation, thread switching lower overhead than context switching
- **Scalability** – process can take advantage of multi-core architectures





Concurrency vs. Parallelism

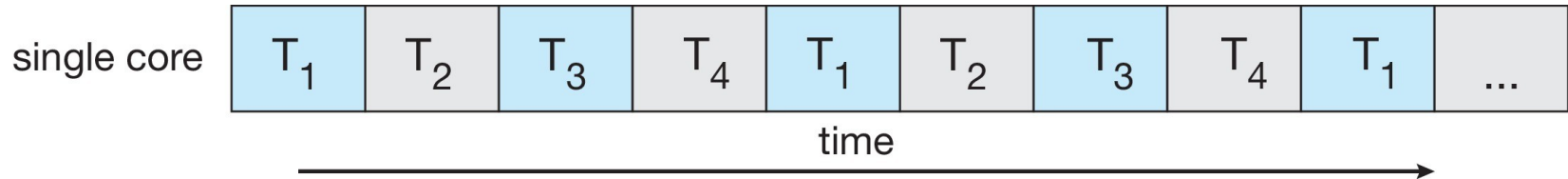
- Two or more sequences of instructions are said to be **concurrent** if no matter what order they are executed in relation to each other, the final result of their combined computation is the same.
- This means that they can be executed simultaneously on different processors, or interleaved on a single processor in any order, and whatever outputs they produce will be the same.
- A system with two or more concurrent processes is called a **concurrent program** or a **concurrent system**.
- Two processes or threads execute **in parallel** if they execute at the same time on different processors.
- **Parallel programs** are those containing instruction sequences that can be executed in parallel. A parallel program is always a concurrent program, but a system can have concurrency even though it is not a parallel program.



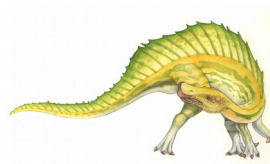
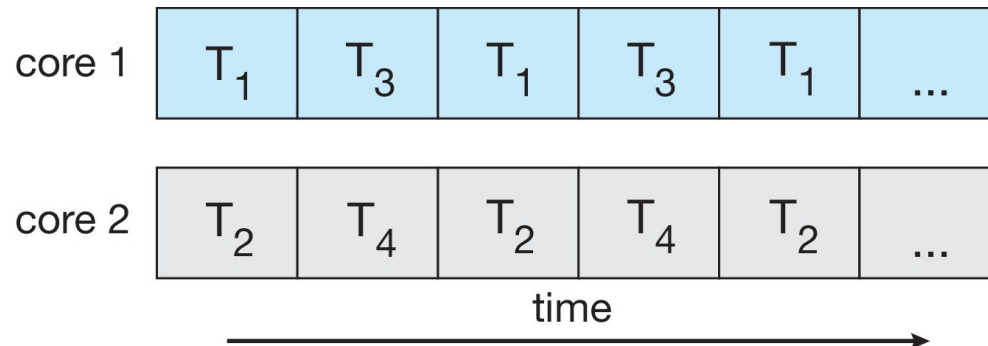


Concurrency vs. Parallelism

■ Concurrent execution on single-core system:



■ Parallelism on a multi-core system:





Multi-core Programming

- **Multi-core** or **multi-processor** systems challenge programmers to take advantage of hardware, but it is not easy:
 - **How to decompose a single task into many independent parallel tasks**
 - **How to load-balance the tasks**
 - **How to split data onto separate cores/processors**
 - **How to identify data dependency and handle synchronization**
 - **How to test and debug parallel programs**





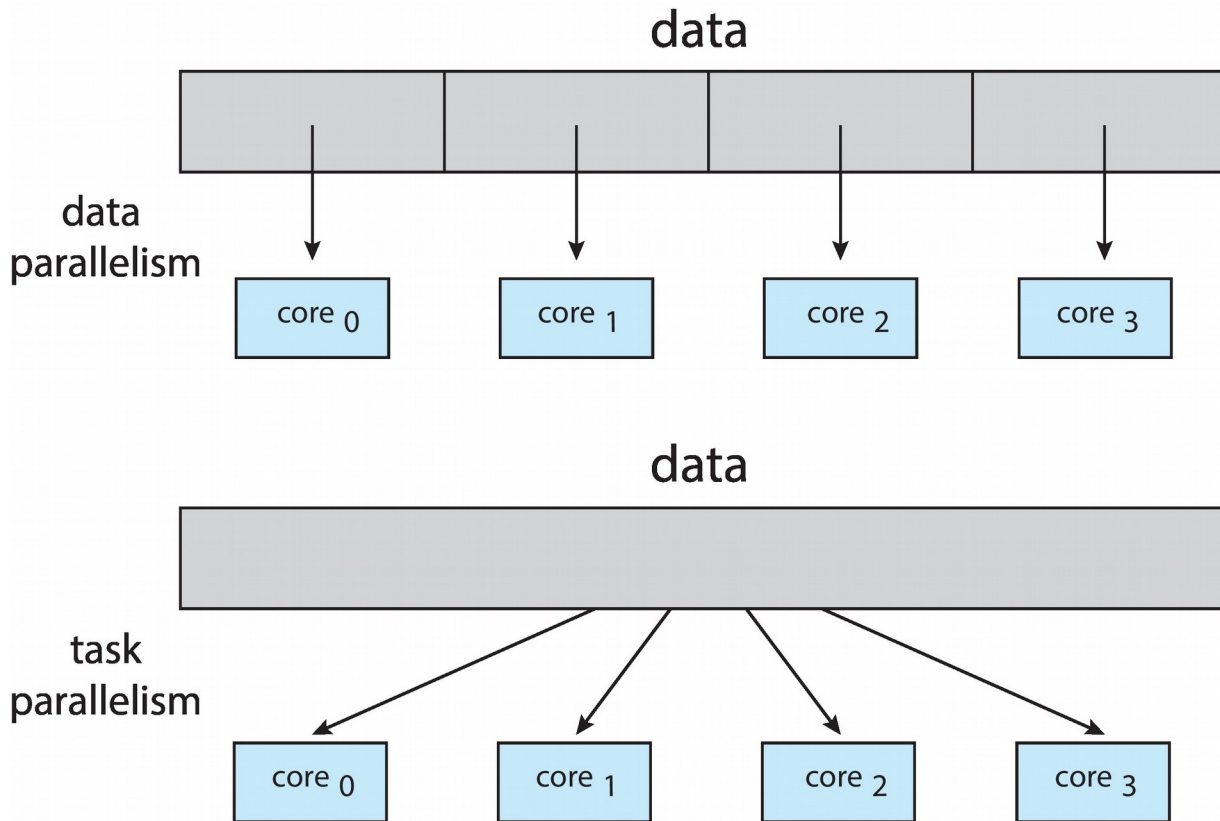
Multi-core Programming (cont.)

- Types of inherent parallelism:
 - **Data parallelism** – distributes subsets of the same data across multiple cores, same operation on each
 - ▶ an image on which the same operation is applied to all pixels
 - ▶ a payroll with taxes to be calculated for all individuals
 - ▶ a set of points to be rotated through same angle in space
 - **Task parallelism** – distributing threads across cores, each thread performing unique operation
 - ▶ same data set evaluated by multiple algorithms for some property (census data analyzed for demographics, financials, geographic, etc)





Data and Task Parallelism





Amdahl's Law

- In 1967, Gene Amdahl argued that there was an inherent limitation to the amount of speedup that could be obtained by performing a computation using more processors. His argument is known as “Amdahl’s Law”. If
 - S , $0 \leq S \leq 1$, is the fraction of operations that must be executed serially (in sequence), and
 - N is the number of processing cores, then the speed-up is bounded above:

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

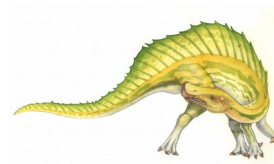
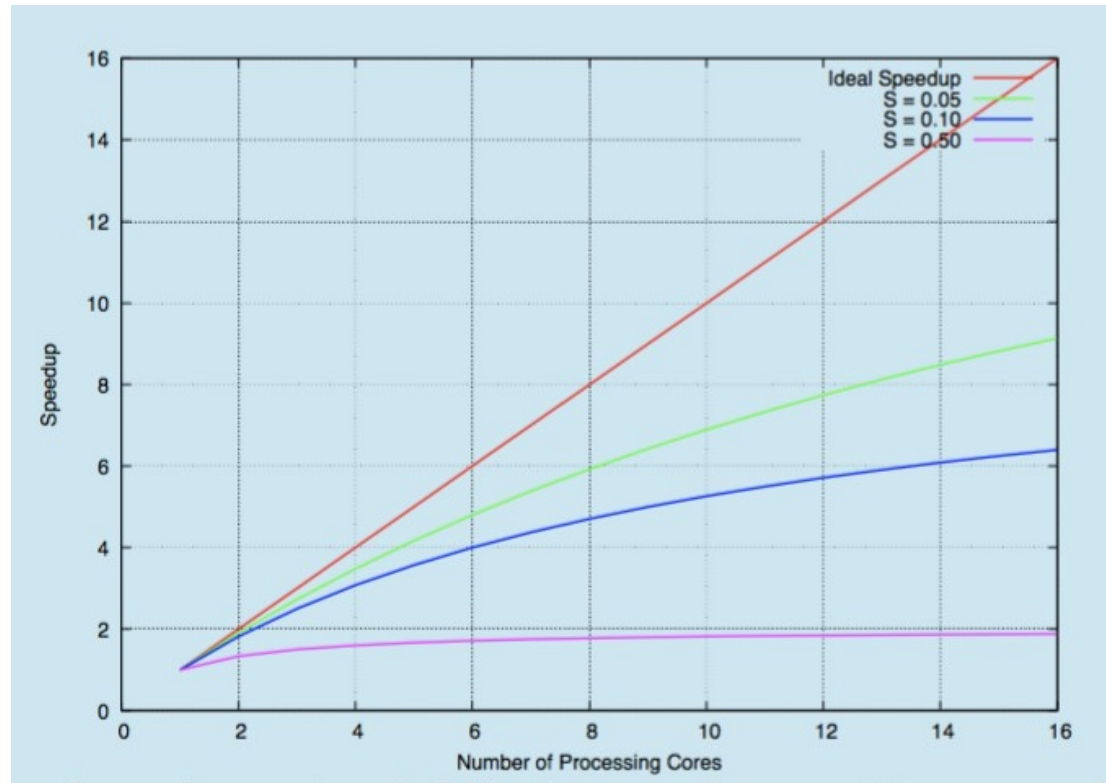
- Example: if program is 75% parallel / 25% serial, ($S=0.25$) moving from 1 to 2 cores ($N=2$) results in speedup of $1/((1/4) + (3/4)/2) = 1.6$
- As N approaches infinity, speedup approaches $1 / S$

Serial portion of an application limits maximum performance gained by adding additional cores





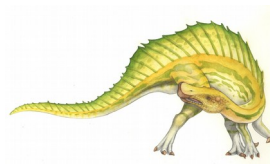
Amdahl's Law Graphically





User Threads and Kernel Threads

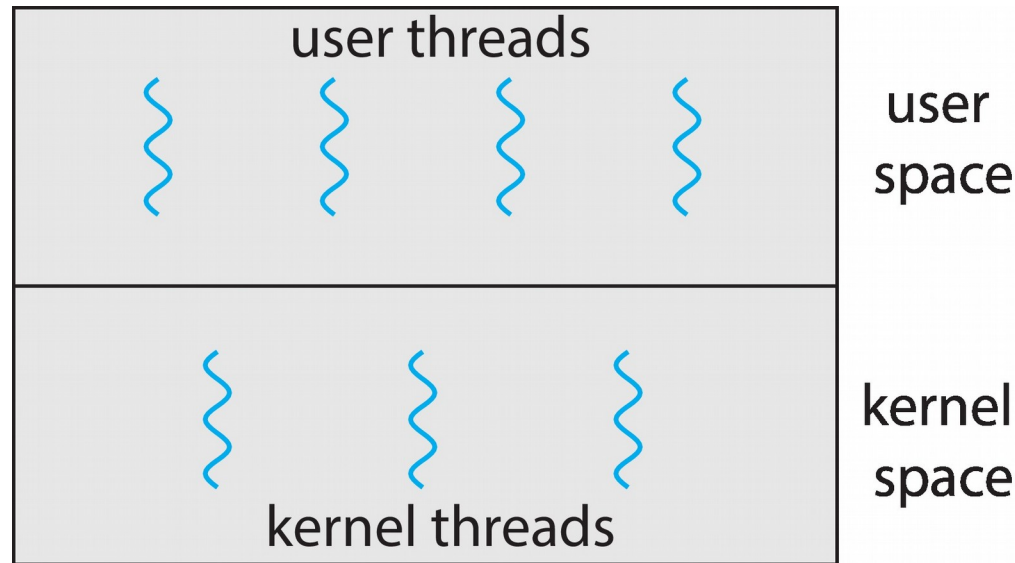
- **User threads** are supported by user-level libraries
- Three primary user thread libraries:
 - POSIX **Pthreads**
 - Windows threads
 - Java threads
- **Kernel threads** are supported directly by the kernel
 - Examples – virtually all modern operating systems, including:
 - ▶ Windows
 - ▶ Linux
 - ▶ Mac OS X
 - ▶ iOS
 - ▶ Android





User and Kernel Threads

- When threads are provided as user threads, they still must be mapped onto kernel threads.
- There is not necessarily an equal number of user and kernel threads.





Multi-threading Models

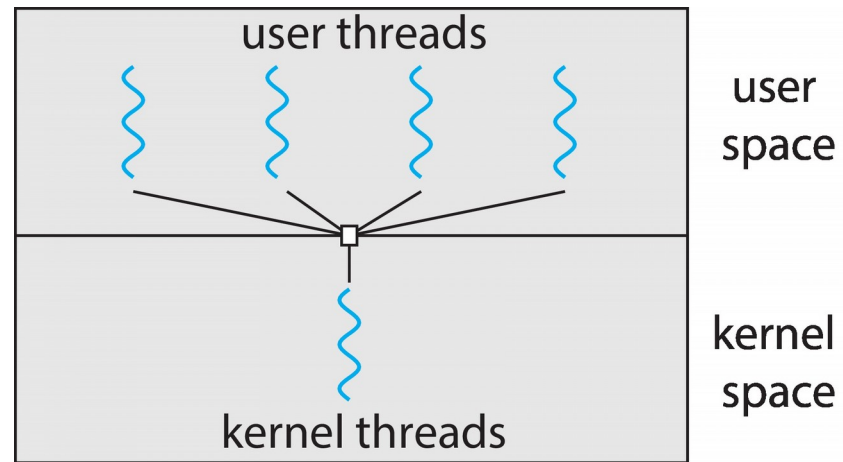
- How to map user threads to kernel threads?
- Three different models:
 - Many-to-One: many user-level threads map to single kernel thread
 - One-to-One: one user-level thread maps to one kernel thread
 - Many-to-Many: many user-level threads map to many kernel threads





Many-to-One

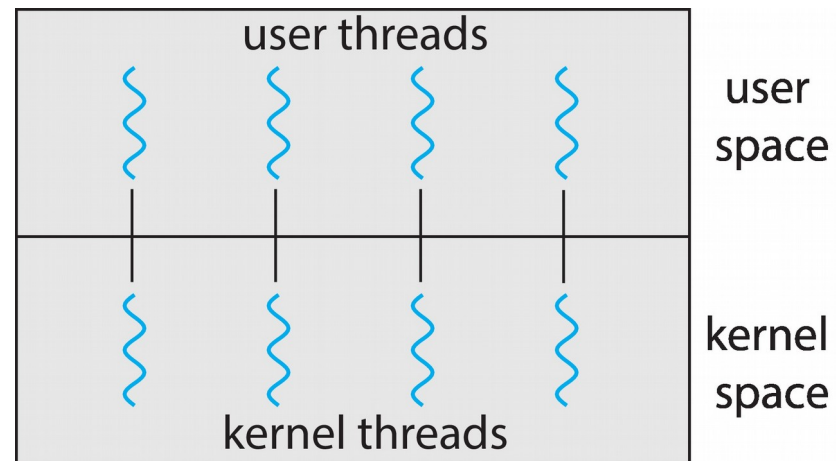
- Many user-level threads mapped to single kernel thread.
- Weaknesses:
 - One thread blocking causes all to block
 - Multiple threads may not run in parallel on multi-core system because only one may be in kernel at a time
- Few systems currently use this model because modern systems have many cores which are not utilized well.
- Examples:
 - **Solaris Green Threads**
 - **GNU Portable Threads**





One-to-One

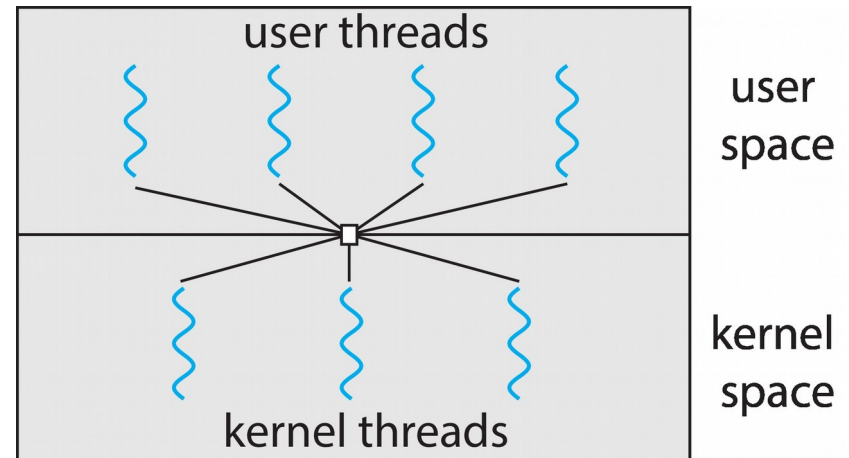
- Each user-level thread maps to one kernel thread
- Creating a user-level thread creates a kernel thread
- More concurrency than many-to-one
- Number of threads per process sometimes restricted due to overhead:
 - Creating a user thread requires creating a kernel thread, and too many kernel threads can degrade performance of system.
- Examples
 - Windows
 - Linux





Many-to-Many Model

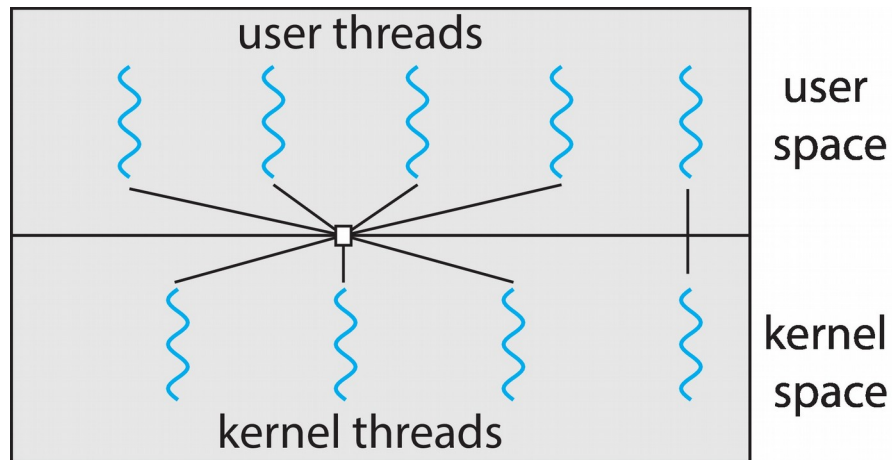
- Allows many user level threads to be multiplexed onto an equal or smaller number of kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Program can have as many user threads as necessary, and the corresponding kernel threads can run in parallel on a multiprocessor. If thread blocks, kernel can schedule a different thread.
- Windows with the ThreadFiber package
- Otherwise not very common





Two-level Model

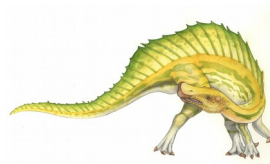
- Similar to the many-to-many, except that it allows a user thread to be **bound** to a kernel thread.





Thread Libraries

- **Thread library** provides programmer with API for creating and managing threads
- Two primary ways of implementing
 - Library entirely in user space
 - Kernel-level library supported by the OS
- Three prevalent libraries: POSIX threads (Pthreads), Windows, and Java threads.





Pthreads

- May be provided either as user-level or kernel-level
- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
- ***Specification***, not ***implementation***
- API specifies behavior of the thread library, implementation is up to development of the library
- Common in UNIX operating systems (Linux & Mac OS X)





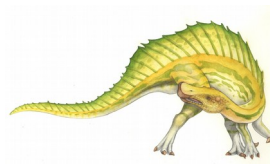
Pthreads Example 1

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void* hello_world( void* unused)
{
    printf("The child says, \"Hello world!\"\n");
    pthread_exit(NULL) ;
}

int main( int argc, char *argv[])
{
    pthread_t  child_thread;

    /* Create the thread and launch it. */
    if ( 0 != pthread_create(&child_thread, NULL,
        hello_world, NULL ) ){
        printf("pthread_create failed.\n");
        exit(1);
    }
    printf("This is the parent thread.\n");
    /* Wait for the child thread to terminate. */
    pthread_join(child_thread, NULL);
    return 0;
}
```





Pthreads Example 2

```
#include <pthread.h>
#include <stdio.h>

#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    /* set the default attributes of the thread */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid, &attr, runner, argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid, NULL);

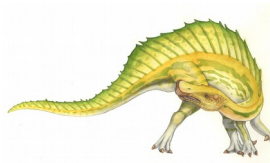
    printf("sum = %d\n", sum);
}
```





Pthreads Example 2 (cont)

```
/* The thread will execute in this function */  
void *runner(void *param)  
{  
    int i, upper = atoi(param);  
    sum = 0;  
  
    for (i = 1; i <= upper; i++)  
        sum += i;  
  
    pthread_exit(0);  
}
```





Pthreads Code for Joining 10 Threads

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
    pthread_join(workers[i], NULL);
```

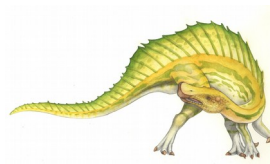




Windows Multi-threaded C Program

```
#include <windows.h>
#include <stdio.h>
DWORD Sum; /* data is shared by the thread(s) */

/* The thread will execute in this function */
DWORD WINAPI Summation(LPVOID Param)
{
    DWORD Upper = *(DWORD*)Param;
    for (DWORD i = 1; i <= Upper; i++)
        Sum += i;
    return 0;
}
```





Windows Multi-threaded C Program (Cont.)

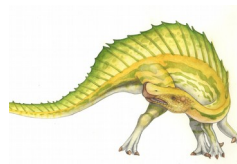
```
int main(int argc, char *argv[])
{
    DWORD ThreadId;
    HANDLE ThreadHandle;
    int Param;

    Param = atoi(argv[1]);
    /* create the thread */
    ThreadHandle = CreateThread(
        NULL, /* default security attributes */
        0, /* default stack size */
        Summation, /* thread function */
        &Param, /* parameter to thread function */
        0, /* default creation flags */
        &ThreadId); /* returns the thread identifier */

    /* now wait for the thread to finish */
    WaitForSingleObject(ThreadHandle, INFINITE);

    /* close the thread handle */
    CloseHandle(ThreadHandle);

    printf("sum = %d\n", Sum);
}
```





Implicit Threading

- Growing in popularity as numbers of threads increase, program correctness more difficult with explicit threads
- Creation and management of threads done by compilers and run-time libraries rather than programmers
- Five methods explored
 - Thread Pools
 - Fork-Join
 - OpenMP
 - Grand Central Dispatch
 - Intel Threading Building Blocks

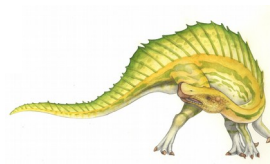




Thread Pools

- Create a number of threads in a pool where they await work
- Advantages:
 - Usually slightly faster to service a request with an existing thread than create a new thread
 - Allows the number of threads in the application(s) to be bound to the size of the pool
 - Separating task to be performed from mechanics of creating task allows different strategies for running task
 - ▶ i.e. Tasks could be scheduled to run periodically
- Windows API supports thread pools:

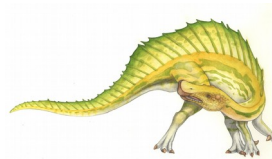
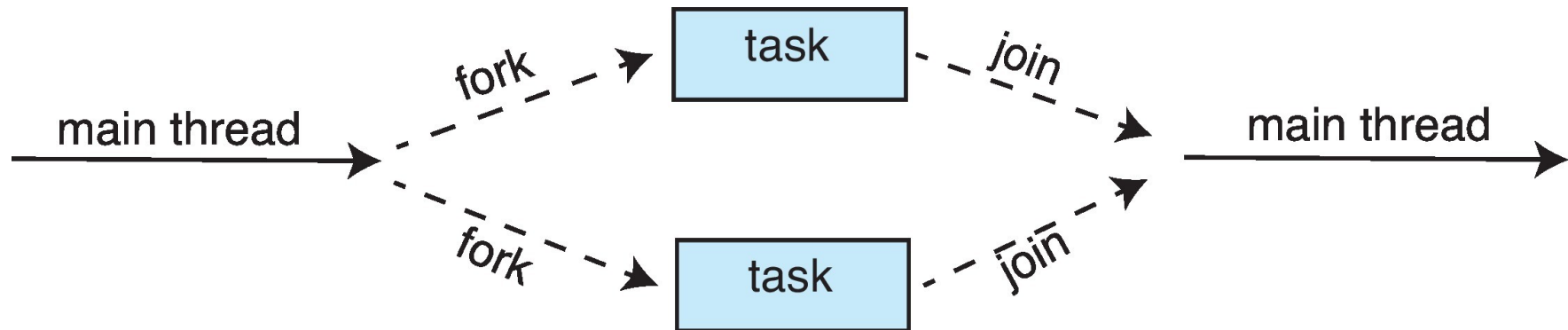
```
DWORD WINAPI PoolFunction(AVOID Param) {  
    /*  
     * this function runs as a separate thread.  
     */  
}
```





Fork-Join Parallelism

- Multiple threads (tasks) are **forked**, and then **joined**.





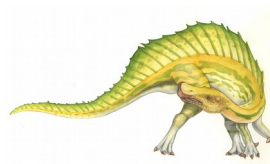
Fork-Join Parallelism

- General algorithm for fork-join strategy:

```
Task(problem)
  if problem is small enough
    solve the problem directly
  else
    subtask1 = fork(new Task(subset of problem))
    subtask2 = fork(new Task(subset of problem))

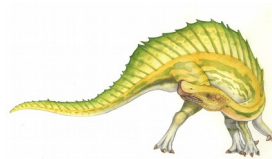
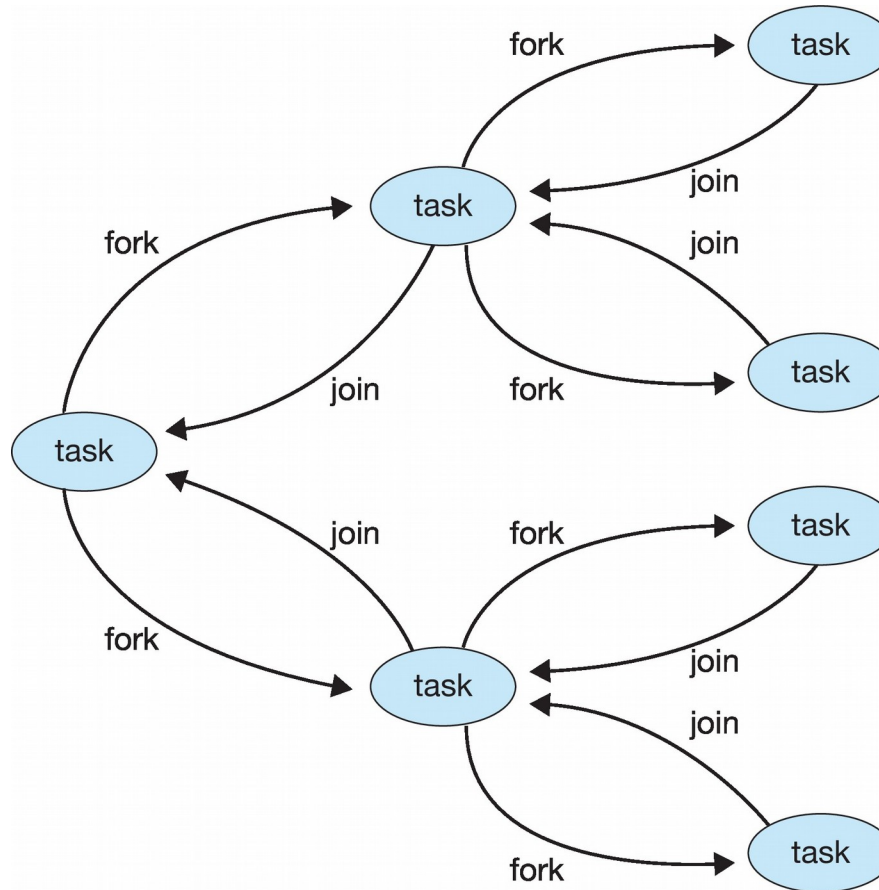
    result1 = join(subtask1)
    result2 = join(subtask2)

    return combined results
```





Fork-Join Parallelism





OpenMP

- Set of compiler directives and an API for C, C++, FORTRAN
- Provides support for parallel programming in shared-memory environments
- Identifies **parallel regions** – blocks of code that can run in parallel

#pragma omp parallel

Create as many threads as there are cores

```
#include <omp.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    /* sequential code */

    #pragma omp parallel
    {
        printf("I am a parallel region.");
    }

    /* sequential code */

    return 0;
}
```





OpenMP Example

- Run the for loop in parallel

```
#pragma omp parallel for  
for (i = 0; i < N; i++) {  
    c[i] = a[i] + b[i];  
}
```





Grand Central Dispatch

- Apple technology for macOS and iOS operating systems
- Extensions to C, C++ and Objective-C languages, API, and run-time library
- Allows identification of parallel sections
- Manages most of the details of threading
- Block is in “`^{}:`” :

```
^{ printf("I am a block"); }
```

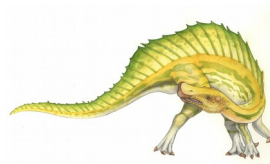
- Blocks placed in dispatch queue
 - Assigned to available thread in thread pool when removed from queue





Grand Central Dispatch (cont)

- Two types of dispatch queues:
 - **serial** – blocks removed in FIFO order, queue is per process, called **main queue**
 - ▶ Programmers can create additional serial queues within program
 - **concurrent** – removed in FIFO order but several may be removed at a time
 - ▶ Four system wide queues divided by quality of service:
 - QOS_CLASS_USER_INTERACTIVE
 - QOS_CLASS_USER_INITIATED
 - QOS_CLASS_USER_UTILITY
 - QOS_CLASS_USER_BACKGROUND





Grand Central Dispatch (3)

- For the Swift language a task is defined as a closure – similar to a block, minus the caret
- Closures are submitted to the queue using the `dispatch_async()` function:

```
let queue = dispatch_get_global_queue  
            (QOS_CLASS_USER_INITIATED, 0)
```

```
dispatch_async(queue, { print("I am a closure.") })
```





Threading Issues

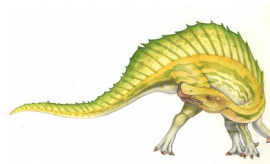
- Semantics of **fork()** and **exec()** system calls
- Signal handling
 - Synchronous and asynchronous
- Thread cancellation of target thread
 - Asynchronous or deferred
- Thread-local storage
- Scheduler Activations





Semantics of `fork()` and `exec()`

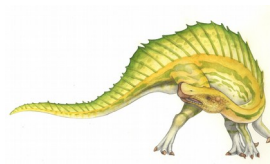
- Does **`fork()`** duplicate only the calling thread or all threads?
 - Some UNIXes have two versions of `fork`
- **`exec()`** usually works as normal – replace the running process including all threads





Signal Handling

- **Signals** are used in UNIX systems to notify a process that a particular event has occurred.
- A **signal handler** is used to process signals
 1. Signal is generated by particular event
 2. Signal is delivered to a process
 3. Signal is handled by one of two signal handlers:
 1. default
 2. user-defined
- Every signal has **default handler** that kernel runs when handling signal
 - **User-defined signal handler** can override default
 - For single-threaded, signal delivered to process





Signal Handling (Cont.)

- Where should a signal be delivered for multi-threaded?
 - Deliver the signal to the thread to which the signal applies
 - Deliver the signal to every thread in the process
 - Deliver the signal to certain threads in the process
 - Assign a specific thread to receive all signals for the process

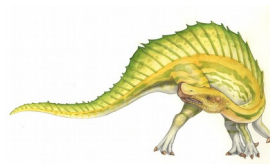




Thread Cancellation

- Terminating a thread before it has finished
- Thread to be canceled is **target thread**
- Two general approaches:
 - **Asynchronous cancellation** terminates the target thread immediately
 - **Deferred cancellation** allows the target thread to periodically check if it should be cancelled
- Pthread code to create and cancel a thread:

```
pthread_t tid;  
  
/* create the thread */  
pthread_create(&tid, 0, worker, NULL);  
  
. . .  
  
/* cancel the thread */  
pthread_cancel(tid);  
  
/* wait for the thread to terminate */  
pthread_join(tid, NULL);
```





Thread Cancellation (Cont.)

- Invoking thread cancellation requests cancellation, but actual cancellation depends on thread state

Mode	State	Type
Off	Disabled	–
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous

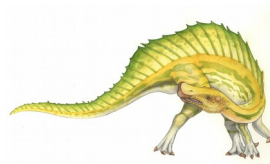
- If thread has cancellation disabled, cancellation remains pending until thread enables it
- Default type is deferred
 - Cancellation only occurs when thread reaches **cancellation point**
 - ▶ I.e. **pthread_testcancel()**
 - ▶ Then **cleanup handler** is invoked
- On Linux systems, thread cancellation is handled through signals





Thread-Local Storage

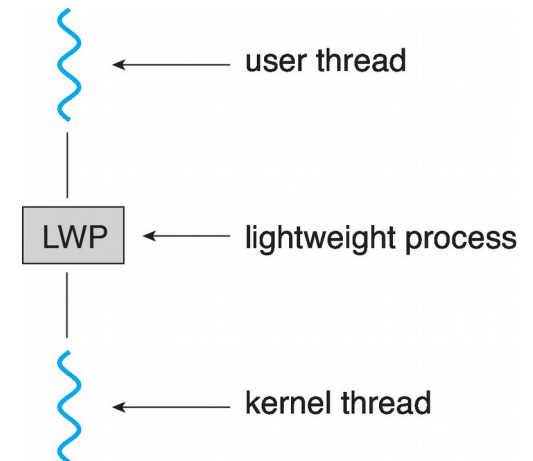
- **Thread-local storage (TLS)** allows each thread to have its own copy of data
- Useful when you do not have control over the thread creation process (i.e., when using a thread pool)
- Different from local variables
 - Local variables visible only during single function invocation
 - TLS visible across function invocations
- Similar to **static** data
 - TLS is unique to each thread





Scheduler Activations

- Both M:M and Two-level models require communication to maintain the appropriate number of kernel threads allocated to the application
- Typically use an intermediate data structure between user and kernel threads – **lightweight process (LWP)**
 - Appears to be a virtual processor on which process can schedule user thread to run
 - Each LWP attached to kernel thread
 - How many LWPs to create?
- Scheduler activations provide **upcalls** - a communication mechanism from the kernel to the **upcall handler** in the thread library
- This communication allows an application to maintain the correct number kernel threads





Operating System Examples

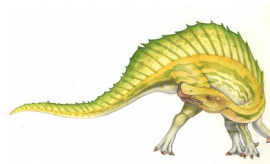
- Windows Threads
- Linux Threads





Windows Threads

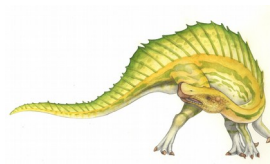
- Windows API – primary API for Windows applications
- Implements the one-to-one mapping, kernel-level
- Each thread contains
 - A thread id
 - Register set representing state of processor
 - Separate user and kernel stacks for when thread runs in user mode or kernel mode
 - Private data storage area used by run-time libraries and dynamic link libraries (DLLs)
- The register set, stacks, and private storage area are known as the **context** of the thread





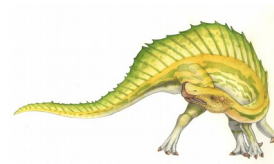
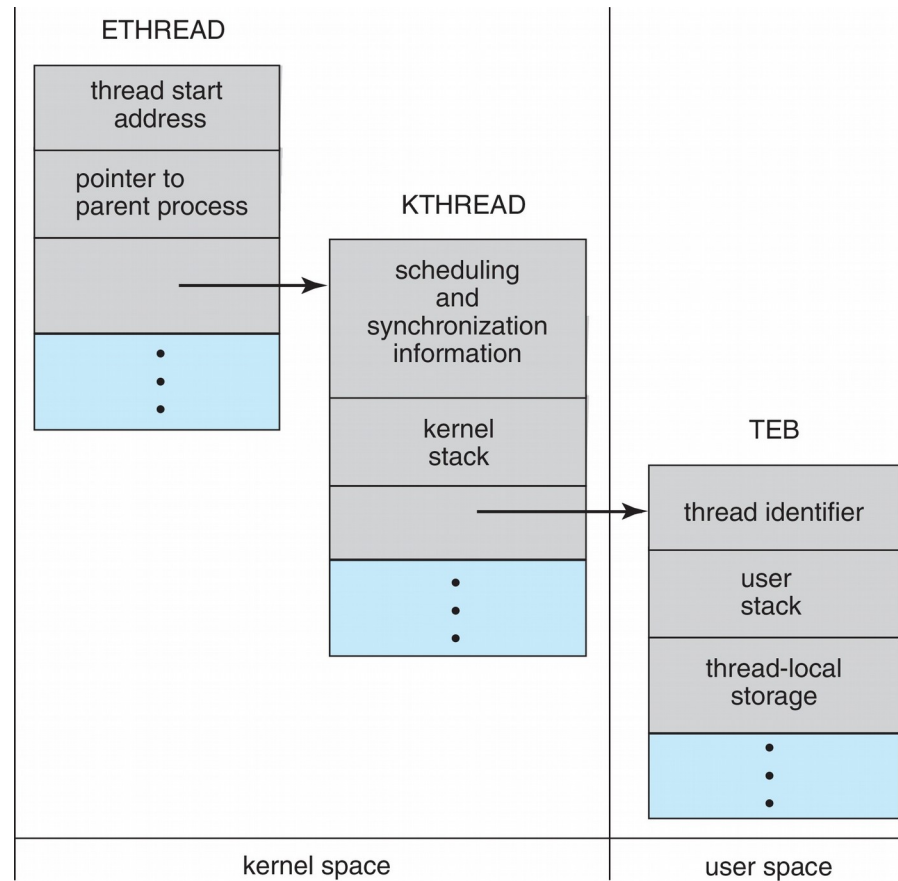
Windows Threads (Cont.)

- The primary data structures of a thread include:
 - ETHREAD (executive thread block) – includes pointer to process to which thread belongs and to KTHREAD, in kernel space
 - KTHREAD (kernel thread block) – scheduling and synchronization info, kernel-mode stack, pointer to TEB, in kernel space
 - TEB (thread environment block) – thread id, user-mode stack, thread-local storage, in user space





Windows Threads Data Structures



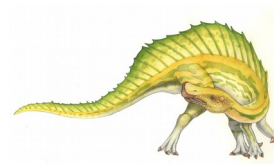


Linux Threads

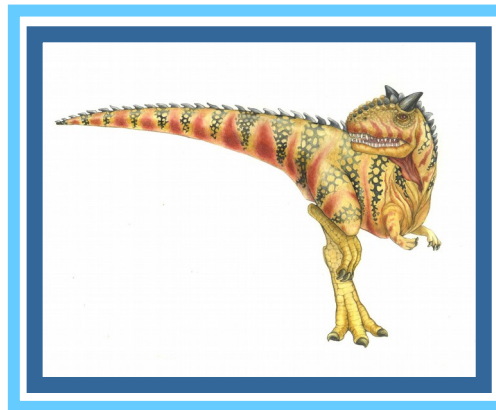
- Linux refers to them as **tasks** rather than **threads**
- Thread creation is done through **clone()** system call
- **clone()** allows a child task to share the address space of the parent task (process)
 - Flags control behavior

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

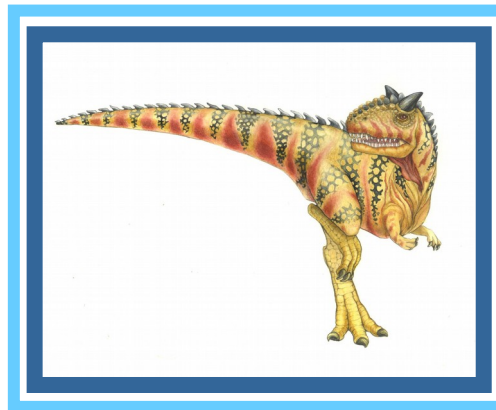
- **struct task_struct** points to process data structures (shared or unique)



End of Chapter 4



Chapter 5: CPU Scheduling





Chapter 5: CPU Scheduling

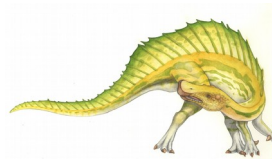
- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Thread Scheduling
- Multi-Processor Scheduling
- Real-Time CPU Scheduling
- Operating Systems Examples
- Algorithm Evaluation





Objectives

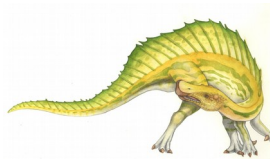
- Understand difference between different types of scheduling
- Describe various CPU scheduling algorithms
- Assess CPU scheduling algorithms based on scheduling criteria
- Explain the issues related to multiprocessor and multicore scheduling
- Describe various real-time scheduling algorithms
- Describe the scheduling algorithms used in the Linux operating system





Medium Term Scheduling

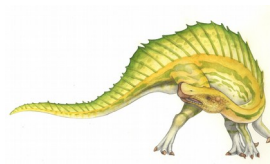
- The **medium term scheduler** controls the degree of multi-programming.
- It admits processes into memory to compete for the CPU
- Also called a **swapper** when it is used to control the **process mix** by removing and restoring processes
- Goal of medium term scheduler is to keep a good mix of processes in memory so that the CPU is always kept busy.





Basic Problem

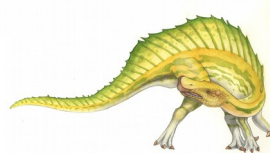
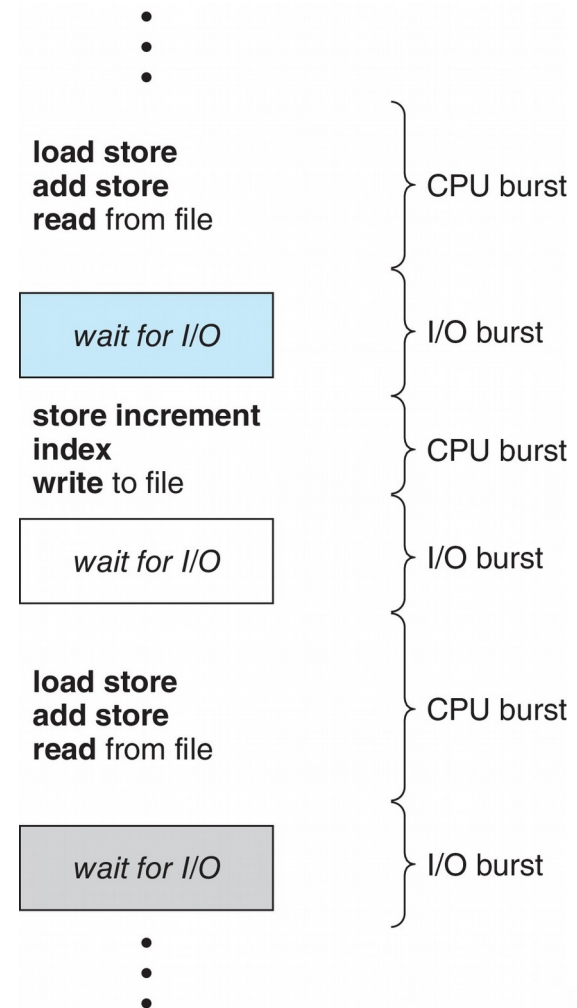
- Objective of multi-programming is to have some process running at all times, to maximize CPU utilization.
- Need to keep many programs in memory
- Process executes on CPU until it executes some instruction for which it has to wait (e.g. I/O)
- Process is removed from CPU and another must be chosen to run
- This is purpose of CPU scheduler: which process should run?





Core Concept

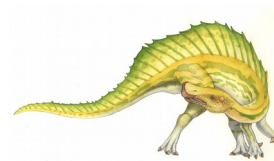
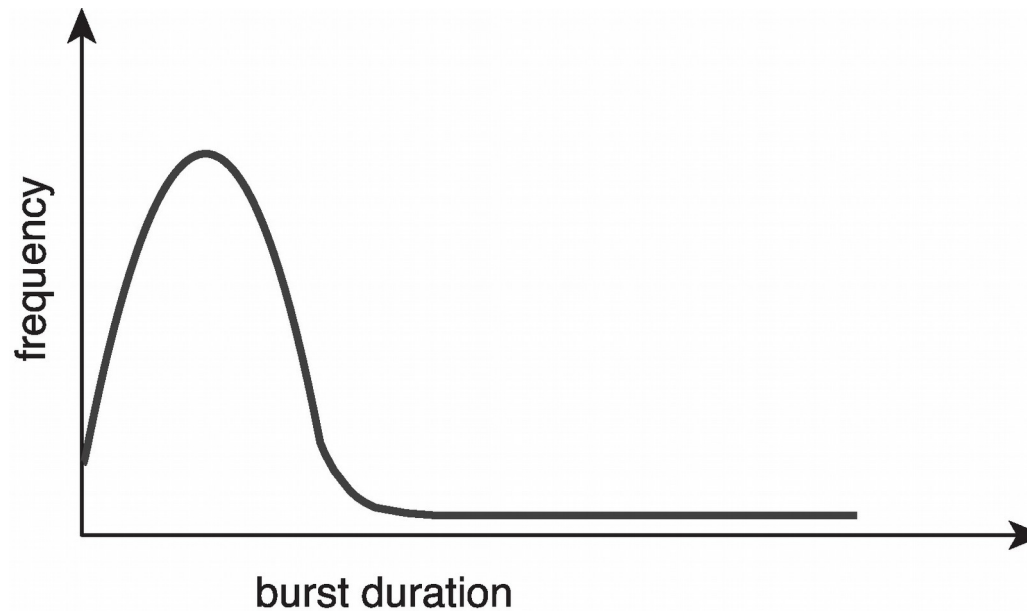
- Every process has the same cycle of execution, called the **CPU-I/O Burst Cycle**
 - Process executes on CPU until it issues I/O, then waits for I/O to complete
 - The time on CPU is called a **CPU burst**
 - When it issues I/O request it is called an **I/O burst**
- It repeats this cycle over and over until it terminates.
- Lengths of CPU bursts affect decisions about scheduling.





Distribution of CPU Burst Times

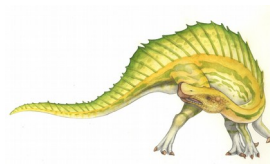
- The duration of CPU bursts has been measured extensively.
 - Many burst times are short; few are long
 - Distribution is generally exponential or hyperexponential,
 - An **I/O-bound** program has many short CPU bursts.
 - A **CPU-bound** program has a few long CPU bursts.





CPU Scheduler

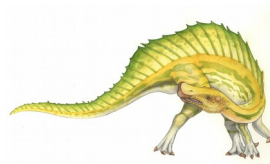
- When the running process is removed from CPU, the OS must choose a process to run. This is the role of the **CPU scheduler**.
- One or more queues of **ready-to-run** processes are kept in memory. These are called **ready queues**.
- The **CPU scheduler** selects from among the processes in ready queue, and allocates the a CPU core to one of them
 - Queue may be ordered in various ways – FIFO, priority queue, unordered, etc
 - Usually process control blocks or pointers to PCBs are kept in queues.





Scheduling Decisions

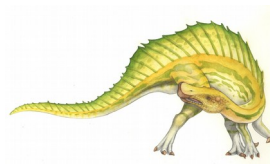
- CPU scheduling decisions may take place when a process:
 1. Switches from running to waiting state (e.g. by issuing an I/O request)
 2. Switches from running to ready state (e.g., an interrupt occurs)
 3. Switches from waiting to ready (e.g., I/O completes)
 4. Terminates
- Condition 3 is called a signal or release event.
- If scheduling only occurs under conditions 1 and 4 it is called **nonpreemptive scheduling**
- If it occurs under any of the conditions it is called **preemptive scheduling**





Preemption

- Without preemption, process runs until it terminates or issues some request that causes it to wait such as I/O or other system call.
- Results in poor response time for other processes.
- **Preemptive scheduling** is the norm for modern operating systems.
- But preemptive scheduling causes other problems:
 - Consider two processes that access shared data
 - Consider preemption while in kernel mode – kernel data structures can become corrupted if kernel is not designed carefully
 - Consider interrupts occurring during crucial OS activities





Non-Preemptive Kernel

- During the processing of a system call, the kernel may be busy with an activity on behalf of a process.
- This can involve changing a kernel data structure.
- If it is interrupted or preempted, the data structure can become corrupted.
- A non-preemptive kernel waits for a system call to complete or for a process to block while waiting for I/O to complete to take place before doing a context switch.
- Makes the kernel structure simple, since the kernel will not preempt a process while the kernel data structures are in an inconsistent state.
- But very inefficient and does not work for real-time operating systems.





Preemptive Kernels

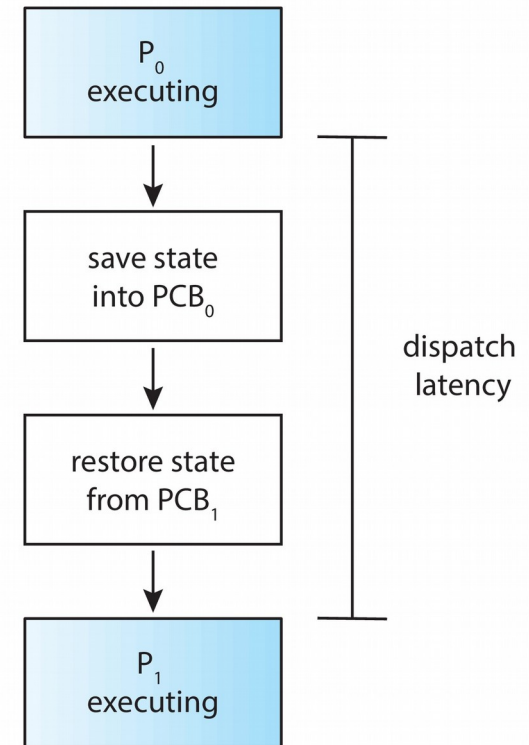
- Kernel is allowed to be interrupted almost any time because they might be important and cannot be ignored by the kernel.
- The kernel needs to handle these interrupts otherwise, input might be lost or output overwritten.
- Solution: disable interrupts at entry and re-enable interrupts at exit of sections of code that handle them.
- Code sections must be very short.





Dispatcher

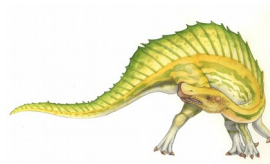
- **Dispatcher** is the component of the kernel that gives control of the CPU to the process selected by the short-term scheduler
- This involves:
 - switching context
 - switching to user mode
 - jumping to the proper location in the user program to restart that program (loading its PC)
- **Dispatch latency** – time it takes for the dispatcher to stop one process and start another running
- Need dispatcher to be very fast to minimize latency.





CPU Scheduling Criteria

- **CPU utilization** – keep the CPU as busy as possible
- **Throughput** – # of processes that complete their execution per time unit
- **Turnaround time** – amount of time to execute a particular process, including wait time
- **Waiting time** – amount of time a process has been waiting in the ready queue
- **Response time** – amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)





Scheduling Algorithm Objectives

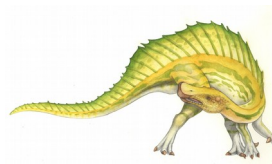
- Maximize CPU utilization
- Maximize throughput
- Minimize turnaround time
- Minimize waiting time
- Minimize response time
- Cannot accomplish all, so it is an **optimization problem**.





Nuances of Optimization

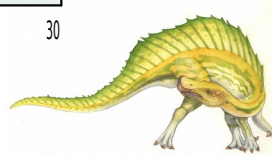
- Do we optimize the maximum, minimum, or average measure of the property. For example,
 - do we optimize design so that maximum response time is minimal or average response time is minimal?
 - do we minimize maximum waiting time or average waiting time?
- In time-sharing systems, reliability is important, so minimizing variance in response time is often the objective rather than mean response time.
- For simple analysis, to assess scheduling algorithms, we can model processes by the lengths of CPU bursts and their average wait times.





Scheduling Algorithms

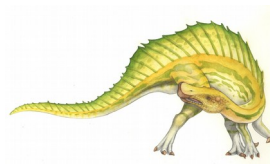
- We look at a few common scheduling algorithms.
- Each is classified as preemptive or non-preemptive.
- An algorithm that is non-preemptive is only used for non-preemptive scheduling; a preemptive algorithm is used for preemptive scheduling.
- We will use Gantt charts to analyze the algorithms.
- A Gantt chart is a horizontal bar chart in which the x-axis is time. A Gantt chart represents the order in which processes are scheduled and how much time each spends in the CPU.
- Example:
 - The chart below represents 3 processes, P1, P2, P3, scheduled at times 0, 26, and 27 respectively. P1 ran 26 units, P2, 1 unit, P3, 3 units.





First-Come-First-Served Scheduling

- The first is **first-come-first-served (FCFS)** which is a non-preemptive algorithm.
- In **first-come-first-served scheduling**, processes arrive and are placed at the rear of a FIFO queue.
- The process at the front of the queue is scheduled next.
- If a process issues a wait, it is placed at the rear of the queue again.





FCFS Example

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3
The Gantt Chart for the schedule is:



- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$





FCFS Example (Cont.)

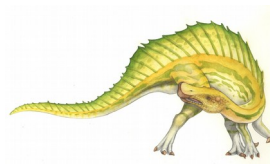
Suppose that the processes arrive in the order:

$$P_2, P_3, P_1$$

- The Gantt chart for the schedule is:



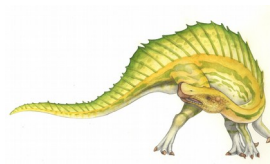
- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case
- Performance is affected by order of arrivals





Convoy Effect

- **Convoy effect** - short process behind long process
 - Consider one CPU-bound and many I/O-bound processes
 - CPU bound process runs, takes long time and short I/O bound processes wait.
 - They all run quickly, issue I/O requests and wait again for long CPU bound process. Their I/O is satisfied and they all move to ready queue, but they have to wait and wait and wait...
- I/O devices are poorly utilized and response times are poor.





Shortest-Job-First (SJF) Scheduling

- Associate with each process the length of its next CPU burst
 - Use these lengths to schedule the process with the shortest time
- SJF is optimal – gives minimum average waiting time for a given set of processes
 - The difficulty is knowing the length of the next CPU request
 - Could ask the user



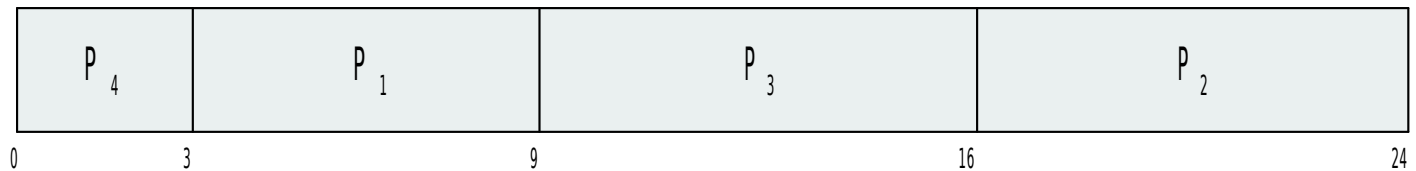


Example of SJF

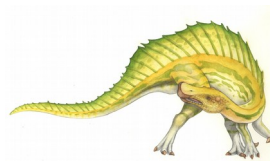
- Assume all processes arrive at time 0 and that CPU burst times are known and as shown below.

<u>Process</u>	<u>Burst Time</u>
P_1	6
P_2	8
P_3	7
P_4	3

- SJF scheduling chart



- Average waiting time = $(3 + 16 + 9 + 0) / 4 = 7$





Predicting Length of Next CPU Burst

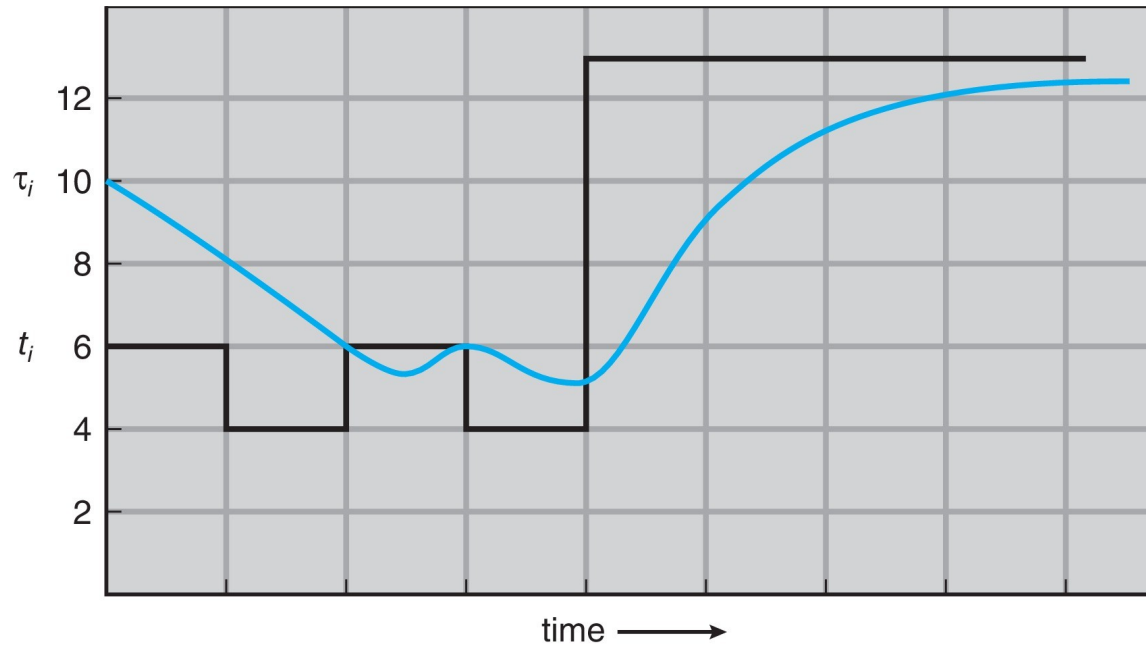
- Can only estimate the length of next burst – should be similar to the previous one, or the the most recent ones.
 - Then pick process with shortest predicted next CPU burst
- Can be done by using the length of previous CPU bursts, using *exponential averaging*:
 1. t_n = actual length of n^{th} CPU burst
 2. τ_{n+1} = predicted value for the next CPU burst
 3. For any α , $0 \leq \alpha \leq 1$
 4. define: $\tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n$.
- Initially, first guess τ_0 is any value.
- When $\alpha=0$, guess never changes. When $\alpha = 1$, guess is always last burst length. (upcoming slides illustrate) Typically α is set to $\frac{1}{2}$
- Preemptive version called **shortest-remaining-time-first**



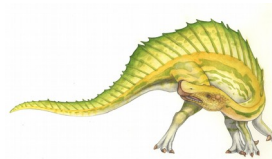


Prediction of the Length of the Next CPU Burst

(This graph is for $\alpha=0.5$.)



CPU burst (t_i)	6	4	6	4	13	13	13	...	
"guess" (τ_i)	10	8	6	6	5	9	11	12	...



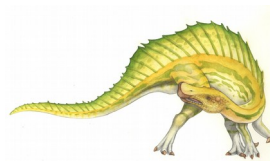


Examples of Exponential Averaging

- $\alpha = 0$
 - $\tau_{n+1} = \tau_n$
 - Recent history does not count
- $\alpha = 1$
 - $\tau_{n+1} = \alpha t_n$
 - Only the actual last CPU burst counts
- If we expand the formula, we get:

$$\begin{aligned}\tau_{n+1} = & \alpha t_n + (1 - \alpha)\alpha t_{n-1} + \dots \\ & + (1 - \alpha)^j \alpha t_{n-j} + \dots \\ & + (1 - \alpha)^{n+1} \tau_0\end{aligned}$$

- Since both α and $(1 - \alpha)$ are less than or equal to 1, each successive term has less weight than its predecessor

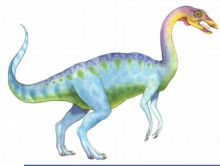




Example of Shortest-Remaining-Time-First

- When SJF is preemptive it is called **Shortest Remaining Time First**, so **SRTF = Preemptive SJF**
- A process runs, gets preempted and is moved to back of ready queue. Its new predicted burst time is the previous one minus the time it just spent on the CPU.
- This is why it is shortest remaining time first, because predicted burst is what is remaining.



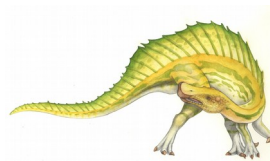


Example of SRTF

- Now we add the concepts of varying arrival times and preemption to the analysis

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

- To create a Gantt Chart, you need to simulate the scheduler.
- Scheduler needs to run when a new process arrives, and it picks shortest burst time remaining process.





SRTF Worked Example

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

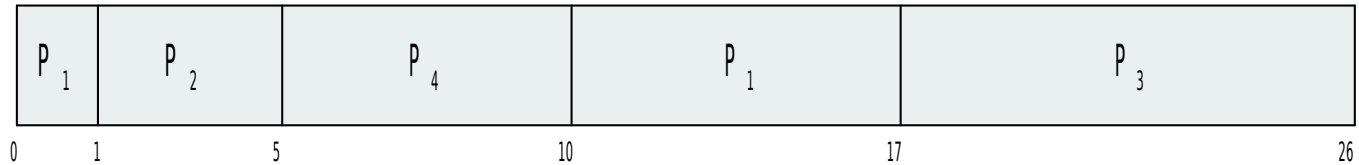
- Time 0: P_1 runs for 1 msec. P_2 arrives and has shorter time so it is scheduled. P_1 has 7 msec remaining time now.
- Time 1: P_2 runs. P_3 and P_4 arrive at times 2 and 3, but both have longer times, so P_2 stays on CPU until it finishes at time 5 (1+4)
- Time 5: P_4 has shortest remaining time (5 msec) so it gets CPU and runs to finish at time 10.
- Time 10: P_1 has shorter time than P_3 , so P_1 runs and finishes at time 17, then P_3 runs.





Example of Shortest-Remaining-Time-First

- SRTF Gantt Chart for this set of processes:



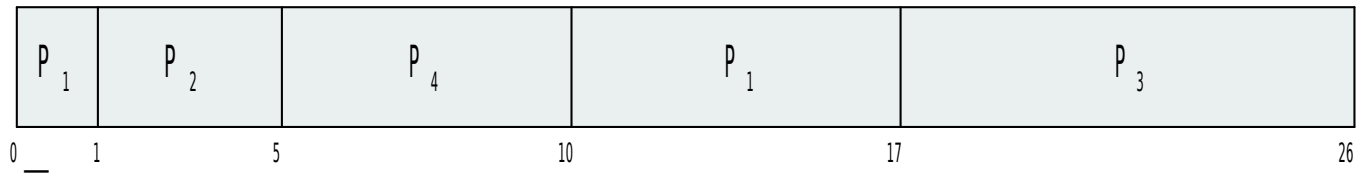
- What is the average waiting time?





Waiting Time for SRTF

- Waiting time for each process is
 $(\text{time it finishes}) - (\text{burst length}) - (\text{arrival time})$



<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>	<u>End Time</u>
P_1	0	8	17
P_2	1	4	5
P_3	2	9	26
P_4	3	5	10

- Average waiting time =
 - $((17-8-0) + (5-4-1) + (26-9-2) + (10-5-3))/4 =$
 - $(9+0+15+2)/4 = 6.5$





Round Robin (RR)

- Each process gets a small unit of CPU time (**time quantum q**), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue. ***This is a scheduled event.***
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1)q$ time units to run again.
- Might be less if processes issue I/O requests.
- Timer interrupts at end of quantum to schedule next process
- Performance
 - q large $\Rightarrow$ FIFO
 - q small $\Rightarrow q$ must be large with respect to context switch, otherwise overhead is too high

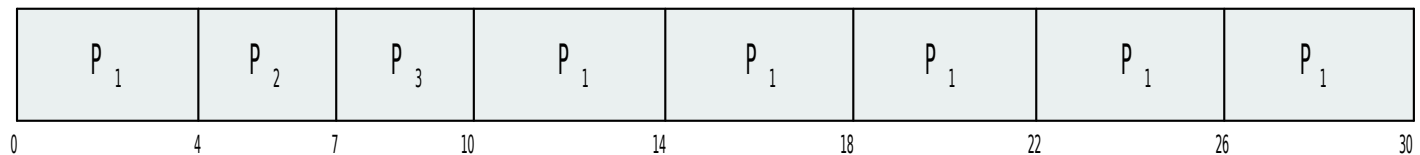




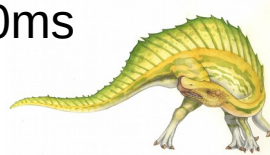
Example of RR with Time Quantum = 4

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- The Gantt chart is:

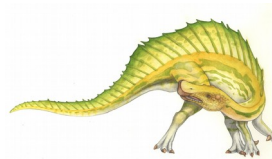
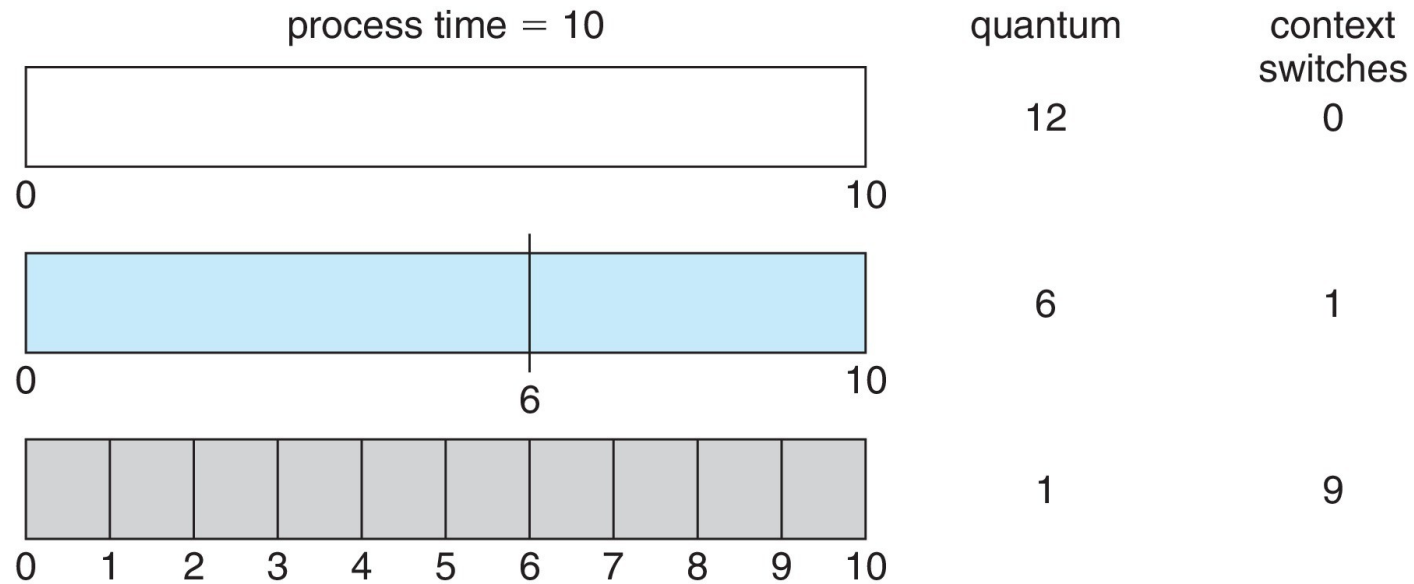


- Turnaround time average here = $(30 + 10 + 7)/3 = 15.7$
- If using SJF, turnaround would be $(30 + 6 + 3)/3 = 13$
- Typically, higher average turnaround than SJF, but better **response time**
- q should be large compared to context switch time; q usually 10ms to 100ms, context switch < 10 usec



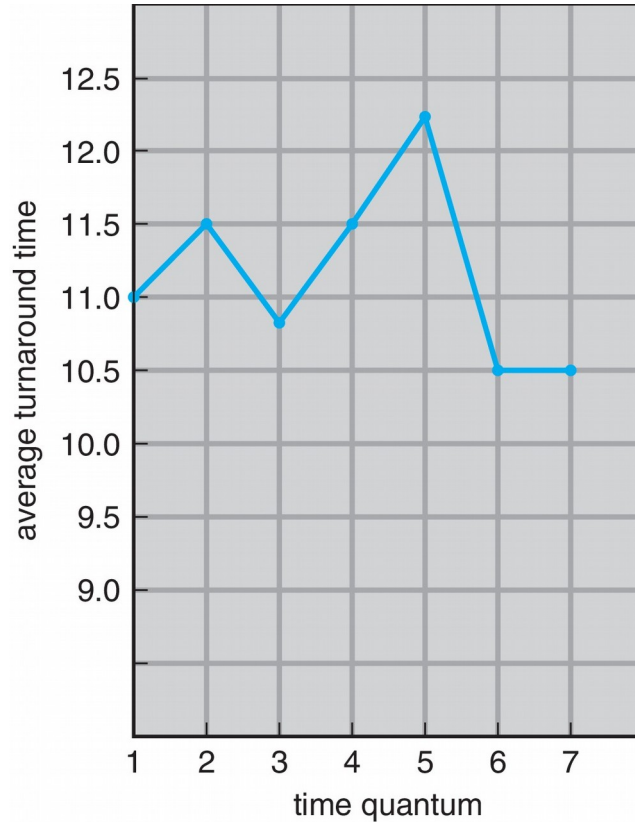


Time Quantum and Context Switch Time





Turnaround Time Varies With The Time Quantum



process	time
P_1	6
P_2	3
P_3	1
P_4	7

Rule of thumb for choosing quantum size:

80% of CPU bursts should be shorter than q

P_1	P_2	P_3	P_4	P_1	P_2	P_4	P_1	P_2	P_4	P_1	P_4	P_1	P_4	P_1	P_4	P_4	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17





Priority Scheduling

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer $\equiv$ highest priority)
- Can be either
 - Preemptive - high priority arriving process preempts lower one
 - Nonpreemptive
- SJF is priority scheduling where priority is the inverse of predicted next CPU burst time
- Problem $\equiv$ **Starvation** – (also called **indefinite blocking**)
 - low priority processes may never execute
- Solution $\equiv$ **Aging** – as time progresses increase the priority of the process

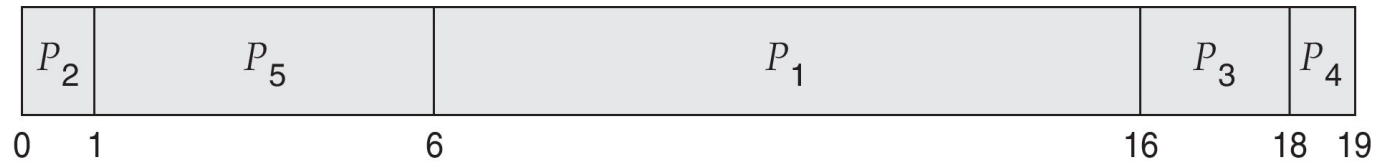




Example of Priority Scheduling

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

■ Priority scheduling Gantt Chart



■ Average waiting time = 8.2 msec



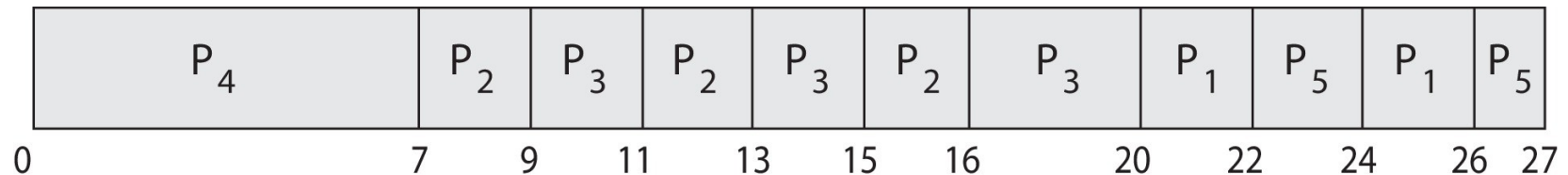


Priority Scheduling w/ Round-Robin

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	4	3
P_2	5	2
P_3	8	2
P_4	7	1
P_5	3	3

- Run the process with the highest priority. Processes with the same priority run round-robin

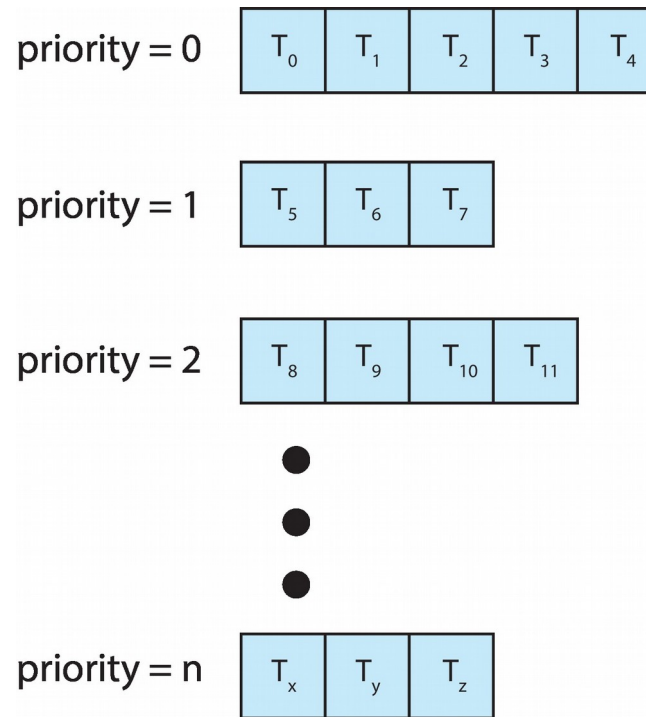
- Gantt Chart with 2 ms time quantum





Multilevel Queue

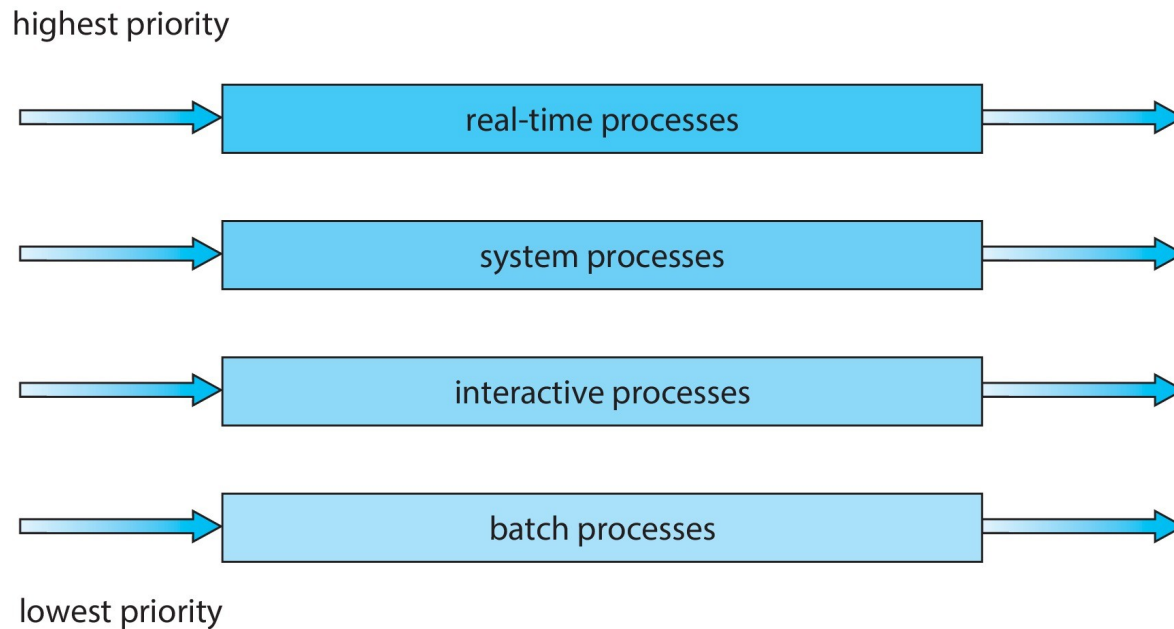
- With priority scheduling, have separate queues for each priority.
- Schedule the process in the highest-priority queue!





Multilevel Queues

■ Prioritization based upon process type





Multilevel Feedback Queue

- A process can move between the various queues; aging can be implemented this way
- Multilevel-feedback-queue scheduler defined by the following parameters:
 - number of queues
 - scheduling algorithms for each queue
 - method used to determine when to upgrade a process
 - method used to determine when to demote a process
 - method used to determine which queue a process will enter when that process needs service





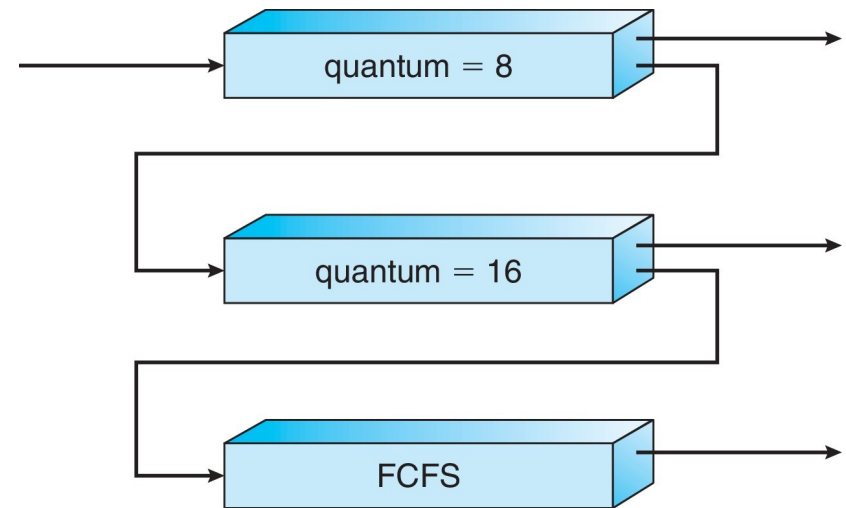
Example of Multilevel Feedback Queue

■ Three queues:

- Q_0 – RR with time quantum 8 milliseconds
- Q_1 – RR time quantum 16 milliseconds
- Q_2 – FCFS

■ Scheduling

- A new job enters queue Q_0 which is served RR with $q=8$
 - ▶ When it gains CPU, job receives 8 milliseconds
 - ▶ If it does not finish in 8 milliseconds, job is moved to queue Q_1
- At Q_1 job is again served RR with $q=16$ and receives 16 additional milliseconds
 - ▶ If it still does not complete, it is preempted and moved to queue Q_2





Thread Scheduling: Contention Scope

- User-level and kernel-level threads are scheduled differently.
- When threads are supported by the kernel, threads are scheduled, not processes.
- Remember – user threads are mapped to kernel threads.
- Question – do all threads of all processes compete for CPUs equally, or do the threads of each process compete against each other for the CPU, with each process being given a share of CPU time?
 - This is the question of thread **contention scope** – which threads does a thread contend with (i.e., compete against) for the CPU?
 - Two types: **process-contention** and **system-contention** scopes.





Process Contention Scope

- With the many-to-one and many-to-many models, the thread library schedules user-level threads to run on LWPs.
 - Implies that all threads in a single process get scheduled onto fixed set of LWPs, which in turn get scheduled onto CPUs.
 - So threads within a process compete for LWPs.
 - Known as **process-contention scope (PCS)** since scheduling competition is within the process.
 - Typically done via priority set by programmer





System Contention Scope

- Kernel threads are scheduled onto available CPUs, so all kernel threads compete with each other across the entire system.
- This is called **system-contention scope (SCS)** – competition among all threads in the system.
- The one-to-one thread model (used by Windows and Linux) uses SCS only.





Pthread Scheduling

- The Pthread API allows specifying either PCS or SCS during thread creation
 - PTHREAD_SCOPE_PROCESS schedules threads using PCS scheduling
 - PTHREAD_SCOPE_SYSTEM schedules threads using SCS scheduling
- Although the API allows the choice, can be overridden by OS – Linux and macOS only allow PTHREAD_SCOPE_SYSTEM





Pthread Scheduling API Example

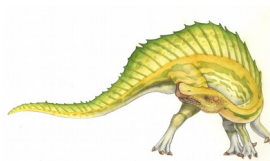
```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[]) {
    int i, scope;
    pthread_t tid[NUM_THREADS];
    pthread_attr_t attr;
    pthread_attr_init(&attr); /* apply default attributes */
    /* inquire about the current scope */
    if (pthread_attr_getscope(&attr, &scope) != 0)
        fprintf(stderr, "Unable to get scheduling scope\n");
    else {
        if (scope == PTHREAD_SCOPE_PROCESS)
            printf("thread has PTHREAD_SCOPE_PROCESS");
        else if (scope == PTHREAD_SCOPE_SYSTEM)
            printf("thread has PTHREAD_SCOPE_SYSTEM");
        else
            fprintf(stderr, "Illegal scope value.\n");
    }
}
```





Pthread Scheduling API

```
/* set the scheduling algorithm to PCS */
pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);
/* create the threads */
for (i = 0; i < NUM_THREADS; i++)
    pthread_create(&tid[i], &attr, runner, NULL);
/* now join on each thread */
for (i = 0; i < NUM_THREADS; i++)
    pthread_join(tid[i], NULL);
}
/* Each thread will begin control in this function */
void *runner(void *param)
{
    /* do some work ... */
    pthread_exit(0);
}
```





Multiple-Processor Scheduling

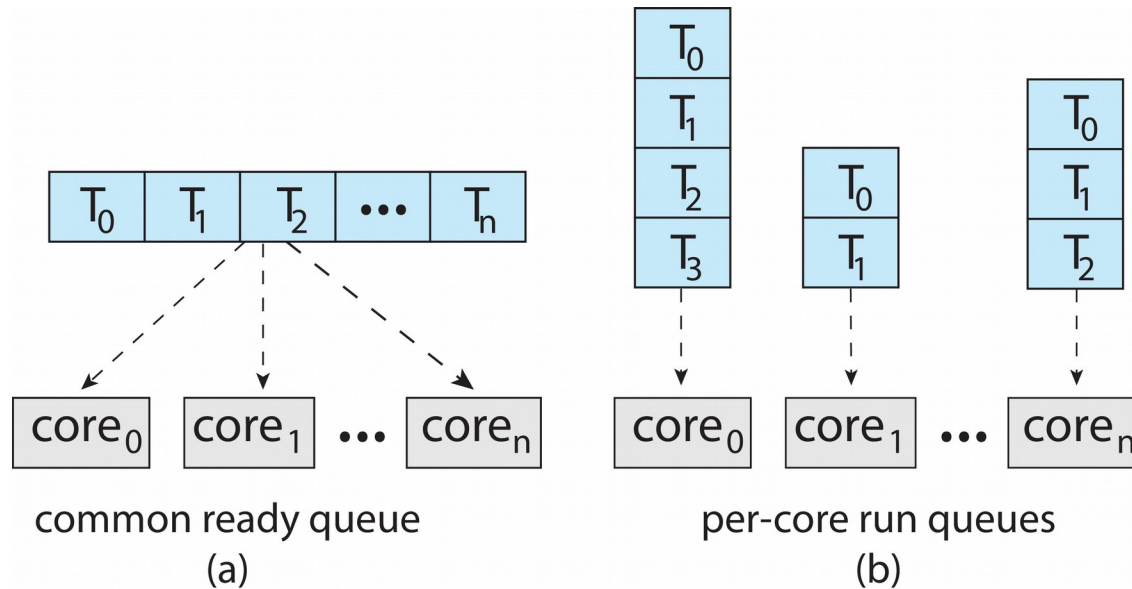
- CPU scheduling more complex when multiple CPUs are available
- A multiprocessor may be any one of the following architectures:
 - Multi-core processor
 - Multi-threaded cores
 - A multi-core processor with multi-threaded cores
 - NUMA systems
 - Heterogeneous multiprocessing





Multiple-Processor Scheduling (2)

- Symmetric multiprocessing (SMP) is where each processor is self scheduling. Two scheduling models:
 - All threads may be in a shared ready queue (a)
 - Each processor may have its own private queue of threads (b)



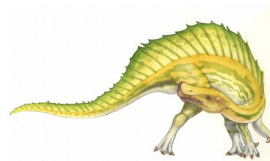
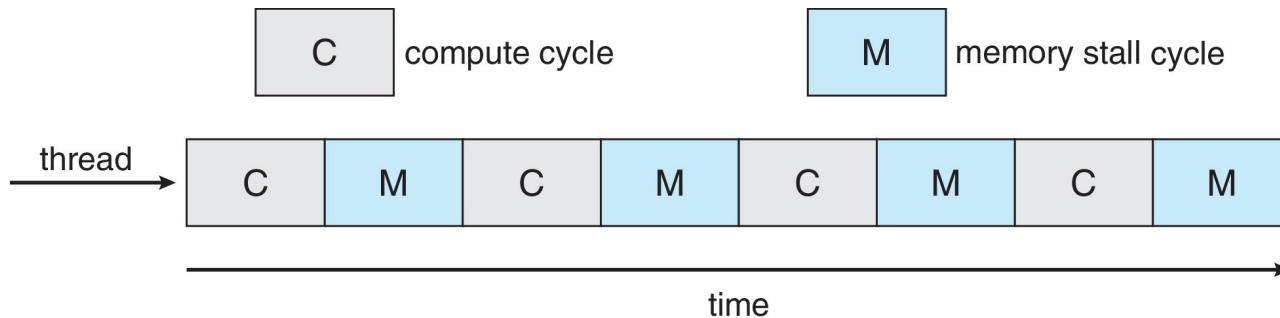
- Shared queue has race condition and needs synchronization/locking





Multi-core Processors

- Multiple processor cores on same physical chip
- Faster and consumes less power
- Multiple threads per core also growing
 - Sometimes called hyperthreading
 - Takes advantage of memory stall to make progress on another thread while memory retrieve happens

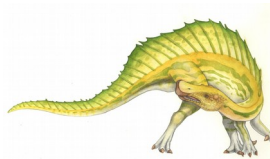
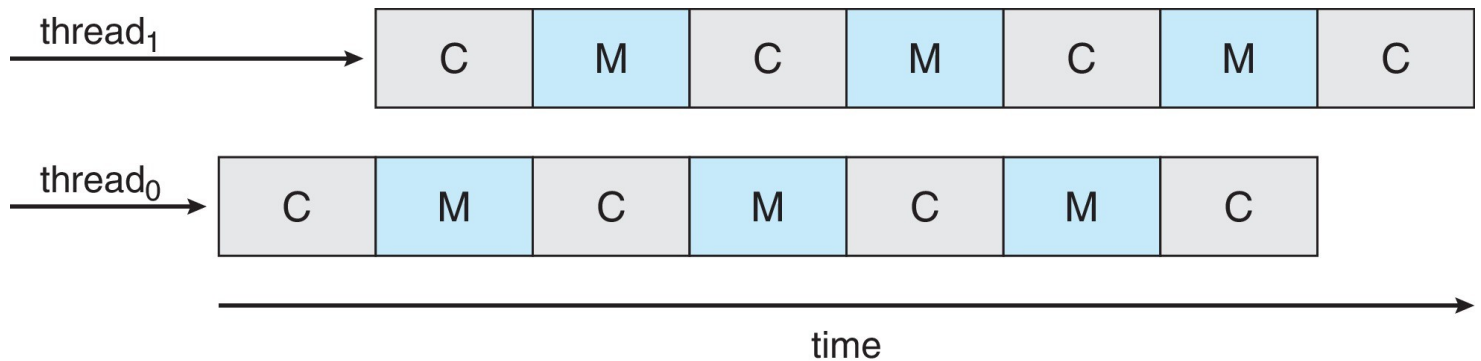




Multi-threaded Multi-core System

Each core has > 1 hardware threads.

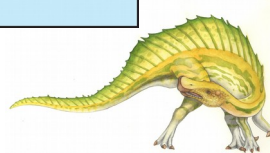
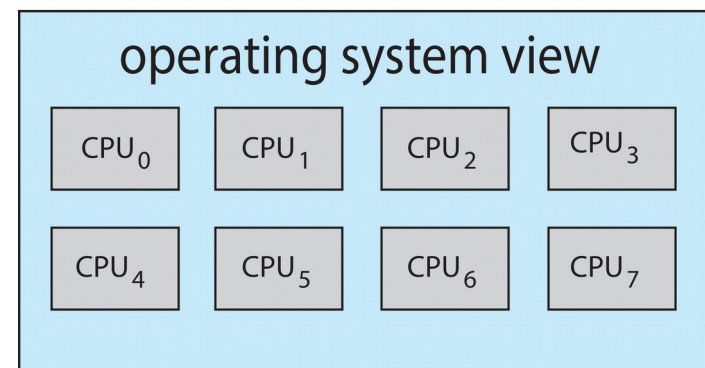
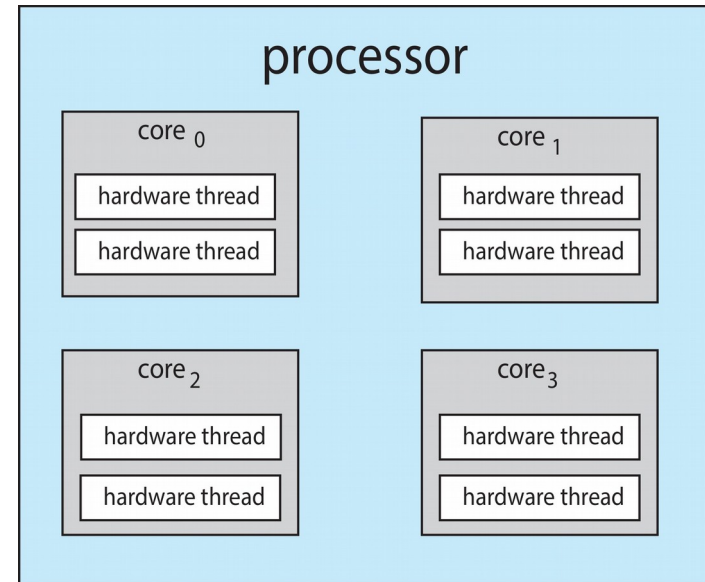
If one thread has a memory stall, switch to another thread!





Multi-threaded Multi-core System

- **Chip-multithreading (CMT)** assigns each core multiple hardware threads. (Intel refers to this as **hyperthreading**.)
- On a quad-core system with 2 hardware threads per core, the operating system sees 8 logical processors.

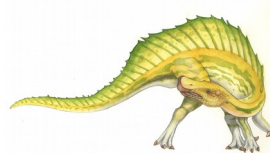
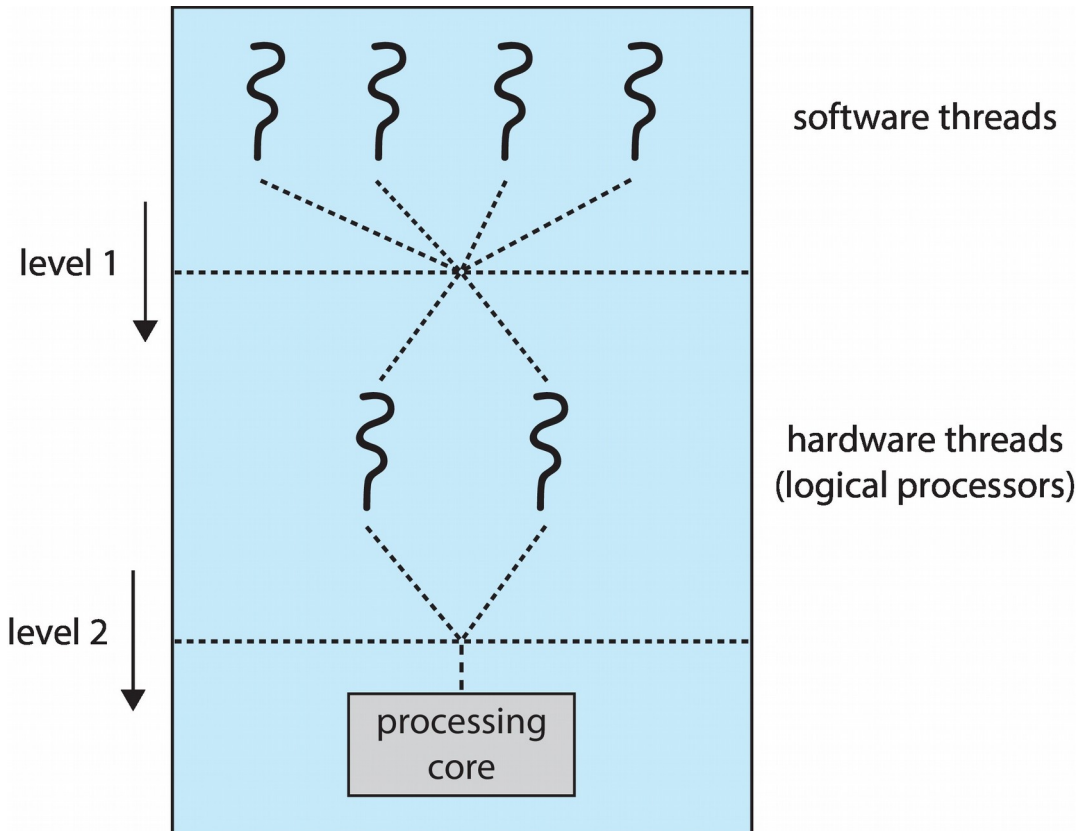




Multi-threaded Multi-core System

■ Two levels of scheduling:

1. The operating system deciding which software thread to run on a logical CPU
2. How each core decides which hardware thread to run on the physical core.





Multiple-Processor Scheduling – Load Balancing

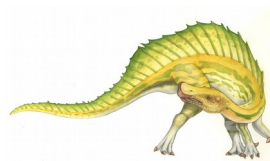
- If SMP, need to keep all CPUs loaded for efficiency
- **Load balancing** attempts to keep workload evenly distributed
- Two paradigms:
 - **Push migration** – periodic task checks load on each processor, and if found pushes task from overloaded CPU to other CPUs
 - **Pull migration** – idle processors pull waiting task from busy processor
 - Both can be used in combination (Linux CFS does)





Multiple-Processor Scheduling – Processor Affinity

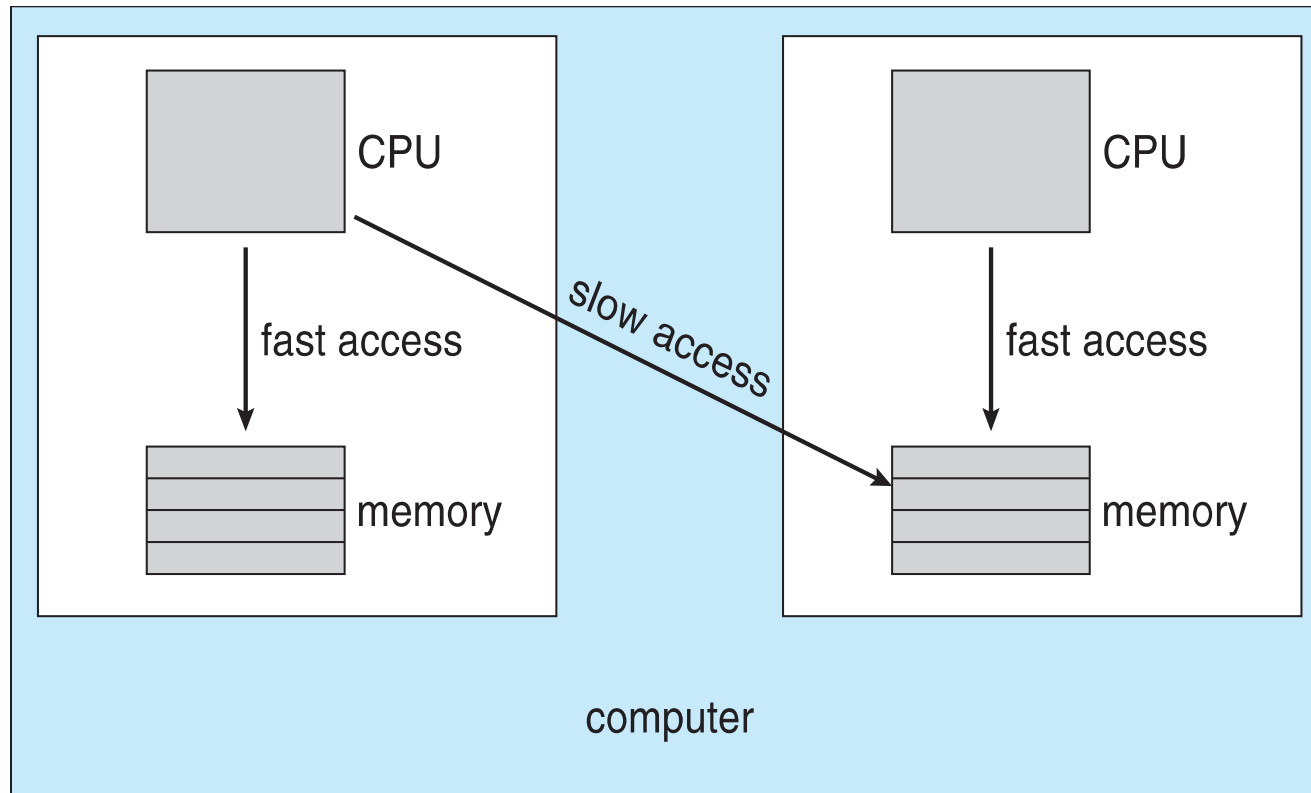
- When a thread has been running on one processor, the cache contents of that processor stores the memory accesses by that thread.
- We refer to this as a thread having affinity for a processor (i.e. “processor affinity”)
- Load balancing may affect processor affinity as a thread may be moved from one processor to another to balance loads, yet that thread loses the contents of what it had in the cache of the processor it was moved off of.
- **Soft affinity** – the operating system attempts to keep a thread running on the same processor, but no guarantees.
- **Hard affinity** – allows a thread to specify a set of processors it may run on.





NUMA and CPU Scheduling

If the operating system is **NUMA-aware**, it will assign memory close to the CPU the thread is running on.





Real-Time CPU Scheduling

- Can present obvious challenges
- **Soft real-time systems** – Critical real-time tasks have the highest priority, but no guarantee as to when tasks will be scheduled
- **Hard real-time systems** – task must be serviced by its deadline
- We will not cover real-time scheduling in this class





Linux Scheduling Through Version 2.5

- Prior to kernel version 2.5, ran variation of standard UNIX scheduling algorithm
- Version 2.5 moved to constant order $O(1)$ scheduling time
 - Preemptive, priority based
 - Two priority ranges: time-sharing and real-time
 - **Real-time** range from 0 to 99 and **nice** value from 100 to 140
 - Map into global priority with numerically lower values indicating higher priority
 - Higher priority gets larger q
 - Task run-able as long as time left in time slice (**active**)
 - If no time left (**expired**), not run-able until all other tasks use their slices
 - All run-able tasks tracked in per-CPU **runqueue** data structure
 - ▶ Two priority arrays (active, expired)
 - ▶ Tasks indexed by priority
 - ▶ When no more active, arrays are exchanged
 - Worked well, but poor response times for interactive processes

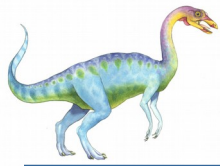




Linux Scheduling in Version 2.6.23 +

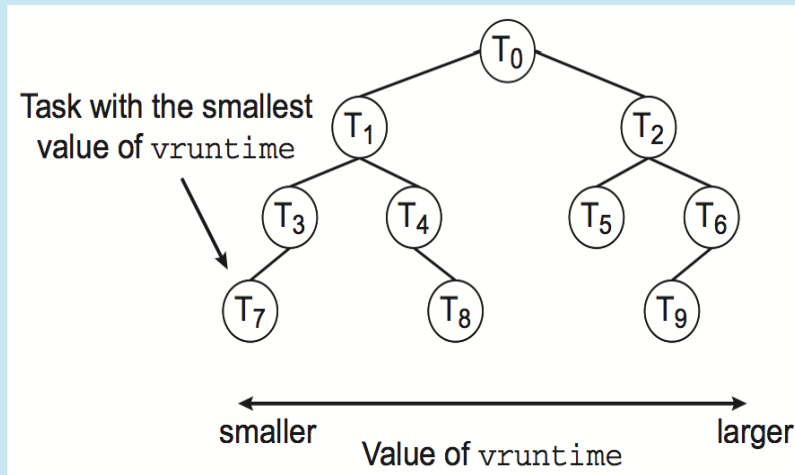
- **Completely Fair Scheduler (CFS)**
- **Scheduling classes**
 - Each has specific priority
 - Scheduler picks highest priority task in highest scheduling class
 - Rather than quantum based on fixed time allotments, based on proportion of CPU time
 - 2 scheduling classes included, others can be added
 1. default
 2. real-time
- Quantum calculated based on **nice value** from -20 to +19
 - Lower value is higher priority
 - Calculates **target latency** – interval of time during which task should run at least once
 - Target latency can increase if say number of active tasks increases
- CFS scheduler maintains per task **virtual run time** in variable **vruntime**
 - Associated with decay factor based on priority of task – lower priority is higher decay rate
 - Normal default priority yields virtual run time = actual run time
- To decide next task to run, scheduler picks task with lowest virtual run time



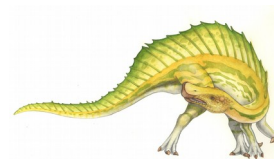


CFS Performance

The Linux CFS scheduler provides an efficient algorithm for selecting which task to run next. Each runnable task is placed in a red-black tree—a balanced binary search tree whose key is based on the value of `vruntime`. This tree is shown below:



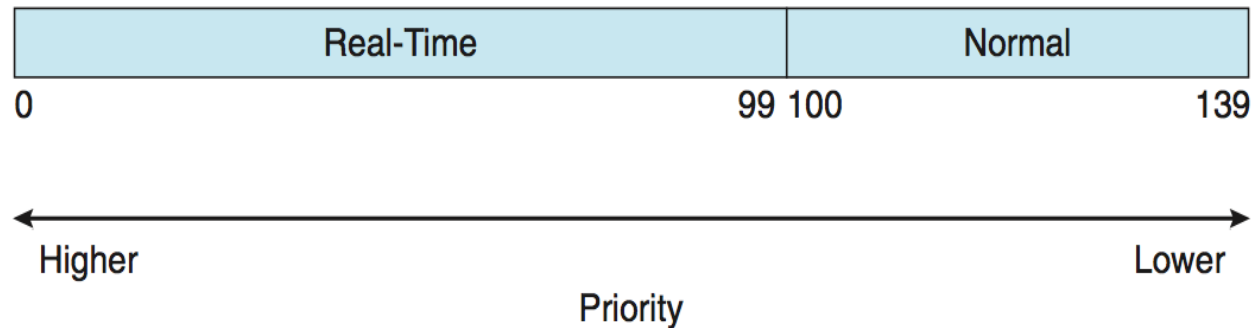
When a task becomes runnable, it is added to the tree. If a task on the tree is not runnable (for example, if it is blocked while waiting for I/O), it is removed. Generally speaking, tasks that have been given less processing time (smaller values of `vruntime`) are toward the left side of the tree, and tasks that have been given more processing time are on the right side. According to the properties of a binary search tree, the leftmost node has the smallest key value, which for the sake of the CFS scheduler means that it is the task with the highest priority. Because the red-black tree is balanced, navigating it to discover the leftmost node will require $O(\lg N)$ operations (where N is the number of nodes in the tree). However, for efficiency reasons, the Linux scheduler caches this value in the variable `rb_leftmost`, and thus determining which task to run next requires only retrieving the cached value.





Linux Scheduling (Cont.)

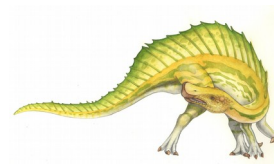
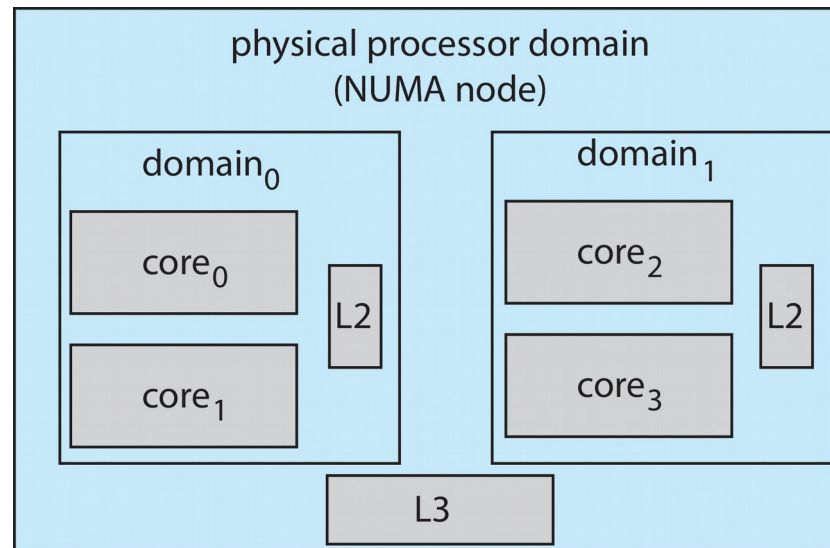
- Real-time scheduling according to POSIX.1b
 - Real-time tasks have static priorities
- Real-time plus normal map into global priority scheme
- Nice value of -20 maps to global priority 100
- Nice value of +19 maps to priority 139



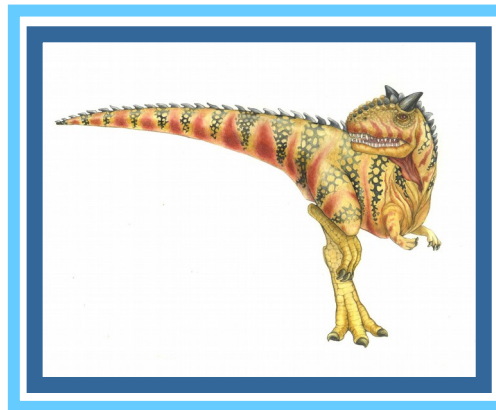


Linux Scheduling (Cont.)

- Linux supports load balancing, but is also NUMA-aware.
- **Scheduling domain** is a set of CPU cores that can be balanced against one another.
- Domains are organized by what they share (i.e. cache memory.) Goal is to keep threads from migrating between domains.



Chapter 6: Synchronization Tools





Chapter 6: Synchronization Tools

- Background
- The Critical-Section Problem
- Software Solution: Peterson's Solution
- Hardware Support for Synchronization
- Mutex Locks
- Semaphores
- Monitors
- Liveness





Objectives

- Describe the critical-section problem and illustrate a race condition
- Illustrate hardware solutions to the critical-section problem using memory barriers, compare-and-swap operations, and atomic variables
- Demonstrate how mutex locks, semaphores, monitors, and condition variables can be used to solve the critical section problem





The Problem

- Processes can execute concurrently
 - May be interrupted at any time, partially completing execution
- Concurrent access to shared data may result in data inconsistency – two processes holding different values of same data object, such as a shared counter
- Maintaining data consistency requires mechanisms to ensure the controlled execution of cooperating processes
- Illustration of the problem:

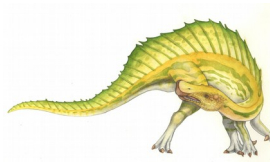
Suppose that we wanted to provide a solution to the consumer-producer problem (from Chapter 4) that fills **all** the buffers. We can do so by having an integer **counter** that keeps track of the number of full buffers. Initially, **counter** is set to 0. It is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.





Producer

```
while (true) {  
    /* produce an item in next produced */  
  
    while (counter == BUFFER_SIZE)  
        ; /* do nothing */  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```





Consumer

```
while (true) {  
    while (counter == 0)  
        ; /* do nothing */  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
    /* consume the item in next consumed */  
}
```





Race Condition

- **counter++** could be implemented as

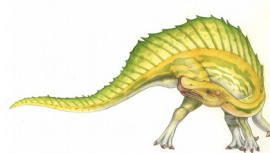
```
register1 = counter
register1 = register1 + 1
counter = register1
```

- **counter--** could be implemented as

```
register2 = counter
register2 = register2 - 1
counter = register2
```

- Consider this execution interleaving with “count = 5” initially:

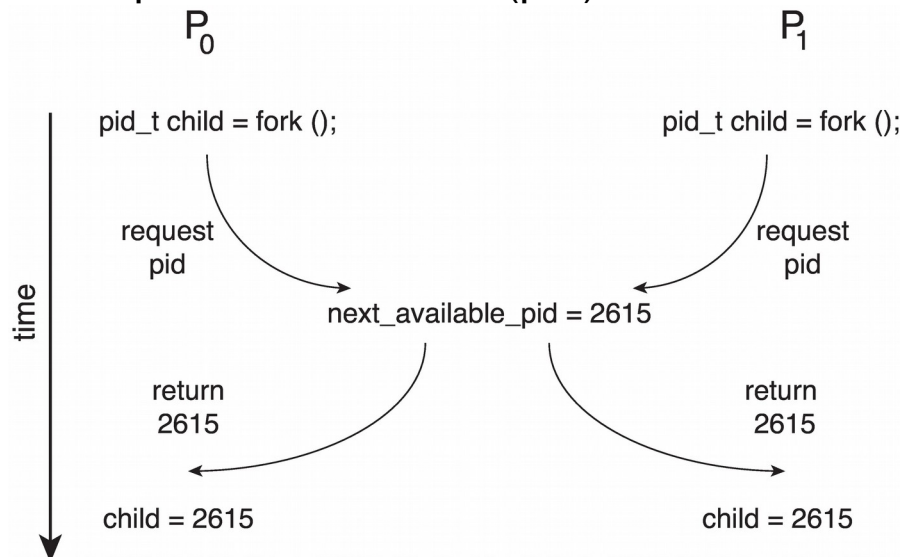
S0: producer execute	register1 = counter	{register1 = 5}
S1: producer execute	register1 = register1 + 1	{register1 = 6}
S2: consumer execute	register2 = counter	{register2 = 5}
S3: consumer execute	register2 = register2 - 1	{register2 = 4}
S4: producer execute	counter = register1	{counter = 6}
S5: consumer execute	counter = register2	{counter = 4}





Race Condition Example 2

- Processes P_0 and P_1 are creating child processes using the `fork()` system call
- Race condition on kernel variable `next_available_pid` which represents the next available process identifier (pid)



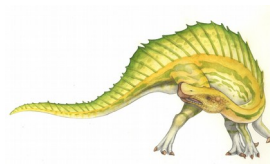
- Unless there is mutual exclusion, the same pid could be assigned to two different processes!





Critical Section Problem

- Consider system of n processes $\{p_0, p_1, \dots p_{n-1}\}$
- Each process has **critical section** segment of code
 - Process may be changing common variables, updating table, writing file, etc
 - When one process in critical section, no other may be in its critical section
- **Critical section problem** is to design protocol to solve this
- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**





Critical Section

- General structure of process P_i

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (true);
```





Solution to Critical-Section Problem

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely
3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
 - Assume that each process executes at a nonzero speed
 - No assumption concerning **relative speed** of the n processes

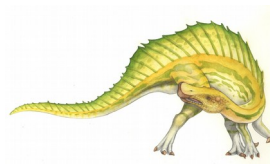




Critical-Section Handling in OS

Two approaches depending on if kernel is preemptive or non-preemptive

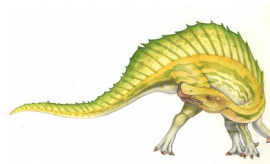
- **Preemptive** – allows preemption of process when running in kernel mode
- **Non-preemptive** – runs until exits kernel mode, blocks, or voluntarily yields CPU
 - ▶ Essentially free of race conditions in kernel mode





Peterson's Solution

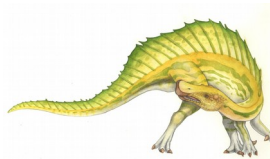
- Not guaranteed to work on modern architectures! (But good algorithmic description of solving the problem)
- Two process solution
- Assume that the **load** and **store** machine-language instructions are atomic; that is, cannot be interrupted
- The two processes share two variables:
 - **int turn;**
 - **boolean flag[2]**
- The variable **turn** indicates whose turn it is to enter the critical section
- The **flag** array is used to indicate if a process is ready to enter the critical section. **flag[i] = true** implies that process **P_i** is ready!





Algorithm for Process P_i

```
while (true){  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j)  
        ;  
  
    /* critical section */  
  
    flag[i] = false;  
  
    /* remainder section */  
  
}
```





Peterson's Solution (3)

- Provable that the three CS requirement are met:

1. Mutual exclusion is preserved

P_i enters CS only if:

either **flag[j] = false** or **turn = i**

2. Progress requirement is satisfied

3. Bounded-waiting requirement is met





Peterson's Solution (4)

- Although useful for demonstrating an algorithm, Peterson's Solution is not guaranteed to work on modern architectures.
- Understanding why it will not work is also useful for better understanding race conditions.
- To improve performance, processors and/or compilers may reorder operations that have no dependencies.
- For single-threaded this is ok as the result will always be the same.
- For multithreaded the reordering may produce inconsistent or unexpected results!





Peterson's Solution (5)

- Two threads share the data:

```
boolean flag = false;  
int x = 0;
```

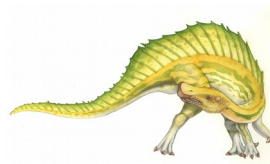
- Thread 1 performs

```
while (!flag)  
;  
print x
```

- Thread 2 performs

```
x = 100;  
flag = true
```

- What is the expected output?



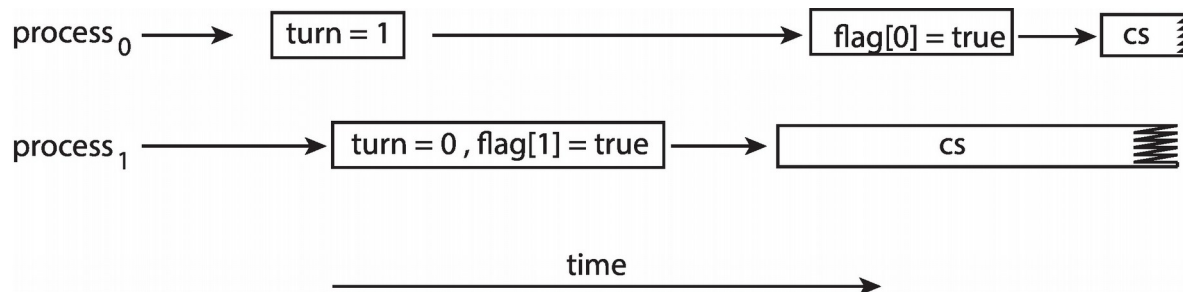


Peterson's Solution (6)

- 100 is the expected output.
- However, the operations for Thread 2 may be reordered:

```
flag = true;  
x = 100;
```

- If this occurs, the output may be 0!
- The effects of instruction reordering in Peterson's Solution



- This allows both processes to be in their critical section at the same time!





Synchronization Hardware

- Many systems provide hardware support for implementing the critical section code.
- Uniprocessors – could disable interrupts
 - Currently running code would execute without preemption
 - Generally too inefficient on multiprocessor systems
 - ▶ Operating systems using this not broadly scalable
- We will look at three forms of hardware support:
 1. Memory barriers
 2. Hardware instructions
 3. Atomic variables





Memory Barriers

- **Memory model** are the memory guarantees a computer architecture makes to application programs.
- Memory models may be either:
 - **Strongly ordered** – where a memory modification of one processor is immediately visible to all other processors.
 - **Weakly ordered** – where a memory modification of one processor may not be immediately visible to all other processors.
- A **memory barrier** is an instruction that forces any change in memory to be propagated (made visible) to all other processors.





Memory Barrier

- We could add a memory barrier to the following instructions to ensure Thread 1 outputs 100:
- Thread 1 now performs

```
while (!flag)
    memory_barrier();
print x
```

- Thread 2 now performs

```
x = 100;
memory_barrier();
flag = true
```





Hardware Instructions

- Special hardware instructions that allow us to either *test-and-modify* the content of a word, or two *swap* the contents of two words atomically (uninterruptibly.)
- **Test-and-Set** instruction
- **Compare-and-Swap** instruction





test_and_set Instruction

Definition:

```
boolean test_and_set (boolean *target)
{
    boolean rv = *target;
    *target = true;
    return rv;
}
```

1. Executed atomically
2. Returns the original value of passed parameter
3. Set the new value of passed parameter to **true**

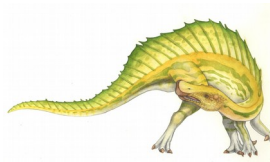




Solution using test_and_set()

- Shared boolean variable **lock**, initialized to **false**
- Solution:

```
do {  
    while (test_and_set(&lock))  
        ; /* do nothing */  
  
    /* critical section */  
  
    lock = false;  
    /* remainder section */  
} while (true);
```





compare_and_swap Instruction

Definition:

```
int compare_and_swap(int *value, int expected, int new_value) {  
    int temp = *value;  
  
    if (*value == expected)  
        *value = new_value;  
    return temp;  
}
```

1. Executed atomically
2. Returns the original value of passed parameter **value**
3. Set the variable **value** the value of the passed parameter **new_value** but only if ***value == expected** is true. That is, the swap takes place only under this condition.

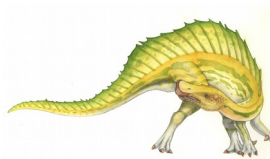




Solution using compare_and_swap

- Shared integer **lock** initialized to 0;
- Solution:

```
while (true){  
    while (compare_and_swap(&lock, 0, 1) != 0)  
        ; /* do nothing */  
  
    /* critical section */  
  
    lock = 0;  
  
    /* remainder section */  
}
```





Bounded-waiting Mutual Exclusion with compare-and-swap

```
while (true) {
    waiting[i] = true;
    key = 1;
    while (waiting[i] && key == 1)
        key = compare_and_swap(&lock, 0, 1);
    waiting[i] = false;
    /* critical section */
    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;
    if (j == i)
        lock = 0;
    else
        waiting[j] = false;
    /* remainder section */
}
```

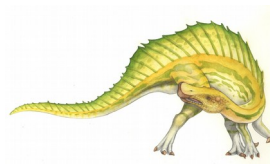




Atomic Variables

- Typically, instructions such as compare-and-swap are used as building blocks for other synchronization tools.
- One tool is an **atomic variable** that provides *atomic* (uninterruptible) updates on basic data types such as integers and booleans.
- For example, the **increment()** operation on the atomic variable **sequence** ensures **sequence** is incremented without interruption:

increment(&sequence);





Atomic Variables (2)

- The `increment()` function can be implemented as follows:

```
void increment(atomic_int *v)
{
    int temp;

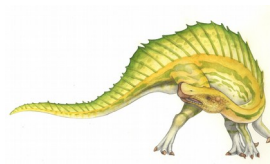
    do {
        temp = *v;
    }
    while (temp != (compare_and_swap(v, temp, temp+1)));
}
```





Mutex Locks

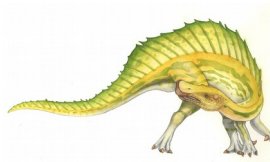
- Previous solutions are complicated and generally inaccessible to application programmers
- OS designers build software tools to solve critical section problem
- Simplest is mutex lock
- Protect a critical section by first **acquire()** a lock then **release()** the lock
 - Boolean variable indicating if lock is available or not
- Calls to **acquire()** and **release()** must be atomic
 - Usually implemented via hardware atomic instructions such as compare-and-swap.
- But this solution requires **busy waiting**
 - This lock therefore called a **spinlock**





Solution to Critical-section Problem Using Locks

```
while (true) {  
    acquire lock  
    critical section  
    release lock  
    remainder section  
}
```

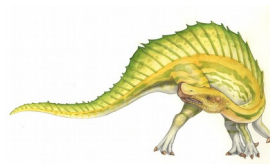


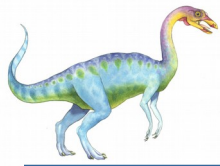


Mutex Lock Definitions

- `acquire() {
 while (!available)
 ; /* busy wait */
 available = false;;
}`
- `release() {
 available = true;
}`

These two functions must be implemented atomically. Both test-and-set and compare-and-swap can be used to implement these functions.





Semaphore

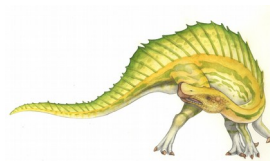
- Synchronization tool that provides more sophisticated ways (than Mutex locks) for process to synchronize their activities.
- Semaphore **S** – integer variable
- Can only be accessed via two indivisible (atomic) operations
 - **wait()** and **signal()**
 - ▶ (Originally called **P()** and **V()**)

- Definition of the **wait()** operation

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

- Definition of the **signal()** operation

```
signal(S) {  
    S++;  
}
```





Semaphore Usage

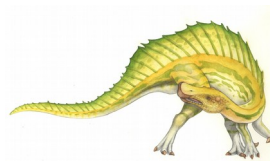
- **Counting semaphore** – integer value can range over an unrestricted domain
 - **Binary semaphore** – integer value can range only between 0 and 1
 - Same as a **mutex lock**
 - Can solve various synchronization problems
 - Consider P_1 and P_2 that require S_1 to happen before S_2
Create a semaphore “**synch**” initialized to 0
- P1:**
- ```
S1;
signal(synch);
```
- P2:**
- ```
wait(synch);  
S2;
```
- Can implement a counting semaphore **S** as a binary semaphore





Semaphore Implementation

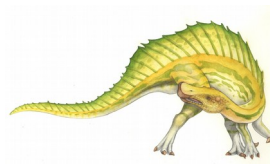
- Must guarantee that no two processes can execute the **wait()** and **signal()** on the same semaphore at the same time
- Thus, the implementation becomes the critical section problem where the **wait** and **signal** code are placed in the critical section
 - Could now have **busy waiting** in critical section implementation
 - ▶ But implementation code is short
 - ▶ Little busy waiting if critical section rarely occupied
- Note that applications may spend lots of time in critical sections and therefore this is not a good solution





Semaphore Implementation with no Busy waiting

- With each semaphore there is an associated waiting queue
- Each entry in a waiting queue has two data items:
 - value (of type integer)
 - pointer to next record in the list
- Two operations:
 - **block** – place the process invoking the operation on the appropriate waiting queue
 - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue
- `typedef struct {
 int value;
 struct process *list;
} semaphore;`





Implementation with no Busy waiting (Cont.)

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}

signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```





Problems with Semaphores

- Incorrect use of semaphore operations:
 - `signal (mutex) .... wait (mutex)`
 - `wait (mutex) ... wait (mutex)`
 - Omitting of `wait (mutex)` and/or `signal (mutex)`
- These – and others – are examples of what can occur when semaphores and other synchronization tools are used incorrectly.





Monitors

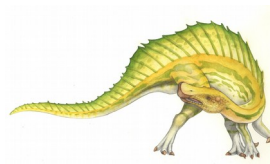
- A high-level abstraction that provides a convenient and effective mechanism for process synchronization
- *Abstract data type*, internal variables only accessible by code within the procedure
- Only one process may be active within the monitor at a time
- Pseudocode syntax of a monitor:

```
monitor monitor-name
{
    // shared variable declarations
    function P1 (...) { ... }

    function P2 (...) { ... }

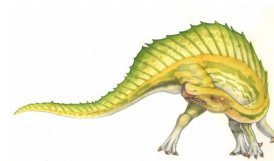
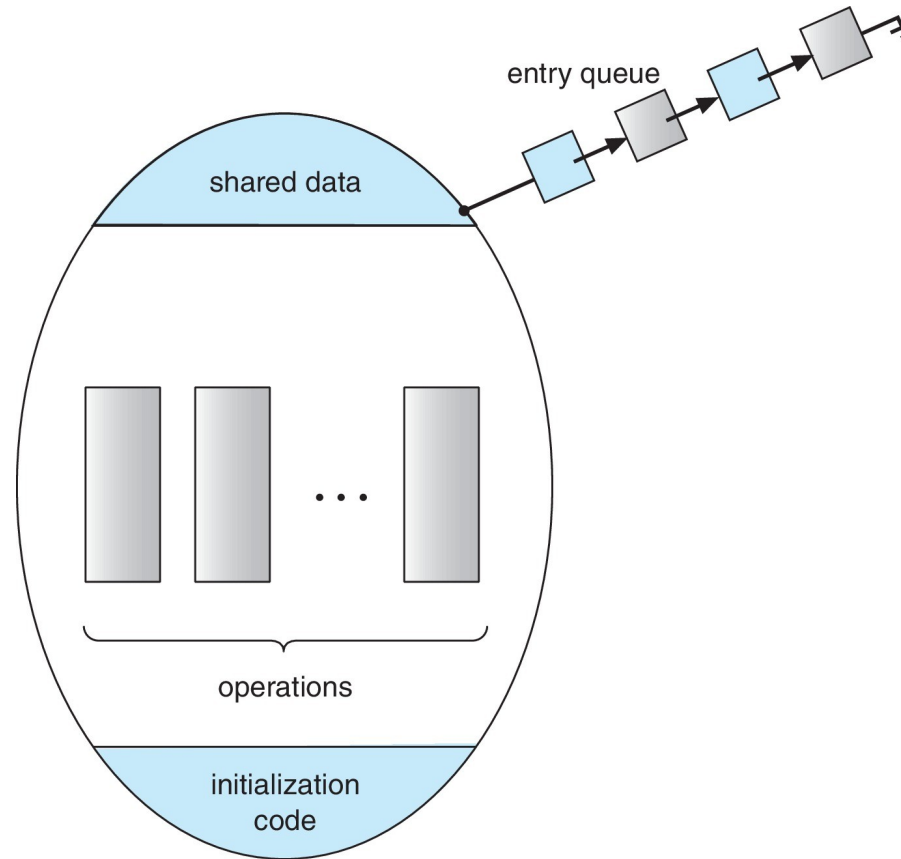
    function Pn (...) {.....}

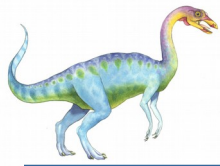
    initialization code (...) { ... }
}
```





Schematic view of a Monitor





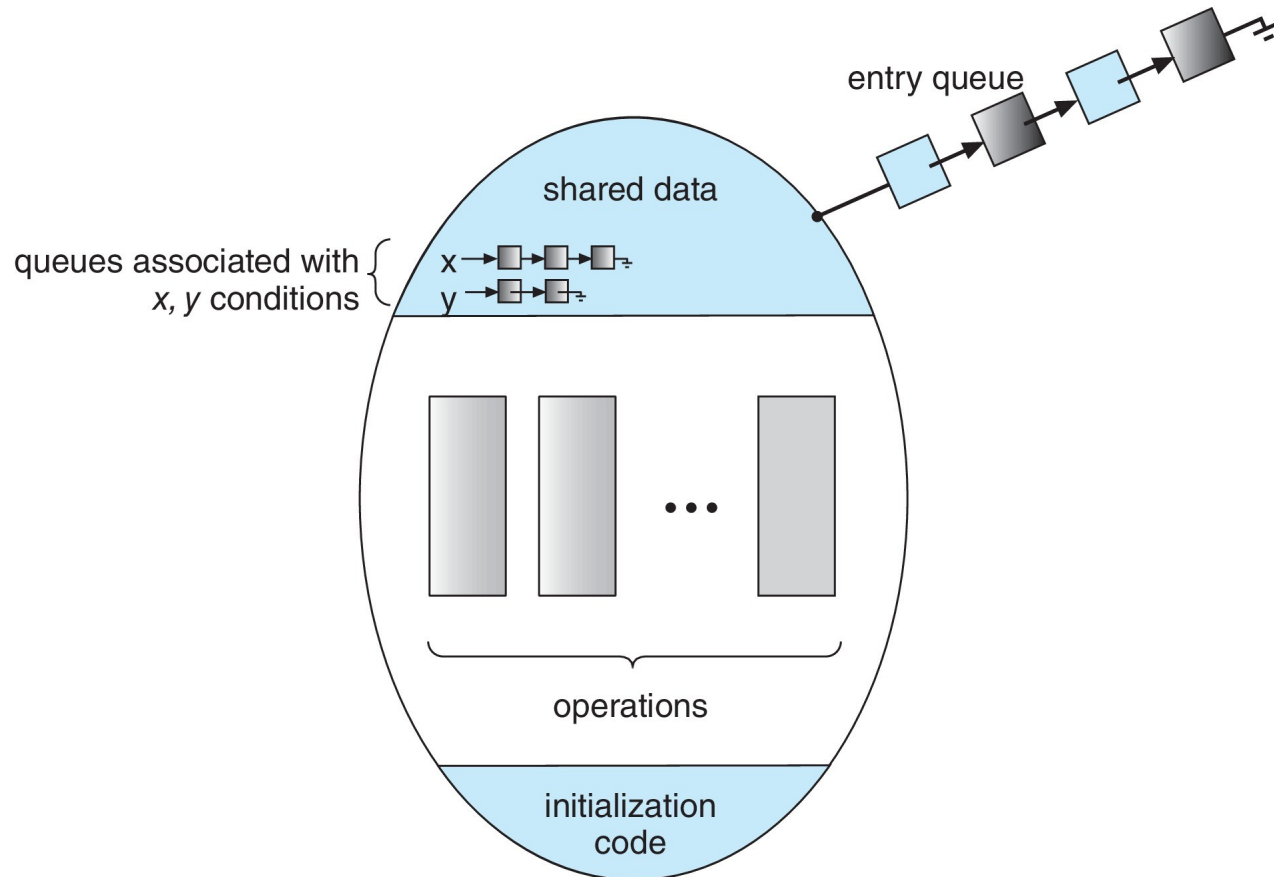
Condition Variables

- **condition** *x*, *y*;
- Two operations are allowed on a condition variable:
 - ***x.wait()*** – a process that invokes the operation is suspended until ***x.signal()***
 - ***x.signal()*** – resumes one of processes (if any) that invoked ***x.wait()***
 - ▶ If no ***x.wait()*** on the variable, then it has no effect on the variable





Monitor with Condition Variables





Condition Variables Choices

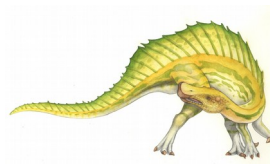
- If process P invokes **x.signal()**, and process Q is suspended in **x.wait()**, what should happen next?
 - Both Q and P cannot execute in parallel. If Q is resumed, then P must wait
- Options include
 - **Signal and wait** – P waits until Q either leaves the monitor or it waits for another condition
 - **Signal and continue** – Q waits until P either leaves the monitor or it waits for another condition
 - Both have pros and cons – language implementer can decide
 - Monitors implemented in Concurrent Pascal compromise
 - ▶ P executing signal immediately leaves the monitor, Q is resumed
 - Implemented in other languages including Mesa, C#, Java

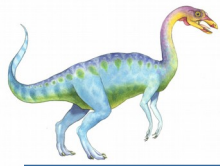




Resuming Processes within a Monitor

- If several processes queued on condition variable `x`, and `x.signal()` is executed, which process should be resumed?
- FCFS frequently not adequate
- **conditional-wait** construct of the form `x.wait(c)`
 - Where `c` is **priority number**
 - Process with lowest number (highest priority) is scheduled next





Monitor Application: Single Resource Allocation

- Allocate a single resource among competing processes using priority numbers that specify the maximum time a process plans to use the resource

```
R.acquire(t);  
    ...  
    access the resource;  
    ...  
  
R.release;
```

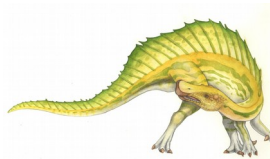
- Where R is an instance of type **ResourceAllocator**





A Monitor to Allocate Single Resource

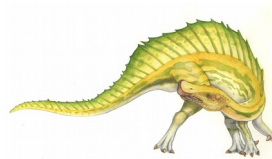
```
monitor ResourceAllocator
{
    boolean busy;
    condition x;
    void acquire(int p) {
        if (busy)
            x.wait(p);
        busy = true;
    }
    void release() {
        busy = false;
        x.signal();
    }
    initialization code() {
        busy = false;
    }
}
```





Liveness

- Processes may have to wait indefinitely while trying to acquire a synchronization tool such as a mutex lock or semaphore.
- Waiting indefinitely violates the progress and bounded-waiting criteria discussed at the beginning of this chapter.
- **Liveness** refers to a set of properties that a system must satisfy to ensure processes make progress.
- Indefinite waiting is an example of a liveness failure.





Liveness

- **Deadlock** – The state in which two or more processes are waiting indefinitely for an event that can be caused only by one of the waiting processes
- Let **S** and **Q** be two semaphores initialized to 1

P_0	P_1
<code>wait(S);</code>	<code>wait(Q);</code>
<code>wait(Q);</code>	<code>wait(S);</code>
<code>...</code>	<code>...</code>
<code>signal(S);</code>	<code>signal(Q);</code>
<code>signal(Q);</code>	<code>signal(S);</code>

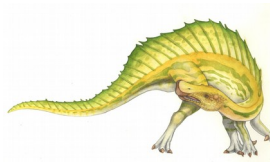
- Consider if P_0 executes `wait(S)` and P_1 `wait(Q)`. When P_0 executes `wait(Q)`, it must wait until P_1 executes `signal(Q)`
- However, P_1 is waiting until P_0 execute `signal(S)`.
- Since these `signal()` operations will never be executed, P_0 and P_1 are **deadlocked**.





Liveness

- Other forms of deadlock:
- **Starvation** – indefinite blocking
 - A process may never be removed from the semaphore queue in which it is suspended
- **Priority Inversion** – Scheduling problem when lower-priority process holds a lock needed by higher-priority process
- Solved via **priority-inheritance protocol**



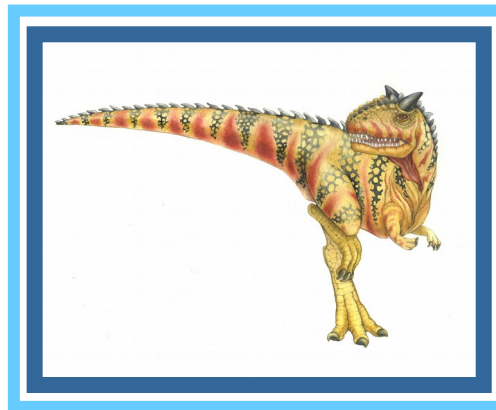


Priority Inheritance Protocol

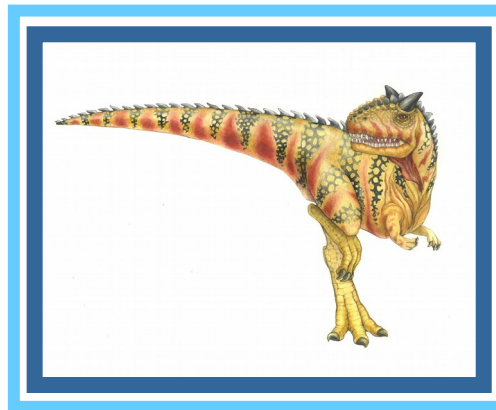
- Consider the scenario with three processes **P1**, **P2**, and **P3**. **P1** has the highest priority, **P2** the next highest, and **P3** the lowest. Assume a process **P3** is currently using a resource **R** that **P1** wants. Thus, **P1** must wait for **P3** to finish using the resource. However, **P2** becomes runnable and preempts **P3**. What has happened is that **P2** - a process with a lower priority than **P1** - has indirectly prevented **P3** from gaining access to the resource.
- To prevent this from occurring, a **priority inheritance protocol** is used:
 - All processes that are accessing resources needed by a higher-priority process inherit the higher priority until they are finished with the resources in question.
 - When they are finished, their priorities revert to their original values.
 - Thus, the current owner of the resource is assigned the priority of the highest priority thread wishing to acquire the resource.



End of Chapter 6



Chapter 7: Synchronization Examples





7: Model Synchronization Problems

- The bounded-buffer, readers-writers, and dining philosophers synchronization problems.
- Tools used by Linux to solve synchronization problems.
- POSIX solutions to synchronization problems.





Classical Problems of Synchronization

- Classical problems used to test newly-proposed synchronization schemes
 - Bounded-Buffer Problem
 - Readers and Writers Problem
 - Dining-Philosophers Problem





Bounded-Buffer Problem

- n buffers, each can hold one item
- Semaphore **mutex** initialized to the value 1
- Semaphore **full** initialized to the value 0
- Semaphore **empty** initialized to the value n





Bounded Buffer Problem (Cont.)

- The structure of the producer process

```
while (true) {  
    ...  
    /* produce an item in next_produced */  
    ...  
    wait(empty);  
    wait(mutex);  
    ...  
    /* add next produced to the buffer */  
    ...  
    signal(mutex);  
    signal(full);  
}
```





Bounded Buffer Problem (Cont.)

- The structure of the consumer process

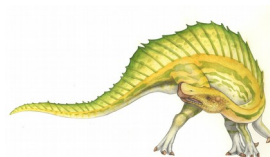
```
while (true) {  
    wait(full);  
    wait(mutex);  
    ...  
    /* remove an item from buffer to next_consumed */  
    ...  
    signal(mutex);  
    signal(empty);  
    ...  
    /* consume the item in next consumed */  
    ...  
}
```





Readers-Writers Problem

- A data set is shared among a number of concurrent processes
 - **Readers** – only read the data set; they do **not** perform any updates
 - **Writers** – can both read and write
- Problem – allow multiple readers to read at the same time
 - Only one single writer can access the shared data at the same time
- Several variations of how readers and writers are considered – all involve some form of priorities
- Shared Data
 - Data set
 - Semaphore **rw_mutex** initialized to 1
 - Semaphore **mutex** initialized to 1
 - Integer **read_count** initialized to 0

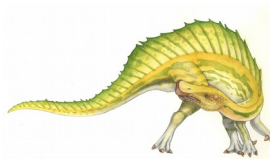




Readers-Writers Problem (Cont.)

- The structure of a writer process

```
while (true) {  
    wait(rw_mutex);  
  
    ...  
    /* writing is performed */  
    ...  
    signal(rw_mutex);  
}
```

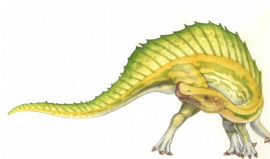




Readers-Writers Problem (Cont.)

- The structure of a reader process

```
while (true){  
    wait(mutex);  
        read_count++;  
        if (read_count == 1)  
            wait(rw_mutex);  
        signal(mutex);  
  
        ...  
        /* reading is performed */  
        ...  
        wait(mutex);  
        read_count--;  
        if (read_count == 0)  
            signal(rw_mutex);  
        signal(mutex);  
}
```





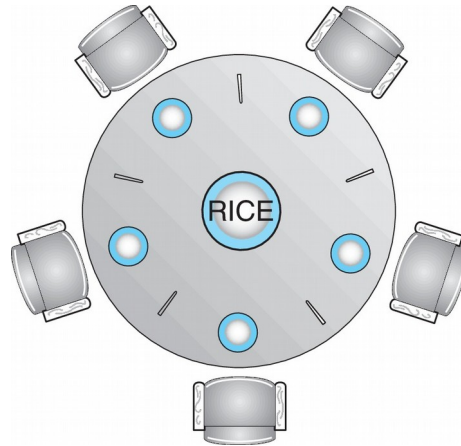
Readers-Writers Problem Variations

- **First** variation – no reader kept waiting unless writer has permission to use shared object
- **Second** variation – once writer is ready, it writes as soon as any existing writer finishes writing
- Both may have starvation leading to even more variations

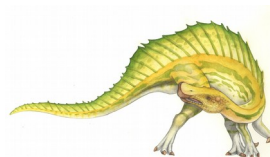




Dining Philosophers Problem

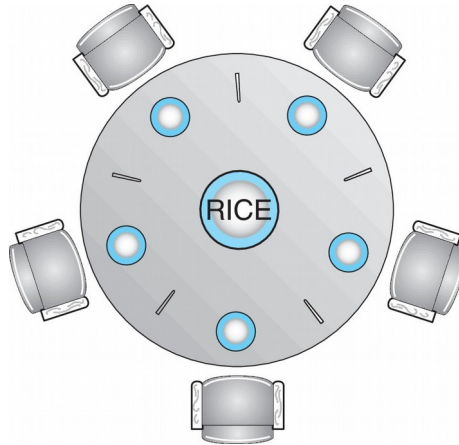


- Originally posed by Edsger Dijkstra in 1965 as a tape drive exercise for his students, and later formalized by C.A.R. Hoare.
- Five philosophers sit at a round table, and spend their lives alternating thinking and eating.
- They each have a bowl of spaghetti in front of them, and five forks are between the five bowls.
- They need two forks to eat. They cannot eat with just one.
- Problem has morphed over the years to bowls of rice and chopsticks.





Dining Philosophers Problem

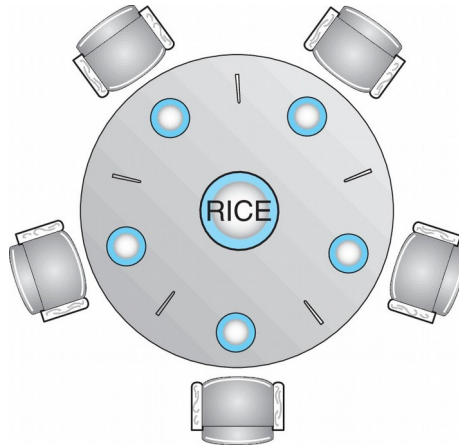


- Philosophers are independent – they do not interact with their neighbors. They try to pick up 2 chopsticks one after the other to eat from bowl
 - Need both chopsticks to eat, then release both when done
- Shared data:
 - Bowl of rice (data set)
 - Semaphore **chopstick** [5] initialized to 1

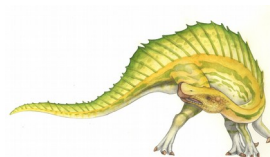




Dining Philosophers Problem



- Formal Solution Requirements:
 - Only one philosopher can hold a chopstick at a time.
 - It must be deadlock-free
 - It must be impossible for a philosopher to starve waiting for a chopstick.
 - It must be possible for more than one philosopher to eat at the same time.





Dining Philosophers Algorithm

- Solution using semaphores
- The structure of Philosopher *i*:

```
while (true){  
    wait (chopstick[i] );  
    wait (chopstick[ (i + 1) % 5] );  
  
    /* eat for awhile */  
  
    signal (chopstick[i] );  
    signal (chopstick[ (i + 1) % 5] );  
  
    /* think for awhile */  
  
}
```

- What is the problem with this algorithm?





Dining Philosophers #2

- A deadlock free solution uses an array of state variables and an array of semaphores:

```
sem_t mutex;  
sem_t S[N];  
int    state[N];
```

- A philosopher tries to take forks as follows

```
void take_forks(int ph_num)  
{  
    sem_wait(&mutex);  
    state[ph_num] = HUNGRY;  
  
    try_to_eat(ph_num);  
    signal(mutex);  
    wait(S[ph_num]); // wait here if could not eat  
    sleep(1);  
}
```





Dining Philosophers #2

- Before a philosopher picks up any chopsticks, she checks whether her neighbors are holding any with something like this:

```
if (state[i] == HUNGRY && state[(i+1)%5] != EATING
    && state[(i+4)%5] != EATING)
{
    state[i] = EATING;
    // can eat!!
    signal(S[i]);
}
```

- The putting down of forks:

```
void put_forks(int i)
{
    wait(mutex);
    state[i] = THINKING;
    try_to_eat((i+1)%5);
    try_to_eat((i+4)%5);
    signal(mutex);
}
```





Solution to Dining Philosophers (Cont.)

The code for trying to eat:

```
void try_to_eat (int i) {
    if ((state[(i + 4) % 5] != EATING) &&
        (state[i] == HUNGRY) &&
        (state[(i + 1) % 5] != EATING) ) {
        state[i] = EATING ;
        signal(S[i]) ;
    }
}

initialization_code() {
    for (int i = 0; i < 5; i++)
        state[i] = THINKING;
}
```





Linux Synchronization

- Linux:
 - Prior to kernel Version 2.6, disables interrupts to implement short critical sections
 - Version 2.6 and later, fully preemptive
- Linux provides:
 - Semaphores
 - atomic integers
 - spinlocks
 - reader-writer versions of both
- On single-cpu system, spinlocks replaced by enabling and disabling kernel preemption





Synchronization in Linux

- There are several options for process and thread synchronization in Linux.

- Atomic variables

atomic_t is the type for atomic integer

- Consider the variables

```
atomic_t counter;  
int value;
```

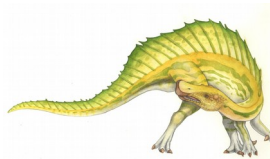
<i>Atomic Operation</i>	<i>Effect</i>
<code>atomic_set(&counter, 5);</code>	<code>counter = 5</code>
<code>atomic_add(10, &counter);</code>	<code>counter = counter + 10</code>
<code>atomic_sub(4, &counter);</code>	<code>counter = counter - 4</code>
<code>atomic_inc(&counter);</code>	<code>counter = counter + 1</code>
<code>value = atomic_read(&counter);</code>	<code>value = 12</code>





POSIX Synchronization

- POSIX API provides
 - mutex locks
 - semaphores
 - condition variable
- Widely used on UNIX, Linux, and macOS





POSIX Mutex Locks

■ Creating and initializing the lock

```
#include <pthread.h>

pthread_mutex_t mutex;

/* create and initialize the mutex lock */
pthread_mutex_init(&mutex, NULL);
```

■ Acquiring and releasing the lock

```
/* acquire the mutex lock */
pthread_mutex_lock(&mutex);

/* critical section */

/* release the mutex lock */
pthread_mutex_unlock(&mutex);
```





POSIX Semaphores

- POSIX provides two versions – **named** and **unnamed**.
- Named semaphores can be used by unrelated processes, unnamed cannot.





POSIX Named Semaphores

- Creating an initializing the semaphore:

```
#include <semaphore.h>
sem_t *sem;

/* Create the semaphore and initialize it to 1 */
sem = sem_open("SEM", O_CREAT, 0666, 1);
```

- Another process can access the semaphore by referring to its name **SEM**.
- Acquiring and releasing the semaphore:

```
/* acquire the semaphore */
sem_wait(sem);

/* critical section */

/* release the semaphore */
sem_post(sem);
```





POSIX Unnamed Semaphores

- Creating and initializing the semaphore:

```
#include <semaphore.h>
sem_t sem;

/* Create the semaphore and initialize it to 1 */
sem_init(&sem, 0, 1);
```

- Acquiring and releasing the semaphore:

```
/* acquire the semaphore */
sem_wait(&sem);

/* critical section */

/* release the semaphore */
sem_post(&sem);
```





POSIX Condition Variables

- Since POSIX is typically used in C/C++ and these languages do not provide a monitor, POSIX condition variables are associated with a POSIX mutex lock to provide mutual exclusion: Creating and initializing the condition variable:

```
pthread_mutex_t mutex;  
pthread_cond_t cond_var;  
  
pthread_mutex_init(&mutex, NULL);  
pthread_cond_init(&cond_var, NULL);
```





POSIX Condition Variables

- Thread waiting for the condition **a == b** to become true:

```
pthread_mutex_lock(&mutex);  
while (a != b)  
    pthread_cond_wait(&cond_var, &mutex);  
  
pthread_mutex_unlock(&mutex);
```

- Thread signaling another thread waiting on the condition variable:

```
pthread_mutex_lock(&mutex);  
a = b;  
pthread_cond_signal(&cond_var);  
pthread_mutex_unlock(&mutex);
```





Alternative Approaches

- Transactional Memory
- OpenMP
- Functional Programming Languages





Transactional Memory

- Consider a function `update()` that must be called atomically. One option is to use mutex locks:

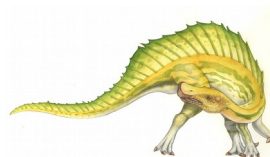
```
void update ()
{
    acquire();

    /* modify shared data */

    release();
}
```

- A **memory transaction** is a sequence of read-write operations to memory that are performed atomically. A transaction can be completed by adding **`atomic{S}`** which ensure statements in **`S`** are executed atomically.

```
void update ()
{
    atomic {
        /* modify shared data */
    }
}
```





OpenMP

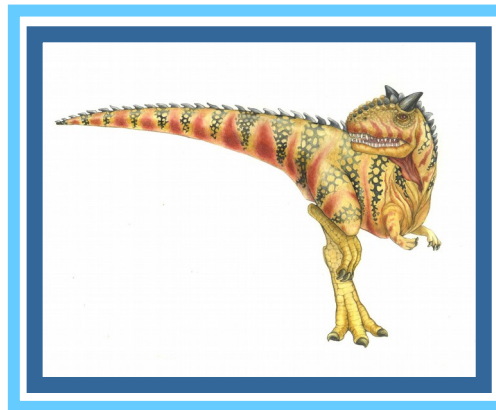
- OpenMP is a set of compiler directives and API that support parallel programming.

```
void update(int value)
{
    #pragma omp critical
    {
        count += value
    }
}
```

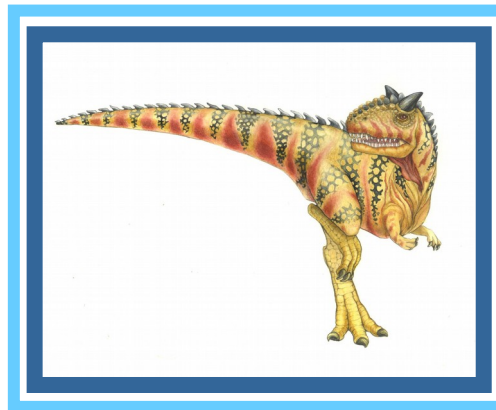
The code contained within the **#pragma omp critical** directive is treated as a critical section and performed atomically.



End of Chapter 7



Chapter 8: Deadlocks





Chapter 8: Deadlocks

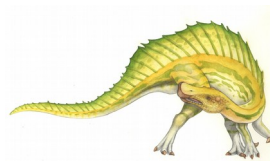
- System Model
- Deadlock in Multi-threaded Applications
- Deadlock Characterization
- Methods for Handling Deadlocks
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection
- Recovery from Deadlock





Chapter Objectives

- Illustrate how deadlock can occur when mutex locks are used
- Define the four necessary conditions that characterize deadlock
- Identify a deadlock situation in a resource allocation graph
- Evaluate the four different approaches for preventing deadlocks
- Apply the banker's algorithm for deadlock avoidance
- Apply the deadlock detection algorithm
- Evaluate approaches for recovering from deadlock





System Model

- System consists of a collection of m resource types and n processes
 - Resource types $R_1, R_2, \dots, R_m$
CPU cycles, memory space, I/O devices
- Each resource type R_i has W_i instances.
- Each process utilizes a resource by executing a sequence of three actions:
 - **request it**
 - **use it**
 - **release it**





Deadlock in Multithreaded Application

- Two mutex locks are created and initialized:

```
pthread_mutex_t first_mutex;  
pthread_mutex_t second_mutex;  
  
pthread_mutex_init(&first_mutex, NULL);  
pthread_mutex_init(&second_mutex, NULL);
```





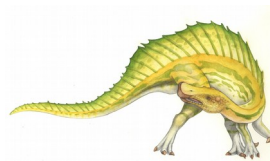
Deadlock in Multithreaded Application

```
/* thread_one runs in this function */
void *do_work_one(void *param)
{
    pthread_mutex_lock(&first_mutex);
    pthread_mutex_lock(&second_mutex);
    /**
     * Do some work
     */
    pthread_mutex_unlock(&second_mutex);
    pthread_mutex_unlock(&first_mutex);

    pthread_exit(0);
}

/* thread_two runs in this function */
void *do_work_two(void *param)
{
    pthread_mutex_lock(&second_mutex);
    pthread_mutex_lock(&first_mutex);
    /**
     * Do some work
     */
    pthread_mutex_unlock(&first_mutex);
    pthread_mutex_unlock(&second_mutex);

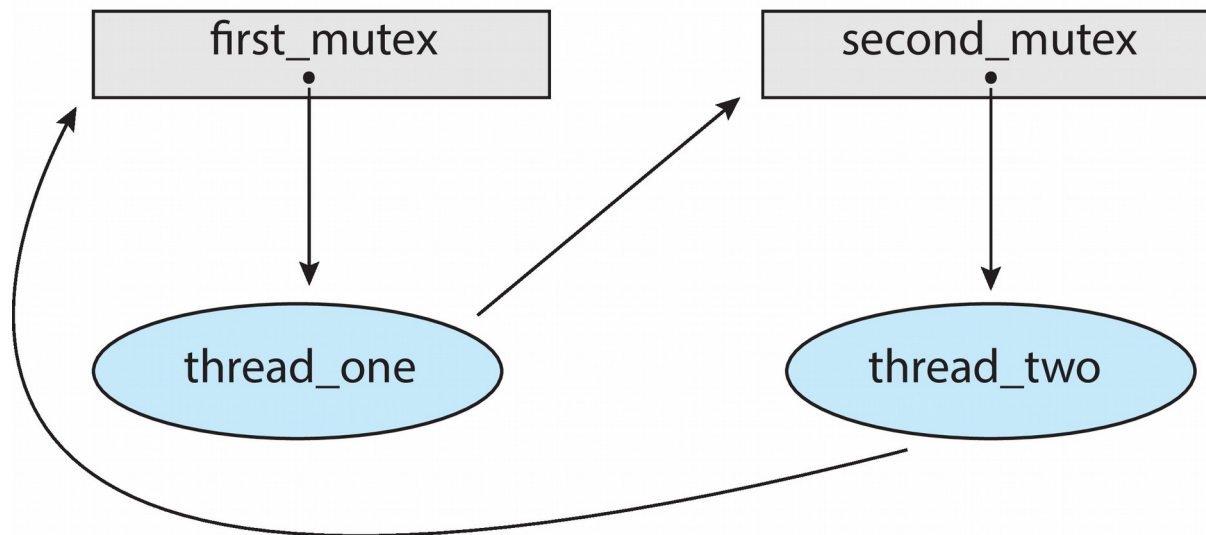
    pthread_exit(0);
}
```





Deadlock in Multithreaded Application

- Deadlock is possible if thread 1 acquires **first_mutex** and thread 2 acquires **second_mutex**. Thread 1 then waits for **second_mutex** and thread 2 waits for **first_mutex**.
- Can be illustrated with a **resource allocation graph**:





Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** only one process at a time can use a resource
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task
- **Circular wait:** there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .

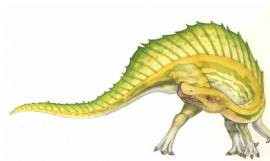




Resource-Allocation Graph

A resource allocation graph is a set of vertices V and a set of edges E such that:

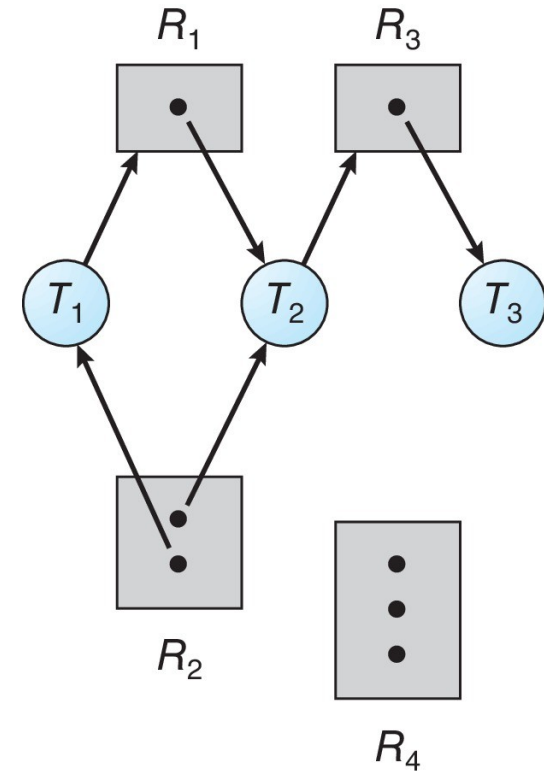
- V is partitioned into two types:
 - $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system
- **request edge** – directed edge $P_i \rightarrow R_j$ indicates P_i has requested a unit of R_j
- **assignment edge** – directed edge $R_j \rightarrow P_i$





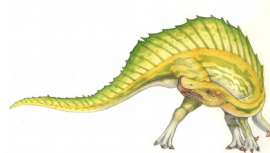
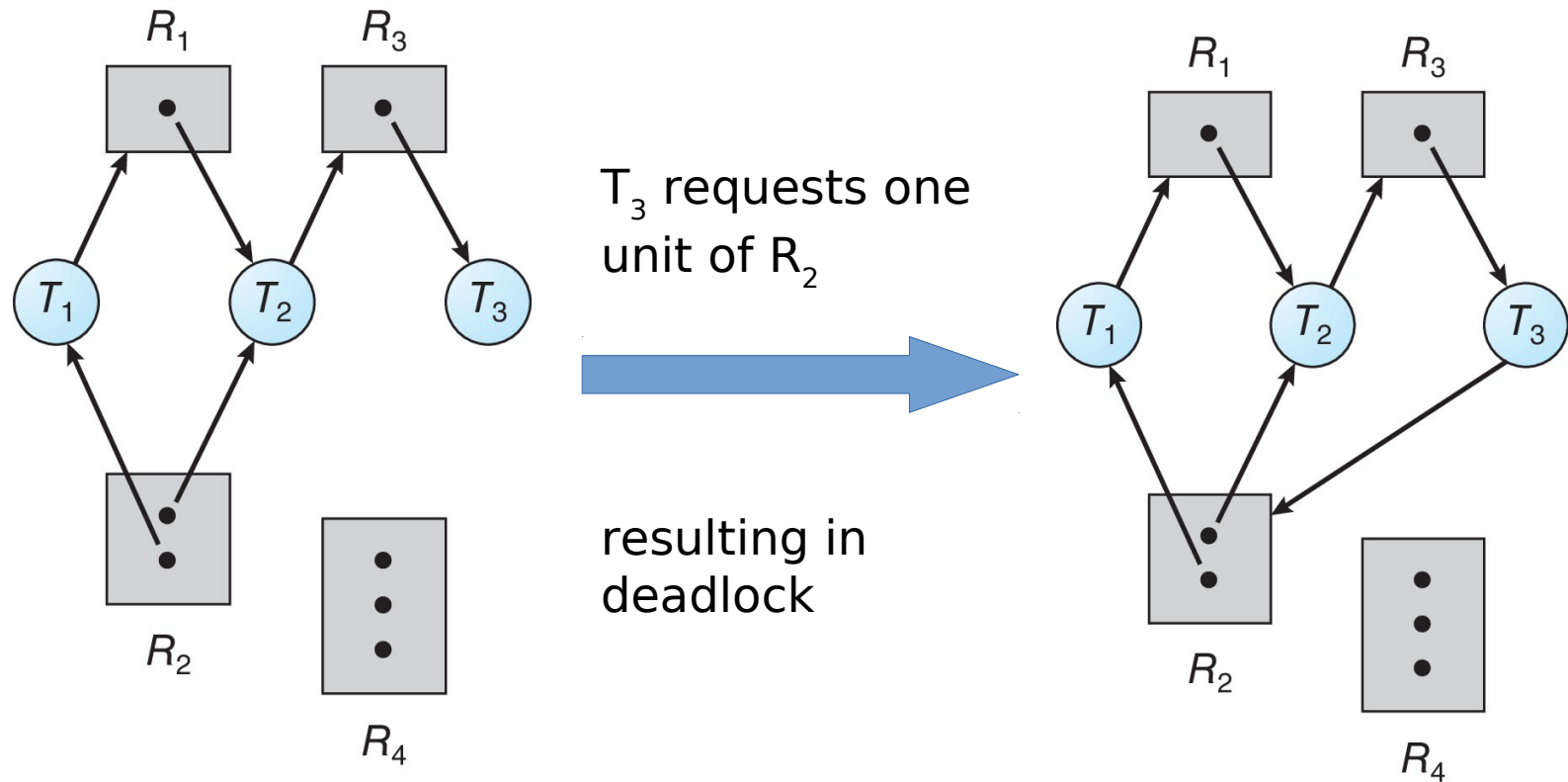
Resource Allocation Graph Example

- One instance of R1
- Two instances of R2
- One instance of R3
- Three instance of R4
- T1 holds one instance of R2 and is waiting for an instance of R1
- T2 holds one instance of R1, one instance of R2, and is waiting for an instance of R3
- T3 is holds one instance of R3





Resource Allocation Graph With A Deadlock

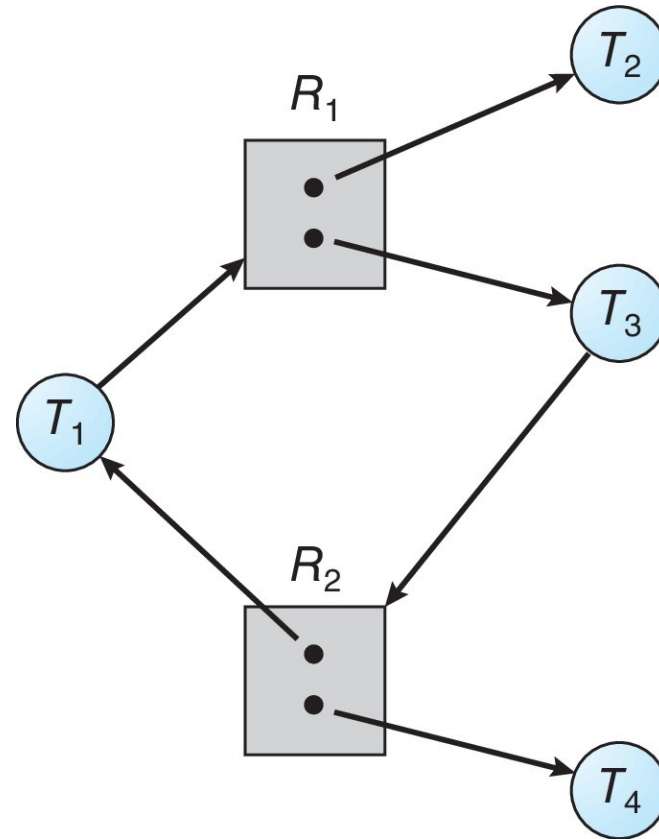




Graph With A Cycle But No Deadlock

Although this graph has a cycle, it is not the graph of a deadlock state.

T_2 holds a unit of R_1 and is not waiting for another resource, so it can release it when it is finished with it and T_1 can acquire the unit.





Resource Allocation Graph Facts

- If graph contains no cycles $\Rightarrow$ no deadlock
- If graph contains a cycle $\Rightarrow$
 - If each resource type has just a single instance of the resource, then a cycle implies deadlock
 - If there is at least one resource type that has multiple instances, a cycle does not necessarily imply deadlock.





Methods for Handling Deadlocks

- Ensure that the system will *never* enter a deadlock state:
 - Deadlock prevention
 - Deadlock avoidance
- Allow the system to enter a deadlock state and then recover
- Ignore the problem and pretend that deadlocks never occur in the system.





Deadlock Prevention

Invalidate one of the four necessary conditions for deadlock:

- **Mutual Exclusion** – not required for shareable resources (e.g., read-only files); must hold for non-shareable resources. Cannot remove mutual exclusion for reusable resources so this is not an option.
- **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources. Two choices:
 1. Require process to request and be allocated **all of its resources before it begins execution.**
 2. Allow process to request resources **only when the process has none allocated to it.**
 - Poor resource utilization; starvation possible





Deadlock Prevention (Cont.)

- **No Preemption** – forcibly taking resources away from processes.
 - If a process that is holding some resources requests another resource that cannot be allocated immediately to it, then all resources currently being held by that process are released.
 - Preempted resources are added to the list of resources for which the process is waiting
 - Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting.
 - On request, if unavailable, we preempt resources from a process that holds them if that process is in a waiting state; if the requested resources are not available or held only by ready processes, the requesting process waits. Process may lose resources while it waits. Process gets restarted only when it is allocated new resources and recovers the preempted ones.





Deadlock Prevention (Cont.)

- **Circular Wait** – impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration.
 - Proposed by Havender:
 - ▶ A process can request resources only in increasing order of its resource number.
 - ▶ If process holds R_i it can only request R_j if $j > i$.
 - ▶ If process wants R_j , it must first release all R_i such that $i \geq j$.
 - ▶ If a process violates these rules, it is terminated.
 - This makes circular waiting impossible, so deadlock impossible.
 - But results in poor resource utilization.



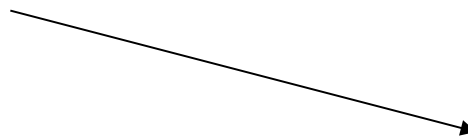


Circular Wait

- Invalidating the circular wait condition is most common.
- Simply assign each resource (i.e. mutex locks) a unique number.
- Resources must be acquired in order.
- If:

first_mutex = 1
second_mutex = 5

code for **thread_two** could not be written as follows:



```
/* thread.one runs in this function */
void *do_work_one(void *param)
{
    pthread_mutex_lock(&first_mutex);
    pthread_mutex_lock(&second_mutex);
    /**
     * Do some work
     */
    pthread_mutex_unlock(&second_mutex);
    pthread_mutex_unlock(&first_mutex);

    pthread_exit(0);
}

/* thread.two runs in this function */
void *do_work_two(void *param)
{
    pthread_mutex_lock(&second_mutex);
    pthread_mutex_lock(&first_mutex);
    /**
     * Do some work
     */
    pthread_mutex_unlock(&first_mutex);
    pthread_mutex_unlock(&second_mutex);

    pthread_exit(0);
}
```





Deadlock Avoidance

- In deadlock prevention schemes, requests are constrained resulting in poor utilization and throughput.
- Alternative is to use additional information to allow systems to decide when and whether to grant requests. This is called **deadlock avoidance**.
- Avoidance is a dynamic strategy.
- Requires that each process's **maximum claim for each resource type is known in advance**. A request is granted only if the resulting system state is a **safe state**, meaning that there is a way to avoid deadlock in this state while still continuing to allocate resources to each process.

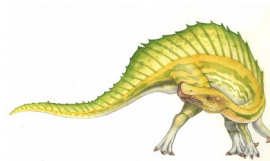




Deadlock Avoidance

Requires that the system has some additional ***a priori*** information available

- Simplest and most useful model requires that each process declare the ***maximum number*** of resources of each type that it may need
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition
- Resource-allocation ***state*** is defined by the number of available and allocated resources, and the maximum demands of the processes





Safe State

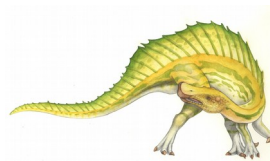
- When a process requests an available resource, system must decide if immediate allocation leaves the system in a safe state
- System is in **safe state** if there exists a **sequence** $\langle P_1, P_2, \dots, P_n \rangle$ of **ALL** the processes in the systems such that for each P_i , the resources that P_i can still request can be satisfied by currently available resources + resources held by all the P_j , with $j < i$
- That is:
 - If P_i 's resource needs are not immediately available, then P_i can wait until all P_j have finished
 - When P_j is finished, P_i can obtain needed resources, execute, return allocated resources, and terminate
 - When P_i terminates, P_{i+1} can obtain its needed resources, and so on





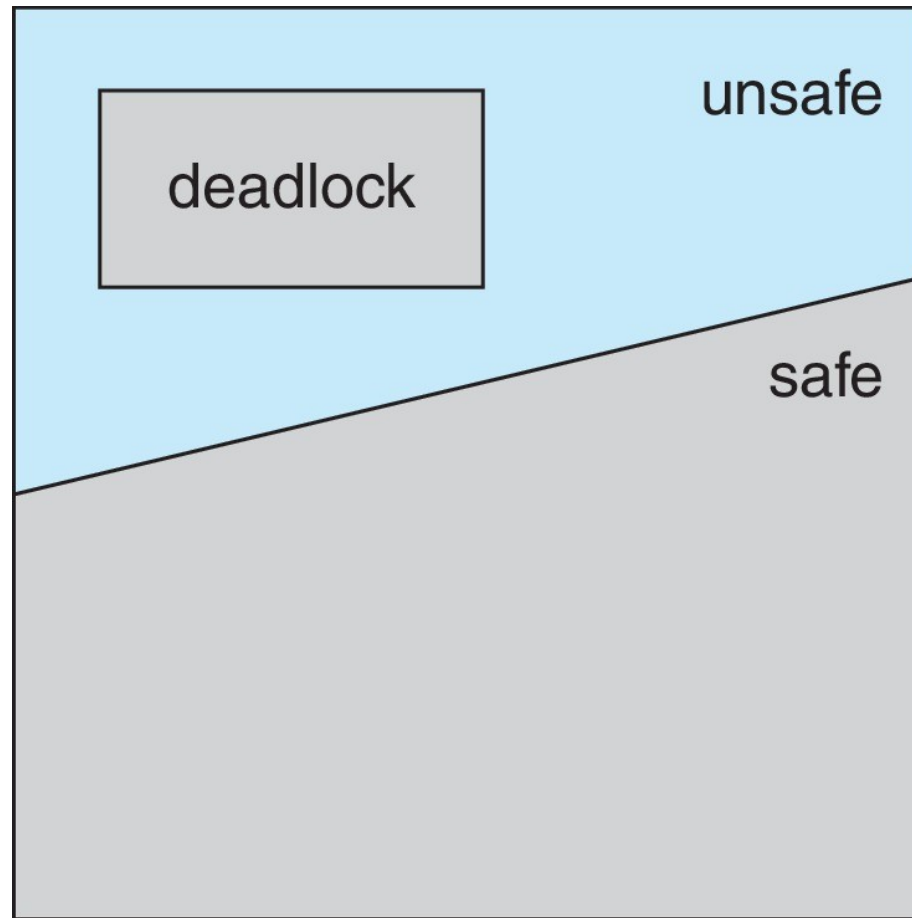
Safe States and Unsafe States

- If a system is in safe state $\Rightarrow$ it cannot be a deadlock state, i.e., there is no deadlock in the current state.
- If a system is in unsafe state $\Rightarrow$ possibility of deadlock – it may not be possible to avoid deadlock.
- Avoidance algorithm $\Rightarrow$ ensure that a system will never enter an unsafe state.





Safe, Unsafe, Deadlock State





Avoidance Algorithms

- Single instance of a resource type
 - Use a resource-allocation graph

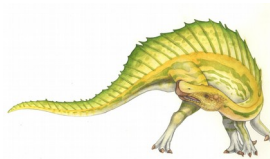
- Multiple instances of a resource type
 - Use the Banker's Algorithm





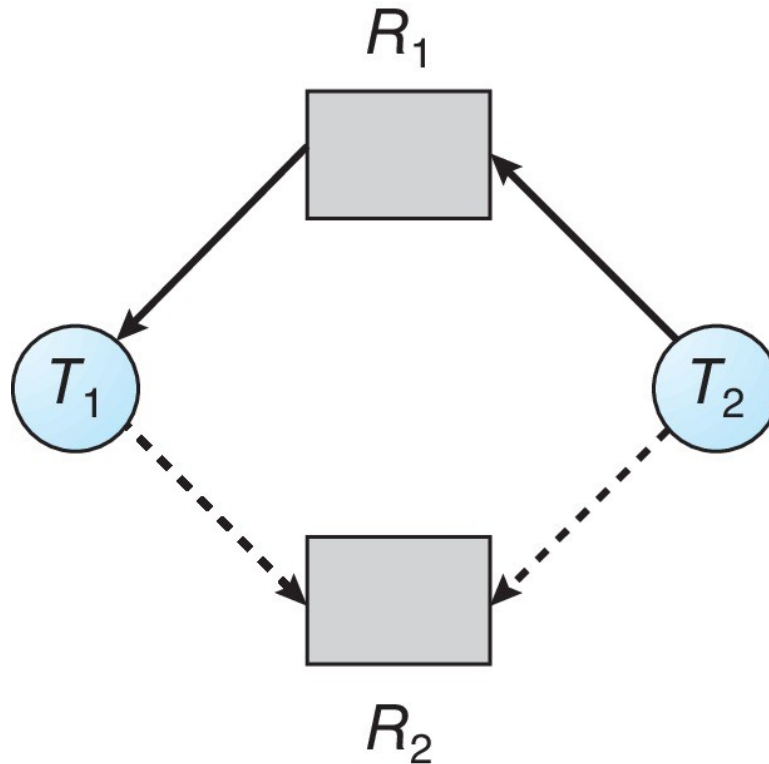
Resource-Allocation Graph Scheme

- **Claim edge** $P_i \rightarrow R_j$ indicated that process P_i may request resource R_j ; represented by a dashed line
- Claim edge converts to request edge when a process requests a resource
- Request edge converted to an assignment edge when the resource is allocated to the process
- When a resource is released by a process, assignment edge reconverts to a claim edge
- Resources must be claimed *a priori* in the system



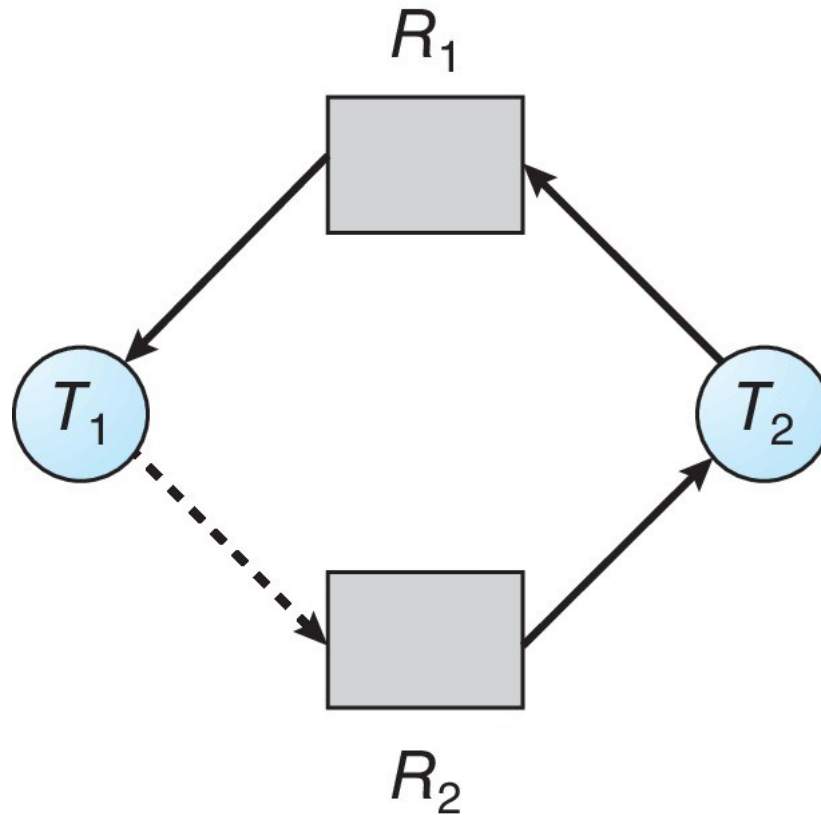


Resource-Allocation Graph





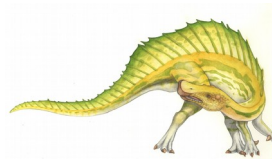
Unsafe State In Resource-Allocation Graph





Resource-Allocation Graph Algorithm

- Suppose that process P_i requests a resource R_j
- The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph





Banker's Algorithm

- Multiple instances of resources
- Each process must a priori claim maximum use
- When a process requests a resource it may have to wait
- When a process gets all its resources it must return them in a finite amount of time



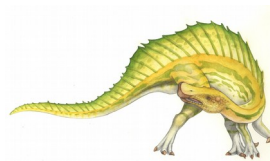


Data Structures for the Banker's Algorithm

Let n = number of processes, and m = number of resources types.

- **Available:** Vector of length m . If available $[j] = k$, there are k instances of resource type R_j available
- **Max:** $n \times m$ matrix. If $Max[i, j] = k$, then process P_i may hold at most k instances of resource type R_j
- **Allocation:** $n \times m$ matrix. If $Allocation[i, j] = k$ then P_i is currently allocated k instances of R_j
- **Need:** $n \times m$ matrix. If $Need[i, j] = k$, then P_i may need k more instances of R_j to complete its task

$$Need[i, j] = Max[i, j] - Allocation[i, j]$$





Safety Algorithm

1. Let **Work** and **Finish** be vectors of length m and n , respectively.
Initialize:

Work = **Available**

Finish [i] = **false** for $i = 0, 1, \dots, n-1$

2. Find an i such that both:

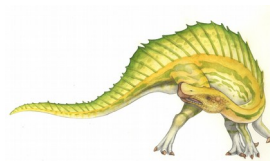
(a) **Finish** [i] = **false**

(b) **Need** _{i} ≤ **Work**

If no such i exists, go to step 4

3. **Work** = **Work** + **Allocation** _{i}
Finish [i] = **true**
go to step 2

4. If **Finish** [i] == **true** for all i , then the system is in a safe state





Resource-Request Algorithm for Process P_i

$Request_i$ = request vector for process P_i . If **$Request_i[j] = k$** then process P_i wants k instances of resource type R_j

1. If **$Request_i \leq Need_i$** go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If **$Request_i \leq Available$** , go to step 3. Otherwise P_i must wait, since resources are not available
3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$$Available = Available - Request_i;$$

$$Allocation_i = Allocation_i + Request_i;$$

$$Need_i = Need_i - Request_i;$$

- If safe $\Rightarrow$ the resources are allocated to P_i
- If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored





Example of Banker's Algorithm

- 5 processes P_0 through P_4 ;

3 resource types:

A (10 instances), B (5 instances), and C (7 instances)

- Snapshot at time T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	$A \ B \ C$	$A \ B \ C$	$A \ B \ C$
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	



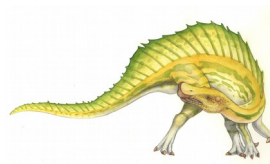


Example (Cont.)

- The content of the matrix ***Need*** is defined to be ***Max – Allocation***

	<u><i>Need</i></u>		
	<i>A</i>	<i>B</i>	<i>C</i>
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

- The system is in a safe state since the sequence $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisfies safety criteria



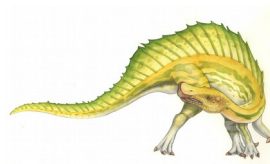


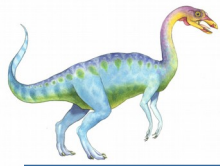
Example: P_1 Request (1,0,2)

- Check that Request $\leq$ Available (that is, $(1,0,2) \leq (3,3,2) \Rightarrow \text{true}$)

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- Executing safety algorithm shows that sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisfies safety requirement
- Can request for (3,3,0) by P_4 be granted?
- Can request for (0,2,0) by P_0 be granted?





Deadlock Detection

- Allow system to enter deadlock state
- Detection algorithm
- Recovery scheme





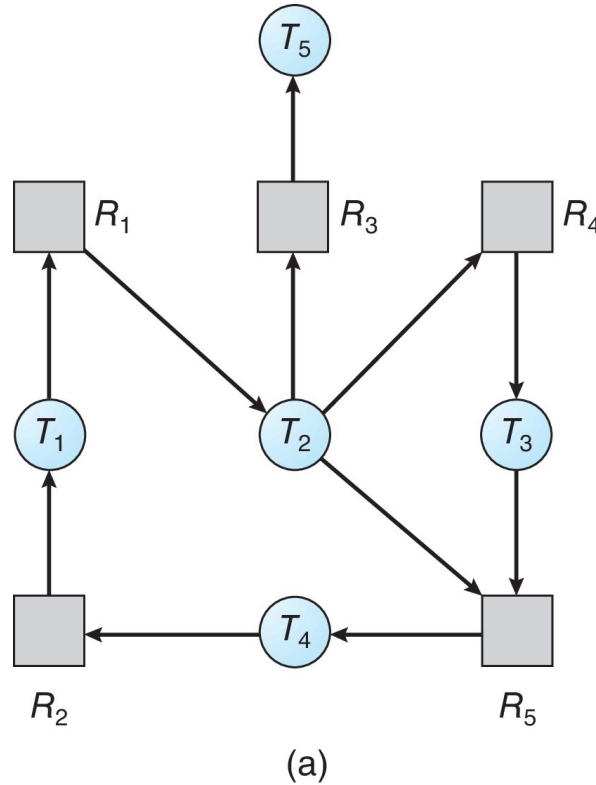
Single Instance of Each Resource Type

- Maintain **wait-for** graph
 - Nodes are processes
 - $P_i \rightarrow P_j$ if P_i is waiting for P_j
- Periodically invoke an algorithm that searches for a cycle in the graph. If there is a cycle, there exists a deadlock
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph

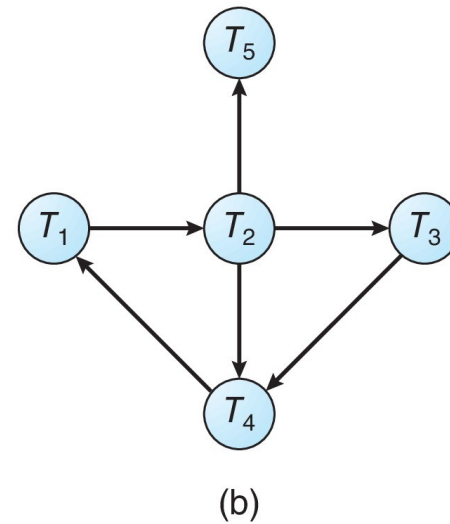




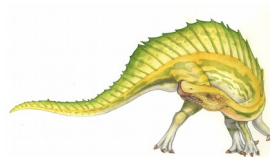
Resource-Allocation Graph and Wait-for Graph



Resource-Allocation Graph



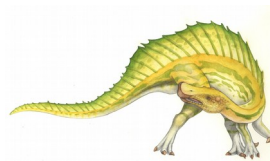
Corresponding wait-for graph





Several Instances of a Resource Type

- **Available:** A vector of length m indicates the number of available resources of each type
- **Allocation:** An $n \times m$ matrix defines the number of resources of each type currently allocated to each process
- **Request:** An $n \times m$ matrix indicates the current request of each process. If $\mathbf{Request}[i][j] = k$, then process P_i is requesting k more instances of resource type R_j .





Detection Algorithm

1. Let **Work** and **Finish** be vectors of length **m** and **n**, respectively
Initialize:
 - (a) **Work = Available**
 - (b) For **i = 1, 2, ..., n**, if **Allocation_i ≠ 0**, then
Finish[i] = false; otherwise, **Finish[i] = true**
2. Find an index **i** such that both:
 - (a) **Finish[i] == false**
 - (b) **Request_i ≤ Work**

If no such **i** exists, go to step 4

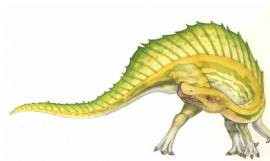




Detection Algorithm (Cont.)

3. **$Work = Work + Allocation_i$**
 $Finish[i] = true$
go to step 2
4. If **$Finish[i] == false$** , for some i , $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if **$Finish[i] == false$** , then P_i is deadlocked

Algorithm requires an order of $O(m \times n^2)$ operations to detect whether the system is in deadlocked state





Example of Detection Algorithm

- Five processes P_0 through P_4 ; three resource types A (7 instances), B (2 instances), and C (6 instances)
- Snapshot at time T_0 :

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

- Sequence $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ will result in ***Finish[i] = true*** for all i





Example (Cont.)

- P_2 requests an additional instance of type **C**

	<u>Request</u>		
	A	B	C
P_0	0	0	0
P_1	2	0	2
P_2	0	0	1
P_3	1	0	0
P_4	0	0	2

- State of system?
 - Can reclaim resources held by process P_0 , but insufficient resources to fulfill other processes; requests
 - Deadlock exists, consisting of processes P_1 , P_2 , P_3 , and P_4





Detection-Algorithm Usage

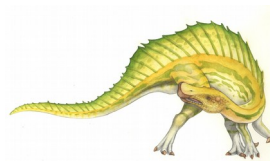
- When, and how often, to invoke depends on:
 - How often a deadlock is likely to occur?
 - How many processes will need to be rolled back?
 - ▶ one for each disjoint cycle
- If detection algorithm is invoked arbitrarily, there may be many cycles in the resource graph and so we would not be able to tell which of the many deadlocked processes “caused” the deadlock.





Recovery from Deadlock: Process Termination

- Abort all deadlocked processes
- Abort one process at a time until the deadlock cycle is eliminated
- In which order should we choose to abort?
 1. Priority of the process
 2. How long process has computed, and how much longer to completion
 3. Resources the process has used
 4. Resources process needs to complete
 5. How many processes will need to be terminated
 6. Is process interactive or batch?





Recovery from Deadlock: Resource Preemption

- **Selecting a victim** – minimize cost
- **Rollback** – return to some safe state, restart process for that state
- **Starvation** – same process may always be picked as victim, include number of rollback in cost factor



End of Chapter 8

